STAT 547 - Data Mining [1]
Homework 3

Rochester Institute of Technology
Ducheng Tan

April 12, 2026

Theoretical Foundations

Consider the classical statistical supervised learning task with a given data set

$$\mathscr{D}_n = \{(\vec{x_i}, y_i \sim p_{xy}(x,y), \vec{x_i} \in \mathscr{X}, y_i \in \mathscr{Y}, i \in [n]\} \tag{1}$$

from some unknown joint probability density function $p_{xy}(x,y)$.

Part A. Support Vector Machines

Consider the soft-margin SVM optimization problem for binary classification with class labels in $\{-1,+1\}$ formulated as follows

$$\min_{\vec{w},b,\xi} \frac{1}{2}||\vec{w}||^2 + C\sum_{i=1}^{n}\xi_i \tag{2}$$

subject to

$$y_i(\vec{w}^T\vec{x}_i + b) \geq 1 - \xi, \quad \xi_i \geq 0, \quad i \in [n] \tag{3}$$

The Primal-Dual Gap and a "Broken" SVM

A student, Alice, is implementing a Soft-Margin SVM solver. She derives the Lagrangian and the KKT conditions correctly. However, due to a bug, her solver fails to enforce the constraint $\alpha_i \leq C$ for the Lagrange multipliers in the dual problem. Her "broken" dual problem is

$$\hat{\vec{\alpha}} = \arg\max_{\alpha \in \mathbb{R}_+^n} \left\{ \sum_{1\leq i\leq n} \alpha_i - \frac{1}{2}\sum_{1\leq i,j\leq n} \alpha_i\alpha_j y_i y_j \vec{x_i}^T\vec{x}_j \right\} \tag{4}$$

subject to $\sum_{1\leq i\leq n}\alpha_i y_i = 0$ and $\alpha_i \geq 0$ for $i \in [n]$.

(a) Clearly specify and write down what is missing in Alice's dual formulation.

Alice is missing the upper bound constraint on α_i, it would be complete to change the constraint to $0 \leq \alpha_i \leq C$. A detailed process is shown in Homework 2 on equation line (49) from the $\nabla\mathcal{L} = 0$ equations. In particular

$$\frac{\partial\mathcal{L}}{\partial\xi_i} = C - \alpha_i - \lambda_i = 0 \Rightarrow C = \alpha_i + \lambda_i \tag{5}$$

Since the λ_i is the multiplier from the constrain for minimizing $C\sum_{i=1}^{n}\xi_i$ with constraint $\xi_i \geq 0$, then it must be that $\lambda_i \geq 0$ so it is clear that $C \geq \alpha_i$ is the upper bound.

(b) Explain the geometric interpretation of the primal SVM objective: to find a hyperplane that maximizes the margin while allowing for slack variables ξ_i. How does the parameter C control the trade-off in this geometric view?

Note that in minimizing the objective in (2) for each $x_i \in \mathcal{X}, \exists\xi_i$, such that $\xi_i = (0, 1 - y_i(\vec{w}^T x_i + b))_+$ this is because the ξ_i are the complement to the value of $y_i(\vec{w}^T\vec{x}_i + b)$. In other words the ξ_i make up for the error to the the original classifier. We can write the constraint of classifying $\vec{x}_i$ correctly as

$$y_i(\vec{w}^T\vec{x}_i + b) + \xi_i \geq 1 \tag{6}$$

So it follows that if we were to have a smaller C then to minimize $||\vec{w}||^2/2 + C\sum\xi_i$ simply means that each $\xi_i, ||\vec{w}||$ is allowed to be larger. This will directly lead to a narrower margin as $2/||w||$ is smaller. The C values can be thought of

as a violation cost, for larger C values the cost of violation is high thus, making the $\xi_i, ||\vec{w}||$ smaller, meaning we again approach the hard margin condition of classifying everything correctly which means that $||w||^2$ will be smaller, meaning that the width of the margins $2/||w||$ will be nearly as big as it can from the hard margin conditions.

(c) Now, consider Alice's broken dual. Assume the data is linearly separable. What will happen to the optimal α_i for a support vector in this broken formulation as the optimization runs ? What does this imply for the resulting weight vector $\vec{w} = \sum \alpha_i y_i \vec{x}_i$?

Assuming that the data is linearly separable, then this broken formulation as the optimization runs, we will approach back to the hard margin problem without the ξ_i and the with of the margin will be the length of the two closest points in two different classes.

(d) Connect your answer in (b) to the KKT condition involving the slack variable ξ_i and the multiplier μ_i (where $\mu_i = C - \alpha_i$ in the correct formulation). Explain why the missing constraints $\alpha_i \leq C$ breaks the model's ability to handle outliers and misclassified points.

Say that there are outliers with true positive class, however since that data is an outlier is it possible that SVM without the constraint will allow such outlier to be misclassified or lie within the separating margins. Having the slack variable ξ_i will allow the margins to be narrower, however allowing those outliers and misclassified points to be classified correctly.

The Kernel Trick: From Theory to Practice

(a) Prove that the prediction function for a kernelized SVM,

$$\hat{f}(\vec{x}_{new}) = \text{sign}\left(\sum_{i \in SV} \alpha_i y_i K(\vec{x}_i, \vec{x}_{new}) + b\right) \tag{7}$$

can be computed without ever explicitly calculated $\phi(\vec{x})$ for any point, where ϕ is the feature map corresponding to the kernel K. *Proof is assisted after some reading on Hilbert Spaces* [2]

Proof. Recall the *Reproducing Kernel Hilbert Space*, this is a special Hilbert space built for the kernel function K such that it acts as the inner product for the feature map ϕ.

Let $\mathcal{H}_k$ be the Reproducing Kernel Hilbert Space (RKHS) associated with kernel $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$, with feature map $\phi(\vec{x}) = K(\cdot, \vec{x})$. The SVM seeks for a function $f^* \in \mathcal{H}_k$ solving

$$f^* = \arg\min_{f \in \mathcal{H}_k} \left\{ \sum_{1 \leq i \leq n} \ell\big(f(\vec{x}_i), y_i\big) + C_0 \|f\|^2_{\mathcal{H}_k} \right\} \tag{8}$$

Suppose we can decompose any $f \in \mathcal{H}_k$ as $f = f_s + f_\perp$, where $f_s \in \text{span}\{K(\vec{x}_i, \cdot)\}_{i=1}^n$ and $f_\perp \perp \text{span}\{K(\vec{x}_i, \cdot)\}_{i=1}^n$. The above sets up a "right angle" in $\mathcal{H}_K$, the it follows that by the Pythagorean identity in $\mathcal{H}_k$

$$\|f\|^2_{\mathcal{H}_k} = \|f_s\|^2_{\mathcal{H}_k} + \|f_\perp\|^2_{\mathcal{H}_k} \geq \|f_s\|^2_{\mathcal{H}_k} \tag{9}$$

By the reproducing property $f(\vec{x}) = \langle K(\cdot, \vec{x}),\ f \rangle_{\mathcal{H}_k}$, and since $f_\perp$ is orthogonal to every $K(\vec{x}_i, \cdot)$:

$$f(\vec{x}_i) = \langle K(\cdot, \vec{x}_i),\ f_s + f_\perp \rangle_{\mathcal{H}_k} = \langle K(\cdot, \vec{x}_i),\ f_s \rangle_{\mathcal{H}_k} = f_s(\vec{x}_i) \tag{10}$$

So $f_\perp$ does not affect the loss but strictly increases the regulariser in (8). Setting $f_\perp = 0$ is the optimal choice, and the minimiser $f^* \in \text{span}\{K(\vec{x}_i, \cdot)\}_{i=1}^n$ and must explicitly take the form

$$f^* \in \text{span}\{K(\vec{x}_i, \cdot) | i \in [n]\} = \left\{ \sum_{1 \leq i \leq n} \alpha_i\, K(\vec{x}_i, \cdot), \quad \alpha_i \in \mathbb{R} \right\} \tag{11}$$

$$f^*(\cdot) = \sum_{i=1}^n \alpha_i\, K(\vec{x}_i, \cdot), \quad \alpha_i \in \mathbb{R} \tag{12}$$

Substituting (12) into (8), the SVM dual problem becomes

$$\min_{\boldsymbol{\alpha} \in \mathbb{R}^n} \sum_{i=1}^n \ell\left(\sum_{j=1}^n \alpha_j\, K(x_j, x_i),\ y_i\right) + C_0 \left\| \sum_{i=1}^n \alpha_i\, K(x_i, \cdot) \right\|^2_{\mathcal{H}_k} \tag{13}$$

We simplify the regulariser using bilinearity of the inner product and the reproducing property. For $f^*(\cdot) = \sum_{i=1}^n \alpha_i K(\vec{x}_i, \cdot)$

$$\left\| \sum_{i=1}^n \alpha_i \, K(\vec{x}_i, \cdot) \right\|_{\mathcal{H}_k}^2 = \left\langle \sum_{i=1}^n \alpha_i K(\vec{x}_i, \cdot), \sum_{j=1}^n \alpha_j K(\vec{x}_j, \cdot) \right\rangle_{\mathcal{H}_k} = \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j K(\vec{x}_i, \vec{x}_j) = \boldsymbol{\alpha}^\top \mathbf{K} \boldsymbol{\alpha} \tag{14}$$

where $\mathbf{K} \in \mathbb{R}^{n \times n}$ is the Gram matrix with entries $\mathbf{K}_{ij} = K(\vec{x}_i, \vec{x}_j)$, and $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_n)^\top \in \mathbb{R}^n$. Similarly, evaluating f^* at each training point $\vec{x}_i$ via the reproducing property yields

$$f^*(\vec{x}_i) = \sum_{j=1}^n \alpha_j \, K(\vec{x}_j, \vec{x}_i) = (\mathbf{K}\boldsymbol{\alpha})_i \tag{15}$$

so the loss depends only on $\mathbf{K}\boldsymbol{\alpha}$. This result is beautiful because the infinite-dimensional optimisation over $f \in \mathcal{H}_k$ therefore collapses to the finite-dimensional problem

$$\min_{\boldsymbol{\alpha} \in \mathbb{R}^n} \sum_{i=1}^n \ell\left((\mathbf{K}\boldsymbol{\alpha})_i, \; y_i\right) + C_0 \, \boldsymbol{\alpha}^\top \mathbf{K} \boldsymbol{\alpha} \tag{16}$$

which depends only on the Gram matrix $\mathbf{K}$. So we can say that the feature map ϕ does not appear at all in this computation. For the SVM hinge loss $\ell(f(\vec{x}_i), y_i) = (0, 1 - y_i f(\vec{x}_i))_+$, solving (16) via its Lagrangian yields the dual problem

$$\max_{\boldsymbol{\alpha} \in \mathbb{R}^n} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \, K(\vec{x}_i, \vec{x}_j) \quad \text{s.t.} \quad 0 \le \alpha_i \le C_0, \;\; \sum_{i=1}^n \alpha_i y_i = 0 \tag{17}$$

At best, most data points will be correctly classified and lies outside the margin so $y_i f^* > 1 \to \alpha_i = 0$. Finally, evaluating f^* at a new point $\vec{x}_{\text{new}} \in \mathscr{X}$ via the reproducing property in (10) twice yields

$$\begin{aligned} f^*(\vec{x}_{\text{new}}) = \langle f^*, K(\cdot, \vec{x}_{\text{new}}) \rangle_{\mathcal{H}_k} &= \left\langle \sum_{1 \le i \le n} \alpha_i K(\vec{x}_i, \cdot), K(\cdot, \vec{x}_{\text{new}}) \right\rangle \\ &= \sum_{1 \le i \le n} \alpha_i \, \langle K(\vec{x}_i, \cdot), K(\cdot, \vec{x}_{new}) \rangle = \sum_{i \in SV} \alpha_i \, K(\vec{x}_i, \vec{x}_{\text{new}}) \end{aligned} \tag{18}$$

and the prediction function is

$$\hat{f}(\vec{x}_{\text{new}}) = \text{sign}\left(\sum_{i \in SV} \alpha_i \, y_i \, K(\vec{x}_i, \vec{x}_{\text{new}}) + b \right) \tag{19}$$

Each term $K(\vec{x}_i, \vec{x}_{\text{new}})$ is a direct evaluation of the kernel function on elements of $\mathscr{X}$. There was no explicit computation of $\phi(\vec{x})$ required. □

(b) Consider a custom kernel function defined as

$$K(\vec{u}, \vec{v}) = (1 + \vec{u}^T \vec{v})^3 \tag{20}$$

(i) What is the dimensionality of the implicit feature space $\phi(\vec{u})$ for this kernel? Justify your answer.

Since the dimension of the vectors $\vec{u}, \vec{v}$ was not given. We will assume the general case $\vec{u}, \vec{v} \in \mathbb{R}^p$. For each of the $(\vec{u}^T \vec{v})^n = \left(\sum_{1 \le i \le p} u_i v_i \right)^n$ is a multinomial expansion from the classic stars and bars problem.

$$(x_1 + x_2 + \cdots + x_m)^n = \sum_{k_1 + k_2 + \cdots + k_m = n} \binom{n}{k_1, k_2, \ldots, k_m} \prod_{i=1}^m x_i^{k_i} \tag{21}$$

As I personally am not that familiar with Combinatorics, a proof will not be provided and we use the consequence that there is $\binom{n+m-1}{m-1}$ terms in the expansion in (21). From (20), This yields

$$\binom{3 + (p+1) - 1}{p} = \binom{p+3}{p} = \binom{p+3}{3} = \frac{(p+3)!}{p!3!} = \frac{1}{6}(p+3)(p+2)(p+1) \tag{22}$$

terms in the expansion. Then $\dim(\phi(\vec{u})) = \frac{1}{6}(p+3)(p+2)(p+1)$

(ii) Write out the explicit formula from one specific component of the feature vector $\phi(\vec{u})$ from a 2-dimensional input $\vec{u} = (u_1, u_2)$. This demonstrates you understand the relationship between the kernel and the feature map.

Using the results in (22) for $p = 2$ in this case, we expect 10 terms in the expansion. Using the trinomial expansion for when $m = 3$ in (22).

$$\begin{aligned} K(\vec{u},\vec{v}) &= \sum_{\substack{k_0+k_1+k_2=3 \\ k_0,k_1,k_2\geq 0}} \binom{3}{k_0,k_1,k_2} (u_1v_1)^{k_1}(u_2v_2)^{k_2} \\ K(\vec{u},\vec{v}) &= 1 + 3u_1v_1 + 3u_2v_2 + 3u_1^2v_1^2 + 6u_1u_2v_1v_2 + 3u_2^2v_2^2 \\ &\quad + u_1^3v_1^3 + 3u_1^2u_2v_1^2v_2 + 3u_1u_2^2v_1v_2^2 + u_2^3v_2^3 \\ &= \phi(\vec{u})^T\phi(\vec{v}) \end{aligned} \tag{23}$$

Taking the square root on each term's coefficients, we obtain

$$\phi(\vec{u}) = \begin{pmatrix} 1 \\ \sqrt{3}\,u_1 \\ \sqrt{3}\,u_2 \\ \sqrt{3}\,u_1^2 \\ \sqrt{6}\,u_1u_2 \\ \sqrt{3}\,u_2^2 \\ u_1^3 \\ \sqrt{3}\,u_1^2u_2 \\ \sqrt{3}\,u_1u_2^2 \\ u_2^3 \end{pmatrix} \tag{24}$$

(c) A common claim is that "the RBF kernel can map data to an infinite dimensional feature space." Explain intuitively why this is both a great strength and a significant danger, linking your answer to the concept of model complexity and the bias-variance trade-off.

Recall that the RBF kernel is defined as

$$K(\vec{u},\vec{v}) = \exp\left(-\frac{||\vec{u}-\vec{v}||^2}{2\sigma^2}\right) \tag{25}$$

Notice we can decompose the exponent as

$$\|\vec{u}-\vec{v}\|^2 = \|\vec{u}\|^2 - 2\vec{u}^T\vec{v} + \|\vec{v}\|^2 \tag{26}$$

We can rewrite this using Taylor expansion of the exponential function

$$K(\vec{u},\vec{v}) = \exp\left(-\frac{\|\vec{u}\|^2}{2\sigma^2}\right)\exp\left(-\frac{\|\vec{v}\|^2}{2\sigma^2}\right)\sum_{n=0}^{\infty}\frac{\left(\vec{u}^T\vec{v}\right)^n}{n!\,\sigma^{2n}} \tag{27}$$

This means that we can always write $\phi(\vec{u})$ as

$$\phi(\vec{u}) = \exp\left(-\frac{\|\vec{u}\|^2}{2\sigma^2}\right)\tilde{\phi}(\vec{u}) \tag{28}$$

where $\tilde{\phi}(\vec{u})$ is the component from the infinite expansions of the Taylor series that only contains $\vec{u}$. This σ is a parameter that can control how quickly the similarity decays with distance. To connect this back to the problem, notice that if we choose a small σ, there will be high variance in the classification,because the Gaussian profile is very narrow and the model memorize the data points that is very close or within the subset of the dataset. On the other hand, if σ is large we have very wide Gaussian profile which may lead to an over-smoothing of the kernel, so it becomes hard for the model to predict where the new data points would belong, hence biased.

Support Vector Regression: An Asymmetric Twist

We now turn to Support Vector Regression Learning corresponding to the use of the large margin paradigm when the outspace $\mathscr{Y} \subseteq \mathbb{R}$. The standard $\epsilon-$insensitive loss in Support Vector Regression (SVR) is symmetric: it penalizes absolute deviations greater than ϵ equally, regardless of sign.

$$L_\epsilon(y, f(\vec{x})) = (0, |y - f(\vec{x})| - \epsilon)_+ \tag{29}$$

Imagine a financial forecasting task where overestimating a company's value ($f(\vec{x}) > y + \epsilon$) is much more costly than underestimating it ($f(\vec{x}) < y + \epsilon$).

(a) Propose a mathematically sound asymmetric loss function for SVR to handle this scenario. Sketch its plot.

The motivation sparks from the fact that we want to have a higher loss when the prediction overestimating and lower loss when the prediction function underestimates.

$$L_\epsilon^{\text{asym}}(y, f(\vec{x})) = \begin{cases} 0 & \text{if } |y - f(\vec{x})| \leq \epsilon \\ C^+(f(\vec{x}) - y - \epsilon) & \text{if } f(\vec{x}) > y + \epsilon \\ C^-(y - f(\vec{x}) - \epsilon) & \text{if } f(\vec{x}) < y - \epsilon \end{cases} \tag{30}$$

Where $C^+ > C^- > 0$ to place penalty on overestimating as it is more costly for the company.

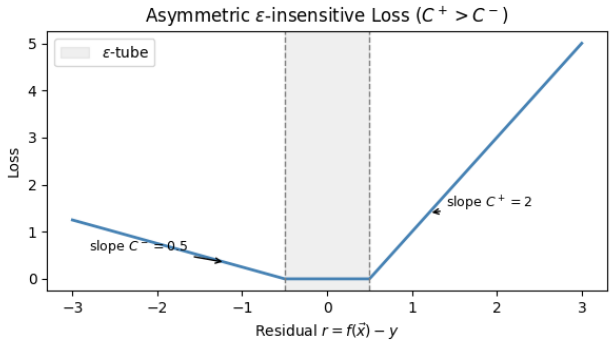


Figure 1: Sketch of Asymmetric Loss Function

(b) Formulate the primal optimization problem for an SVR using your proposed asymmetric loss. You do not need to derive the dual, but clearly define all variables and constraints.

Since we know have two different penalty on the loss, it follows that the primal optimization for an SVR is given as

$$\begin{aligned} \min_{w,b,\xi^+,\xi^-} \quad & \frac{1}{2}\|w\|^2 + C^+ \sum_{i=1}^{n} \xi_i^+ + C^- \sum_{i=1}^{n} \xi_i^- \\ f(\vec{x}_i) - y_i \leq \epsilon + \xi_i^+, \quad & y_i - f(\vec{x}_i) \leq \epsilon + \xi_i^- \quad \forall i \end{aligned} \tag{31}$$

where $\xi_i^+ \geq 0$ are slack variables penalising overestimation ($\hat{y}_i > y_i + \epsilon$) with cost C^+, and $\xi_i^- \geq 0$ are slack variables penalising underestimation ($\hat{y}_i < y_i - \epsilon$) with cost C^-.

(c) Propose and write down the mathematical expression of $\hat{f}_{svr}(\vec{x}_{new})$ in terms of the kernel and the Lagrange multipliers.

The prediction function

$$\hat{f}_{svr}(\vec{x}_{new}) = \sum_{1 \leq i \leq n} (\alpha_i^+ - \alpha_i^-) K(\vec{x}_i, \vec{x}_{new}) + b, \quad \alpha_i^+ \in [0, C^+], \quad \alpha_i^- \in [0, C^-] \tag{32}$$

Part B. Connecting Learning Paradigms

The Grand Unified View: Loss + Penalty

Many statistical learning algorithms can be expressed in the form:

$$\min_{f \in \mathcal{H}} \sum_{i=1}^{n} L(y_i, f(\vec{x}_i)) + \lambda J(f) \tag{33}$$

where L is a loss function and $J(f)$ is a regularizer.
(a) Complete the following table, specifying the Loss $L(\cdot, \cdot)$ and Regularizer $J(f)$ for each method.

Method	**Loss** $L(y, f(\vec{x}))$	**Regularizer** $J(f)$	**Function Class** $\mathcal{H}$
Ridge Regression	$(y - f(\vec{x}))^2$	$\|\beta\|_2^2$	Linear functions
Lasso	$(y - f(\vec{x}))^2$	$\|\beta\|_1$	Linear functions
SVM (Primal)	$(0,\ 1 - y_i f(\vec{x}_i))_+$	$\|\vec{w}\|^2$	Linear/RKHS
Logistic Regression	$\log\left(1 + e^{-y_i f(\vec{x}_i)}\right)$	$\|\beta\|_2^2$	Linear functions

(b) Based on your table, explain the fundamental philosophical difference between the "loss" used in SVM classification and the "loss" used in Logistic Regression. How does this difference manifest itself in the resulting models (e.g., in their sensitivity to outliers)? It is clear that from the loss of the SVM we use the hinge loss

$$\ell_{SVM}(y, \hat{f}) = (0,\ 1 - y_i \hat{f}(\vec{x}_i))_+ \tag{34}$$

If the model correctly classifies the data $\vec{x}$ then the component $y_i \hat{f} \geq 1$ which means $1 - y_i \hat{f} \leq 0$ so $\ell_{SVM} = 0$, but even a correct classification in Logistic Regression will still add positive loss, we can see that from the negative Bernoulli Log-Likelihood

$$p(y_i = y_0 \mid \vec{x}_i) = \pi_i^{y_i}(1 - \pi_i)^{1 - y_i}, \qquad \mathcal{L}(\beta) = \prod_{i=1}^{n} \pi_i^{y_i}(1 - \pi_i)^{1 - y_i} \tag{35}$$

Taking the log yields

$$\ell(\beta) = \sum_{i=1}^{n} \left[y_i \log \pi_i + (1 - y_i) \log(1 - \pi_i) \right] \tag{36}$$

$$\log \pi_i = \log \frac{1}{1 + e^{-\eta_i}} = -\log(1 + e^{-\eta_i}), \qquad \log(1 - \pi_i) = \log \frac{e^{-\eta_i}}{1 + e^{-\eta_i}} = -\eta_i - \log(1 + e^{-\eta_i}) \tag{37}$$

We can then write the log-likelihood using (37)

$$\begin{aligned} \ell(\beta) &= \sum_{i=1}^{n} \left[-y_i \log(1 + e^{-\eta_i}) + (1 - y_i) \left(-\eta_i - \log(1 + e^{-\eta_i}) \right) \right] \\ &= \sum_{i=1}^{n} \left[y_i \eta_i - \log(1 + e^{\eta_i}) \right] \end{aligned} \tag{38}$$

Maximizing ℓ in (38) is equivalent to minimizing $-\ell$

$$\arg\min_{\beta} \{-\ell(\beta)\} = \arg\min_{\beta} \left\{ \sum_{i=1}^{n} \log(1 + e^{\eta_i}) - y_i \eta_i \right\} = \arg\min_{\beta} \left\{ \sum_{i=1}^{n} \log(1 + e^{-y_i \eta_i}) \right\} \tag{39}$$

$$L(y_i, f(\vec{x}_i)) = \log\left(1 + e^{-y_i f(\vec{x}_i)}\right) \tag{40}$$

Circling back, under the notation for binary class $y_i \in \{-1, +1\}$ we can see that for correctly classified points and incorrectly classified points, the loss can be directly compared as

$$\log(1 + e^{-1}) < \log(1 + e^{+1}) \tag{41}$$

The Dimensionality Curse and Blessing

Consider the RBF kernel $K(\vec{u}, \vec{v}) = \exp(-\gamma\|\vec{u} - \vec{v}\|^2)$.

(a) In high-dimensional spaces ($p \gg n$), explain the phenomenon known as "distance concentration," where many pairs of points become almost equidistant. What is the implication of this for the RBF kernel matrix as p grows very large?

As p grows we can see that the distance measure $||\vec{u} - \vec{v}||^2$ in the kernel loses it meaning. Mathematically we can show that as p grows the difference between the maximum distance of two points to the minimum distance of two points shrinks faster than the minimum distance.

$$\lim_{p\to\infty} \frac{\max \|\vec{u} - \vec{v}\| - \min \|\vec{u} - \vec{v}\|}{\min \|\vec{u} - \vec{v}\|} \to 0 \tag{42}$$

This means that for $i \neq j$ we have the distances reduce to a constant D, it follows that the kernel matrix becomes $\exp(-\gamma D)$, and recall that the kernel matrix measures similarity, in this case $K(x_i, x_j) \approx 1$ if $\gamma \to 0$ and $K(x_i, x_j) \approx 0$ as $\gamma \to \infty$. This means that every data point has perfect similarity as every other data point or totally different, rendering the RBF kernel to be meaningless for any learning.

(b) Propose a theoretical argument for why, in such high-dimensional settings, an RBF-SVM might struggle to learn a meaningful pattern without very careful tuning of γ. (*Hint: Think about the kernel matrix becoming increasingly similar to the identity matrix*).

This is discussed in (a) of The Dimensionality Curse and Blessing and also in (c) of The Kernel Trick: From Theory to Practice. Essentially, for RBF kernel, if we increase γ, then that is the same as decreasing σ in (28), and vice versa. If γ is too large, the distances are amplified in the exponent, which causes $K \approx 0$ for $i \neq j$ and $K = 1$ on the identity (since $\exp(-\gamma(0)) = 1$. The results in $K \approx I$. This will lead to a perfect memorization of the training set $\mathscr{X}_{tr} \subsetneq \mathscr{X}$, there will be almost no predictive or generalization power from such K.

(c) Surprisingly, SVMs often work well in practice even when $p > n$. Connect this to the "blessing of dimensionality" argument: that in high dimensions, data is often inherently linearly separable. Explain briefly why this might be the case.

Due to the ability of SVM to project the feature space into a higher dimensional feature space $\mathcal{F}$ where the data is linearly separable by hyper-planes. The kernel trick allows us to avoid the explicit necessity of searching for the form of such projection function ϕ. For example in dimensional datasets like genes and text the data matrix is very sparse, they will have a high chance of being separable in their original feature space.

Proof. Recall Cover's Theorem [3], we can show the probability of obtain a separable hyperplane in higher dimension is high, we can show such probability measure is $C(n, p)/2^n$ given that the data is rich. The 2^n is the total possible partitions of n points into two classes. If the points are in general position (meaning no subset of $p + 1$ points lies on a $(p - 1)$-dimensional hyperplane), the number of ways to linearly partition these points into two classes is given by the function

$$C(n, p) = 2\sum_{i=0}^{p-1} \binom{n-1}{i}, \qquad P(n, p) = 2^{1-n}\sum_{i=0}^{p-1} \binom{n-1}{i} \tag{43}$$

In this case if $p \gg n$, then we can see that $\binom{n}{p}$ will be 0, so the index i will allow us to sum on the discrete set $0 \leq i \leq n-1$, and for $i \geq n - 1, C(n, i) = 0$. Theoretically, the probability of separability will be 1 assuming the general positioned of points.

$$P(n, p_{\gg n}) = 2^{1-n}2^{n-1} = 1 \tag{44}$$

□

It can also be shown that for large n the Gaussian profile can be used to approximate the binomial coefficients.

$$P(n, p_{\gg n}) \approx \sum_{1\leq i\leq n-1} \mathcal{N}(i : \frac{n}{2}, \frac{n}{4}) \approx \Phi(p - 1; \frac{n}{2}, \frac{n}{4}) \tag{45}$$

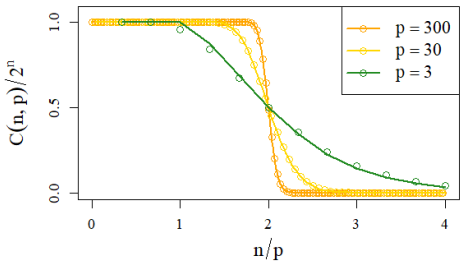


Figure 2: Probability of Separability

Favorite Picture of the Report

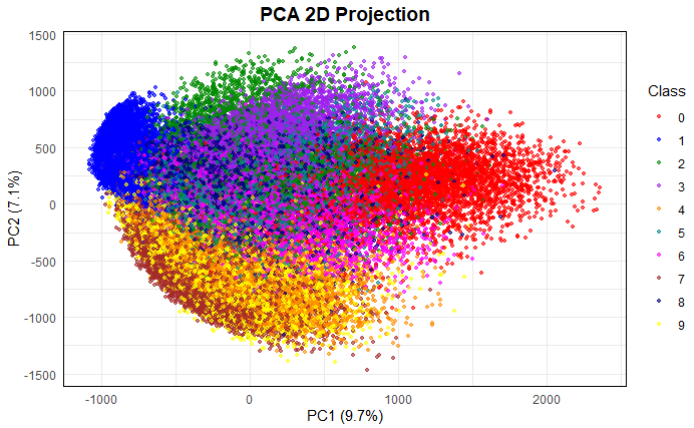


Figure 3: PCA 2D Project

References

[1] Bertrand Clarke, Ernest Fokoue, and Hao Helen Zhang. *Principles and theory for data mining and machine learning*. Springer Science & Business Media, 2009.

[2] Jonathan H Manton and Pierre-Olivier Amblard. A primer on reproducing kernel hilbert spaces. *Foundations and trends in signal processing*, 8(1-2):1–126, 2015.

[3] Frank Samuelson and David G Brown. Application of cover's theorem to the evaluation of the performance of ci observers. In *The 2011 International Joint Conference on Neural Networks*, pages 1020–1026. IEEE, 2011.

STAT 547 - Data Mining Homework 3 - Computations and Applications

Ducheng Tan

2026-03-25

```
#install.packages("BMS")
#install.packages("ggplot2")
#install.packages("latex2exp")
#install.packages("gbm")
library(ggplot2)
library(BMS)
library(forcats)
library(dplyr)
library(tidyr)
library(caret)
library(e1071)
library(rpart)
library(rpart.plot)
library(glmnet)
library(latex2exp)
library(randomForest)
library(gbm)
library(tictoc)
library(patchwork)
library(pdp)
```

```
get_metrics <- function(actual, predicted, label = "")
{
  rmse   <- sqrt(mean((actual - predicted)^2))
  mae    <- mean(abs(actual - predicted))
  ss_res <- sum((actual - predicted)^2)
  ss_tot <- sum((actual - mean(actual))^2)
  r2     <- 1 - ss_res / ss_tot
  if (label != "") cat(label, "\n")
  cat("  RMSE:", round(rmse, 6), "\n")
  cat("  MAE :", round(mae,  6), "\n")
  cat("  R2  :", round(r2,   6), "\n\n")
  return(c(RMSE = round(rmse, 6),
           MAE  = round(mae,  6),
           R2   = round(r2,   6)))
}
```

Exercise 2: Applied Regression Learning

The dataset for statistical learning task is `datafls` from `library(BMS)`, and the Response variable: `y` (continuous outcome).

Part A. Data Exploration and Preparation

1. Data Understanding

E2.A.1a Load the data and provide summary statistics.

```
data(datafls)

dim(datafls)
```

```
## [1] 72 42
```

```
names(datafls)[1]
```

```
## [1] "y"
```

```
y <- datafls$y
x <- datafls[, !names(datafls) == "y"]



# Check
dim(x)
```

```
## [1] 72 41
```

```
length(y)
```

```
## [1] 72
```

```
summary(x)
```

```
##      Abslat          Spanish          French           Brit       
##  Min.   : 0.228   Min.   :0.0000   Min.   :0.000   Min.   :0.0000  
##  1st Qu.:12.107   1st Qu.:0.0000   1st Qu.:0.000   1st Qu.:0.0000  
##  Median :22.120   Median :0.0000   Median :0.000   Median :0.0000  
##  Mean   :25.733   Mean   :0.2222   Mean   :0.125   Mean   :0.3194  
##  3rd Qu.:37.437   3rd Qu.:0.0000   3rd Qu.:0.000   3rd Qu.:1.0000  
##  Max.   :60.212   Max.   :1.0000   Max.   :1.000   Max.   :1.0000  
##     WarDummy        LatAmerica       SubSahara        OutwarOr      
##  Min.   :0.0000   Min.   :0.0000   Min.   :0.0000   Min.   :0.0000  
##  1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.:0.0000  
##  Median :0.0000   Median :0.0000   Median :0.0000   Median :0.0000  
##  Mean   :0.4028   Mean   :0.2778   Mean   :0.2083   Mean   :0.3889  
##  3rd Qu.:1.0000   3rd Qu.:1.0000   3rd Qu.:0.0000   3rd Qu.:1.0000
```

```
##  Max.   :1.0000   Max.   :1.0000   Max.   :1.0000   Max.   :1.0000
##       Area           PrScEnroll        LifeExp          GDP60
##  Min.   :   1.00   Min.   :0.0700   Min.   :37.90   Min.   :5.518
##  1st Qu.:  87.75   1st Qu.:0.6475   1st Qu.:45.85   1st Qu.:6.915
##  Median : 312.50   Median :0.9450   Median :54.60   Median :7.388
##  Mean   : 972.92   Mean   :0.7953   Mean   :56.58   Mean   :7.492
##  3rd Qu.: 762.50   3rd Qu.:1.0000   3rd Qu.:68.80   3rd Qu.:8.104
##  Max.   :9976.00   Max.   :1.0000   Max.   :73.40   Max.   :9.188
##      Mining            EcoOrg         YrsOpen            Age
##  Min.   :0.00000   Min.   :0.000   Min.   :0.0000   Min.   : 0.00
##  1st Qu.:0.00475   1st Qu.:3.000   1st Qu.:0.0890   1st Qu.: 0.00
##  Median :0.01850   Median :3.000   Median :0.4110   Median : 0.00
##  Mean   :0.04487   Mean   :3.542   Mean   :0.4393   Mean   :23.71
##  3rd Qu.:0.05025   3rd Qu.:5.000   3rd Qu.:0.7780   3rd Qu.:76.00
##  Max.   :0.53300   Max.   :5.000   Max.   :1.0000   Max.   :90.00
##      Buddha           Catholic        Confucian           EthnoL
##  Min.   :0.00000   Min.   :0.0000   Min.   :0.00000   Min.   :0.0000
##  1st Qu.:0.00000   1st Qu.:0.0200   1st Qu.:0.00000   1st Qu.:0.0950
##  Median :0.00000   Median :0.2700   Median :0.00000   Median :0.3150
##  Mean   :0.05611   Mean   :0.4218   Mean   :0.01903   Mean   :0.3706
##  3rd Qu.:0.00000   3rd Qu.:0.9000   3rd Qu.:0.00000   3rd Qu.:0.6400
##  Max.   :0.95000   Max.   :1.0000   Max.   :0.60000   Max.   :0.9300
##      Hindu             Jewish            Muslim         PrExports
##  Min.   :0.00000   Min.   :0.00000   Min.   :0.0000   Min.   :0.0410
##  1st Qu.:0.00000   1st Qu.:0.00000   1st Qu.:0.0000   1st Qu.:0.4427
##  Median :0.00000   Median :0.00000   Median :0.0000   Median :0.8035
##  Mean   :0.01806   Mean   :0.01271   Mean   :0.1475   Mean   :0.6732
##  3rd Qu.:0.00000   3rd Qu.:0.00000   3rd Qu.:0.1425   3rd Qu.:0.9210
##  Max.   :0.83000   Max.   :0.82000   Max.   :1.0000   Max.   :0.9980
##   Protestants       RuleofLaw           Popg             WorkPop
##  Min.   :0.000   Min.   :0.0000   Min.   :0.002369   Min.   :-1.3509
##  1st Qu.:0.010   1st Qu.:0.1667   1st Qu.:0.009345   1st Qu.:-1.1150
##  Median :0.050   Median :0.5000   Median :0.024373   Median :-0.9391
##  Mean   :0.173   Mean   :0.5509   Mean   :0.020386   Mean   :-0.9537
##  3rd Qu.:0.250   3rd Qu.:1.0000   3rd Qu.:0.028054   3rd Qu.:-0.8022
##  Max.   :0.950   Max.   :1.0000   Max.   :0.035654   Max.   :-0.5968
##     LabForce          HighEnroll        PublEdupct         RevnCoup
##  Min.   :   210.9   Min.   :0.00000   Min.   :0.00410   Min.   :0.0000
##  1st Qu.:  1183.3   1st Qu.:0.00575   1st Qu.:0.01645   1st Qu.:0.0000
##  Median :  2898.4   Median :0.02600   Median :0.02320   Median :0.0800
##  Mean   :  9305.4   Mean   :0.04335   Mean   :0.02465   Mean   :0.1824
##  3rd Qu.:  7487.5   3rd Qu.:0.06750   3rd Qu.:0.03200   3rd Qu.:0.3018
##  Max.   :195781.5   Max.   :0.32100   Max.   :0.04630   Max.   :1.1900
##    PolRights         CivlLib          English           Foreign
##  Min.   :1.000   Min.   :1.000   Min.   :0.00000   Min.   :0.0000
##  1st Qu.:1.653   1st Qu.:2.028   1st Qu.:0.00000   1st Qu.:0.0000
##  Median :3.694   Median :3.667   Median :0.00000   Median :0.0555
##  Mean   :3.451   Mean   :3.466   Mean   :0.07571   Mean   :0.3744
##  3rd Qu.:5.111   3rd Qu.:4.847   3rd Qu.:0.00000   3rd Qu.:0.8407
##  Max.   :6.667   Max.   :6.611   Max.   :0.97400   Max.   :1.0040
##     RFEXDist          EquipInv         NequipInv          stdBMP
##  Min.   : 51.00   Min.   :0.00270   Min.   :0.0371   Min.   :  0.001
##  1st Qu.: 96.25   1st Qu.:0.01500   1st Qu.:0.1125   1st Qu.:  0.001
##  Median :112.50   Median :0.03610   Median :0.1519   Median : 13.149
```

```
##  Mean   :121.71   Mean   :0.04429   Mean   :0.1495   Mean   : 45.596
##  3rd Qu.:131.00   3rd Qu.:0.06513   3rd Qu.:0.1875   3rd Qu.: 38.699
##  Max.   :277.00   Max.   :0.14820   Max.   :0.2803   Max.   :588.627
##     BlMktPm
##  Min.   :0.0000
##  1st Qu.:0.0000
##  Median :0.0285
##  Mean   :0.1567
##  3rd Qu.:0.1855
##  Max.   :1.8550
```

```
summary(y)
```

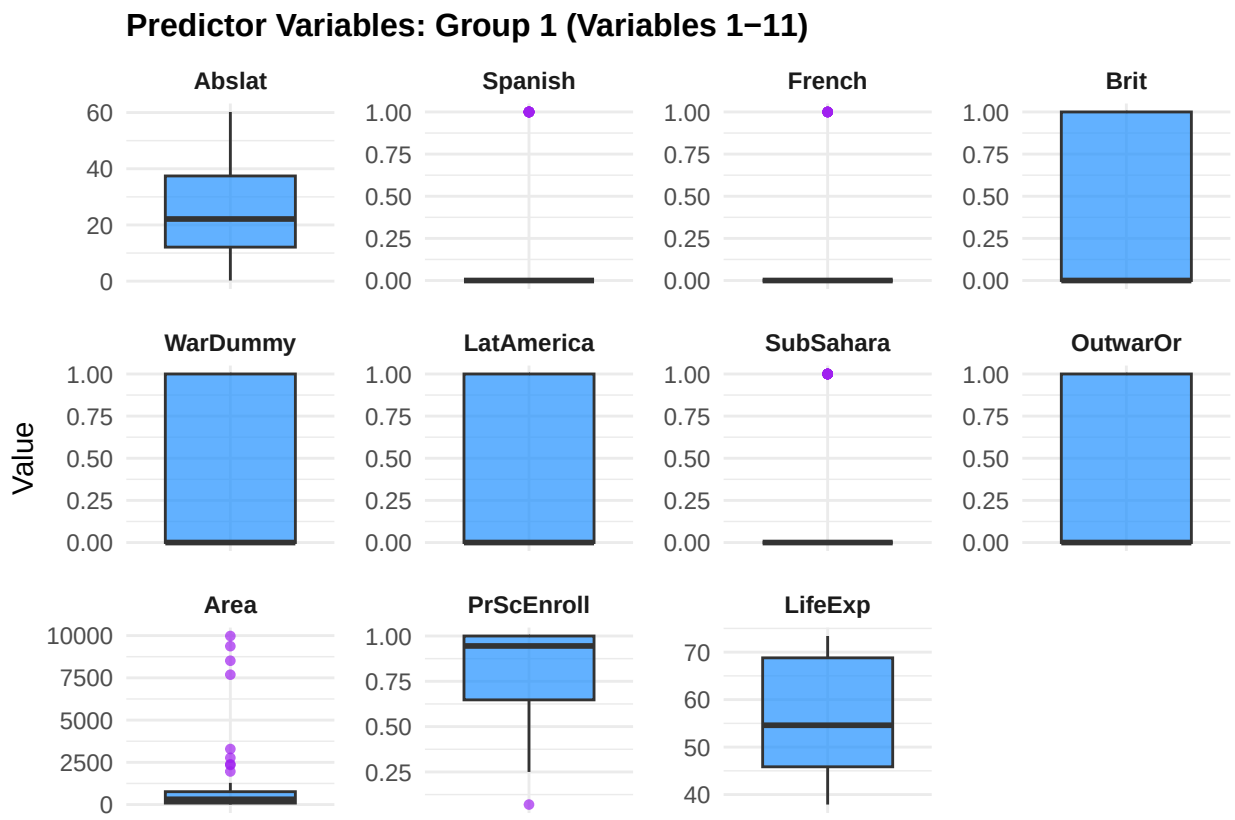
```
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## -0.020769  0.009387  0.020306  0.020728  0.028733  0.066179
```

```
Kappa <- nrow(datafls)/ ncol(datafls)
Kappa
```

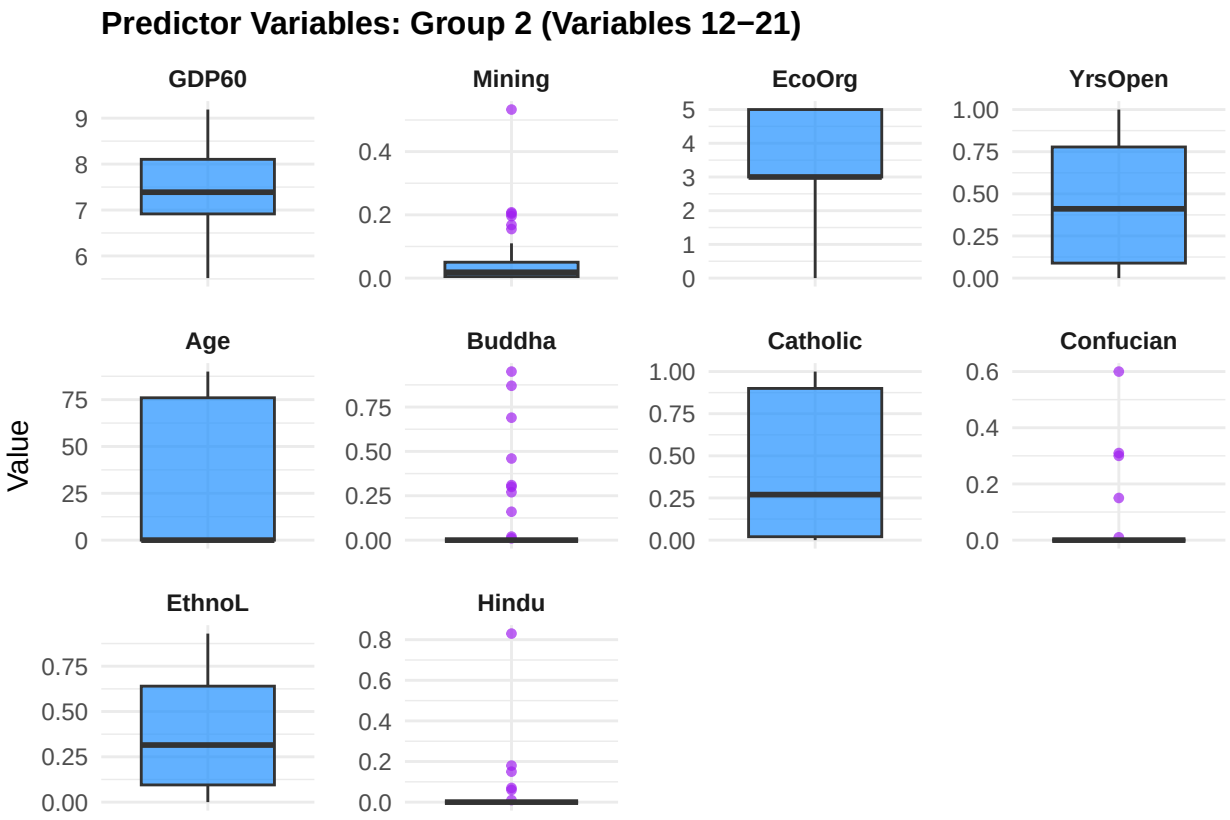
```
## [1] 1.714286
```

```
var_names <- names(x)
group1 <- var_names[1:11]
group2 <- var_names[12:21]
group3 <- var_names[22:31]
group4 <- var_names[32:41]


plot_boxplots <- function(data, vars, title) {
  x_sub  <- data[, vars, drop = FALSE]
  x_long <- stack(x_sub)
  colnames(x_long) <- c("Value", "Variable")

  ggplot(x_long, aes(y = Value, x = "")) +
    geom_boxplot(fill = "dodgerblue", alpha = 0.7,
                 outlier.colour = "purple", outlier.size = 1.2) +
    facet_wrap(~ Variable, scales = "free_y", ncol = 4) +
    labs(title = title, x = "", y = "Value") +
    theme_minimal() +
    theme(
      strip.text       = element_text(size = 9, face = "bold"),
      axis.text.x      = element_blank(),
      axis.ticks.x     = element_blank(),
      plot.title       = element_text(size = 12, face = "bold"),
      panel.spacing    = unit(0.8, "lines")
    )
}


p1 <- plot_boxplots(x, group1, "Predictor Variables: Group 1 (Variables 1-11)")
p2 <- plot_boxplots(x, group2, "Predictor Variables: Group 2 (Variables 12-21)")
p3 <- plot_boxplots(x, group3, "Predictor Variables: Group 3 (Variables 22-31)")
```

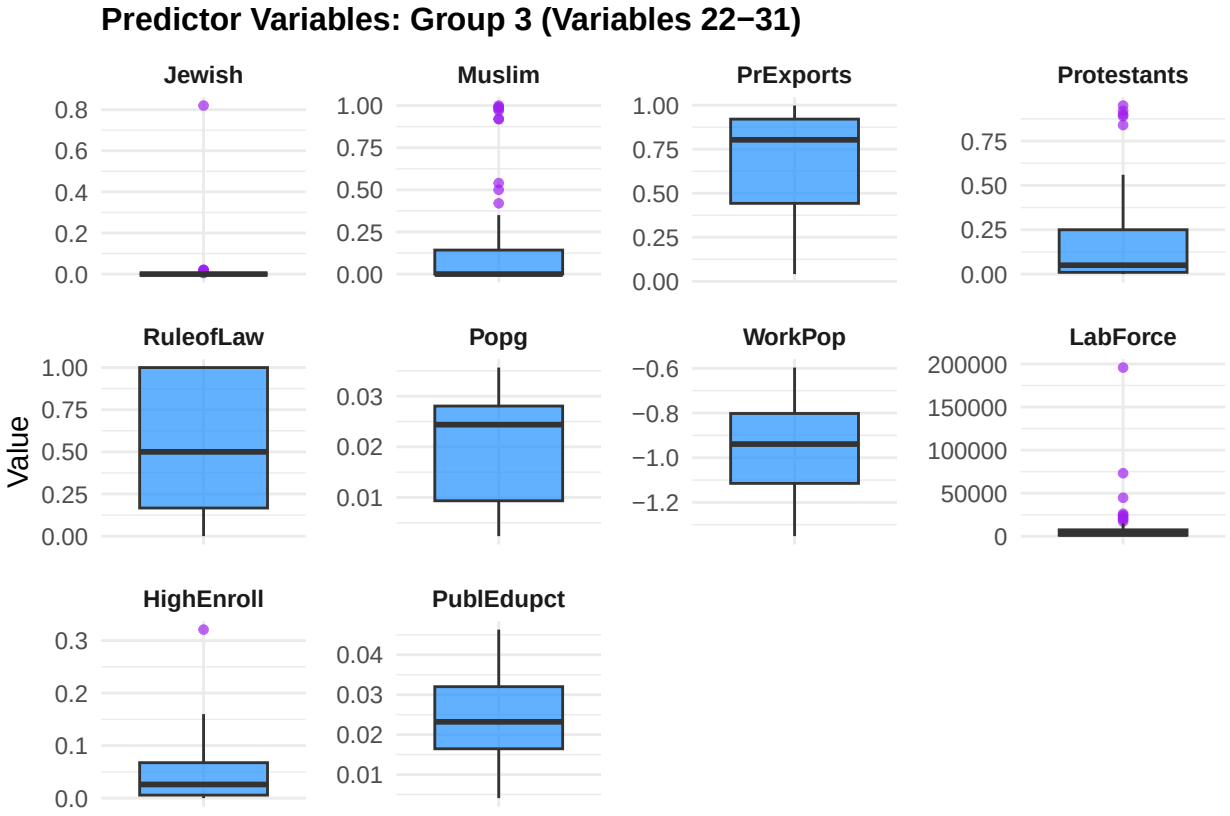
```
p4 <- plot_boxplots(x, group4, "Predictor Variables: Group 4 (Variables 32-41)")


print(p1)
```



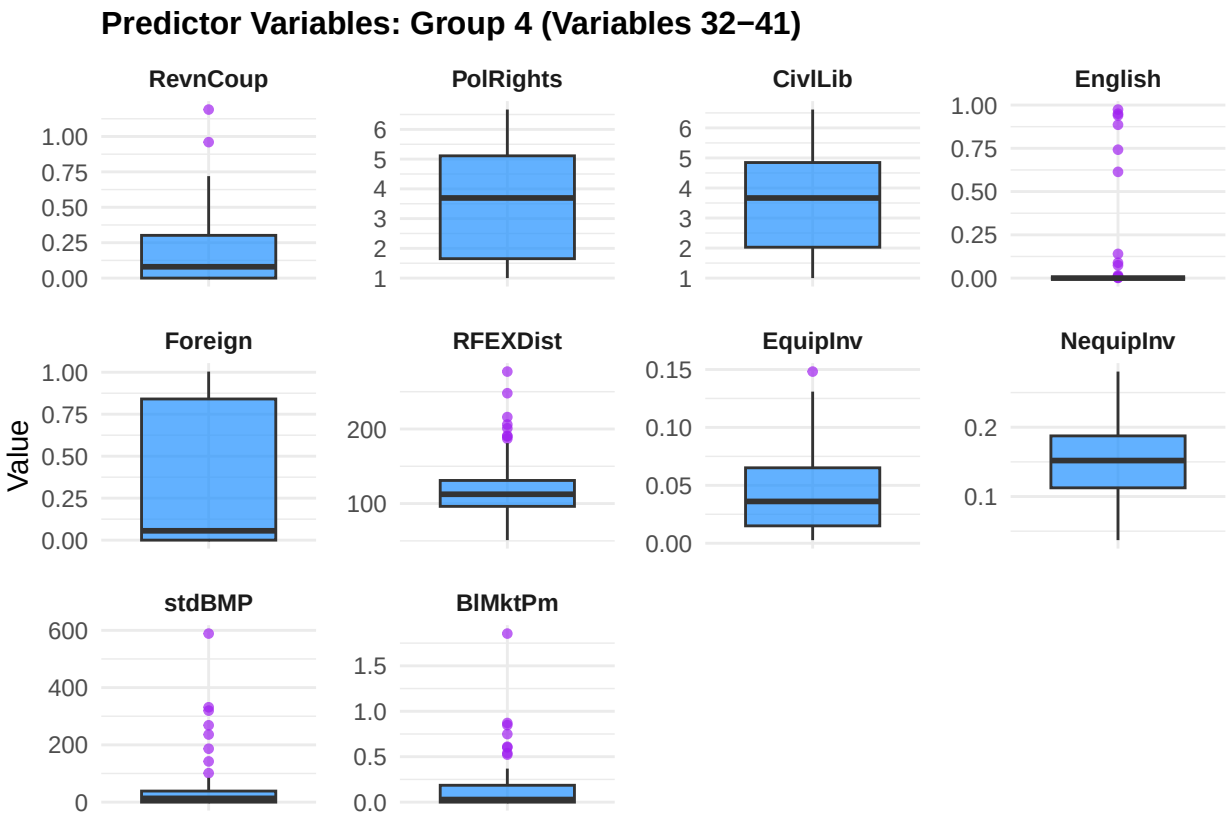

```
print(p2)
```


Predictor Variables: Group 2 (Variables 12−21)
GDP60
Mining
EcoOrg
YrsOpen
Age
Buddha
Catholic
Confucian
EthnoL
Hindu
Value

```
print(p3)
```


Predictor Variables: Group 3 (Variables 22−31)
Jewish
Muslim
PrExports
Protestants
RuleofLaw
Popg
WorkPop
LabForce
HighEnroll
PublEdupct
Value

```
print(p4)
```




```
ggsave("boxplots_group1.png", p1, width = 12, height = 8, dpi = 300)
ggsave("boxplots_group2.png", p2, width = 12, height = 8, dpi = 300)
ggsave("boxplots_group3.png", p3, width = 12, height = 8, dpi = 300)
ggsave("boxplots_group4.png", p4, width = 12, height = 8, dpi = 300)
```

Although we plot the box plots for every variable for completeness but we recognize that some predictor variables are binary so the boxplots are less meaning full for them.

E2.A.1b Create visualizations showing relationships between predictors and response.

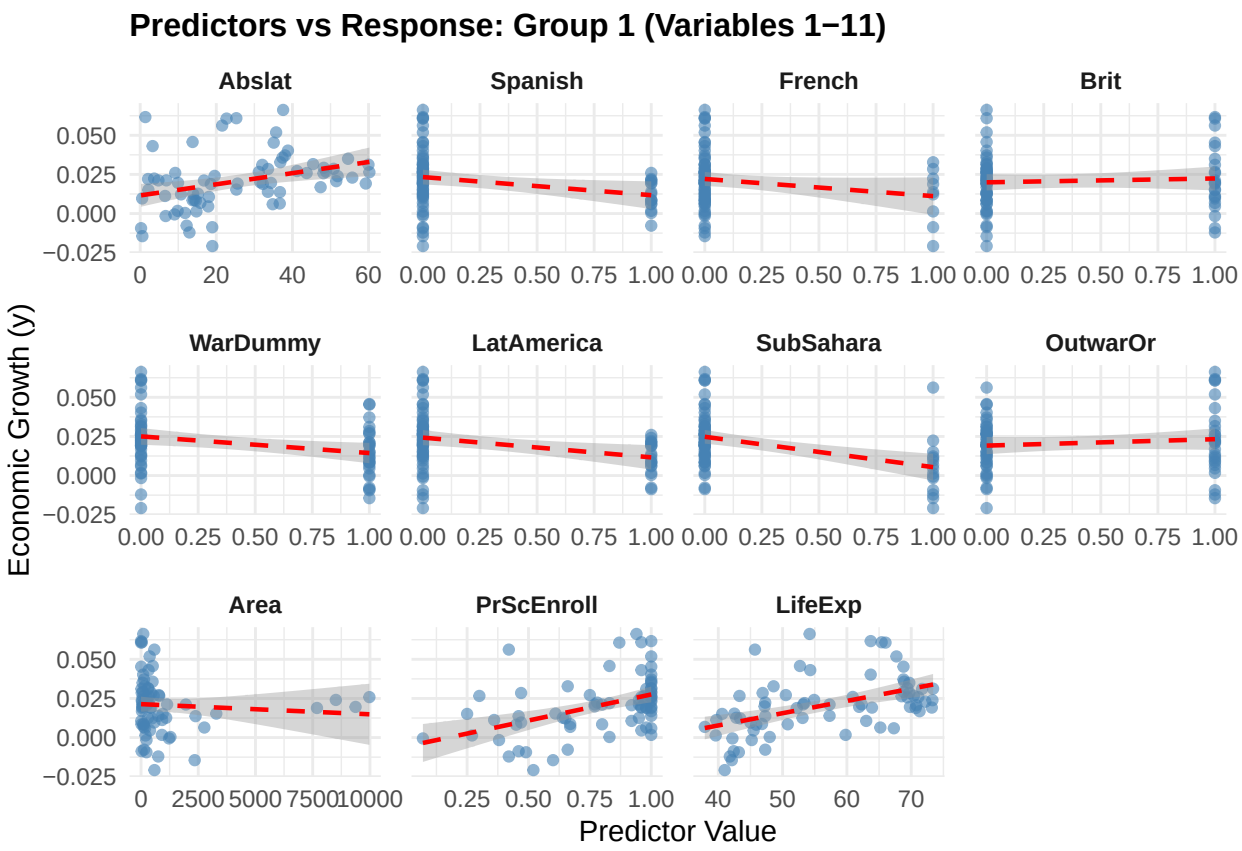
Again the binary variables are there for completeness we understand that the linear regression line may be statistically insignificant to the binary class of points, however we know that if $x_i \in \{0,1\} \subset X$, then the regression line simply becomes either $\hat{y} = \hat{\beta}_0$ if $x_i = 0$ and $\hat{y} = \hat{\beta}_0 + \hat{\beta}_i$ if $x_i = 1$. Then $\hat{\beta}_i$ is the difference caused by the difference of being in a different class.

$$\hat{y} = \hat{\beta}_j x_j + \hat{\beta}_0$$

```
group1 <- var_names[1:11]
group2 <- var_names[12:21]
group3 <- var_names[22:31]
group4 <- var_names[32:41]

plot_scatter <- function(data, vars, y, title) {
  df_long <- stack(data[, vars, drop = FALSE])
  colnames(df_long) <- c("Value", "Variable")
```

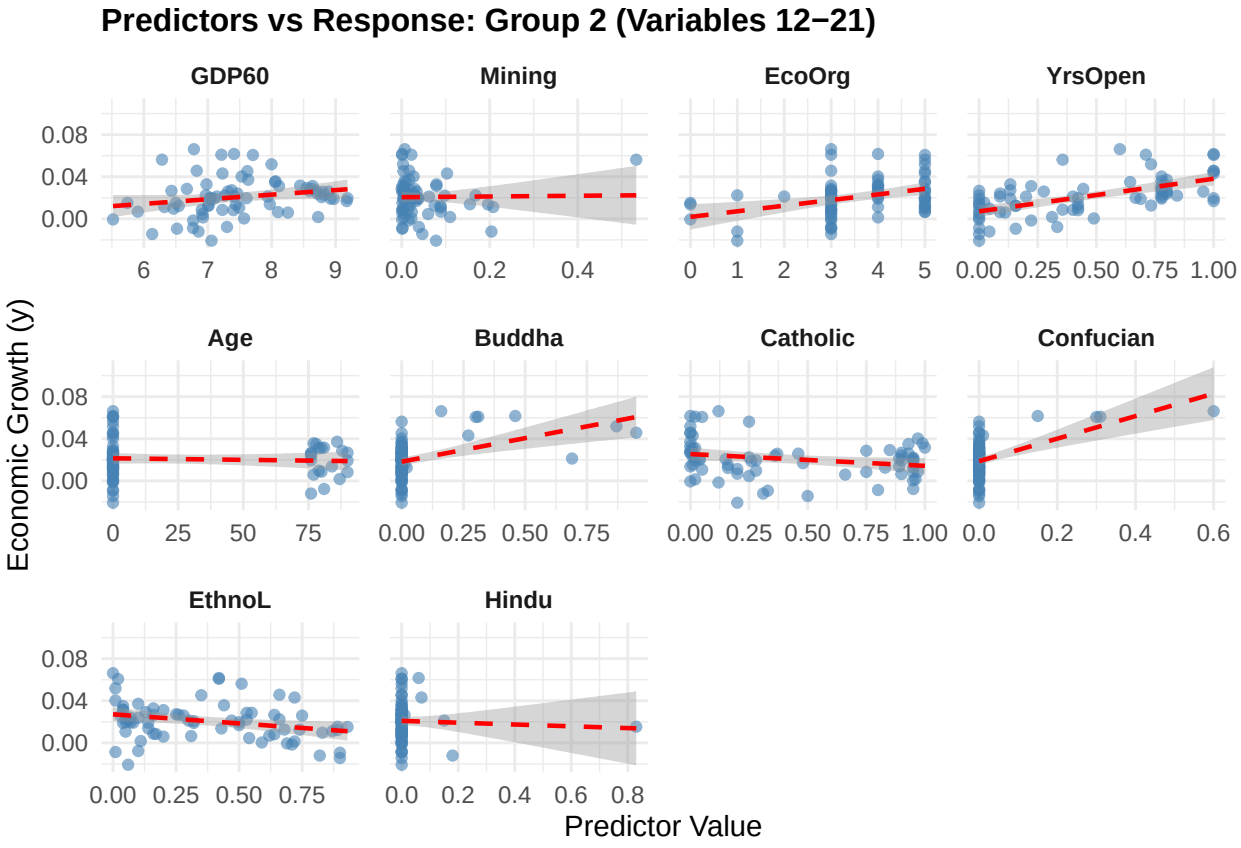
```r
  df_long$y <- rep(y, length(vars))

  ggplot(df_long, aes(x = Value, y = y)) +
    geom_point(colour = "steelblue", alpha = 0.6, size = 1.5) +
    geom_smooth(method = "lm", se = TRUE, colour = "red",
                linewidth = 0.8, linetype = "dashed") +
    facet_wrap(~ Variable, scales = "free_x", ncol = 4) +
    labs(title = title, x = "Predictor Value",
         y = "Economic Growth (y)") +
    theme_minimal() +
    theme(
      strip.text    = element_text(size = 9, face = "bold"),
      plot.title    = element_text(size = 12, face = "bold"),
      panel.spacing = unit(0.8, "lines")
    )
}

s1 <- plot_scatter(x, group1, y, "Predictors vs Response: Group 1 (Variables 1-11)")
s2 <- plot_scatter(x, group2, y, "Predictors vs Response: Group 2 (Variables 12-21)")
s3 <- plot_scatter(x, group3, y, "Predictors vs Response: Group 3 (Variables 22-31)")
s4 <- plot_scatter(x, group4, y, "Predictors vs Response: Group 4 (Variables 32-41)")

print(s1)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



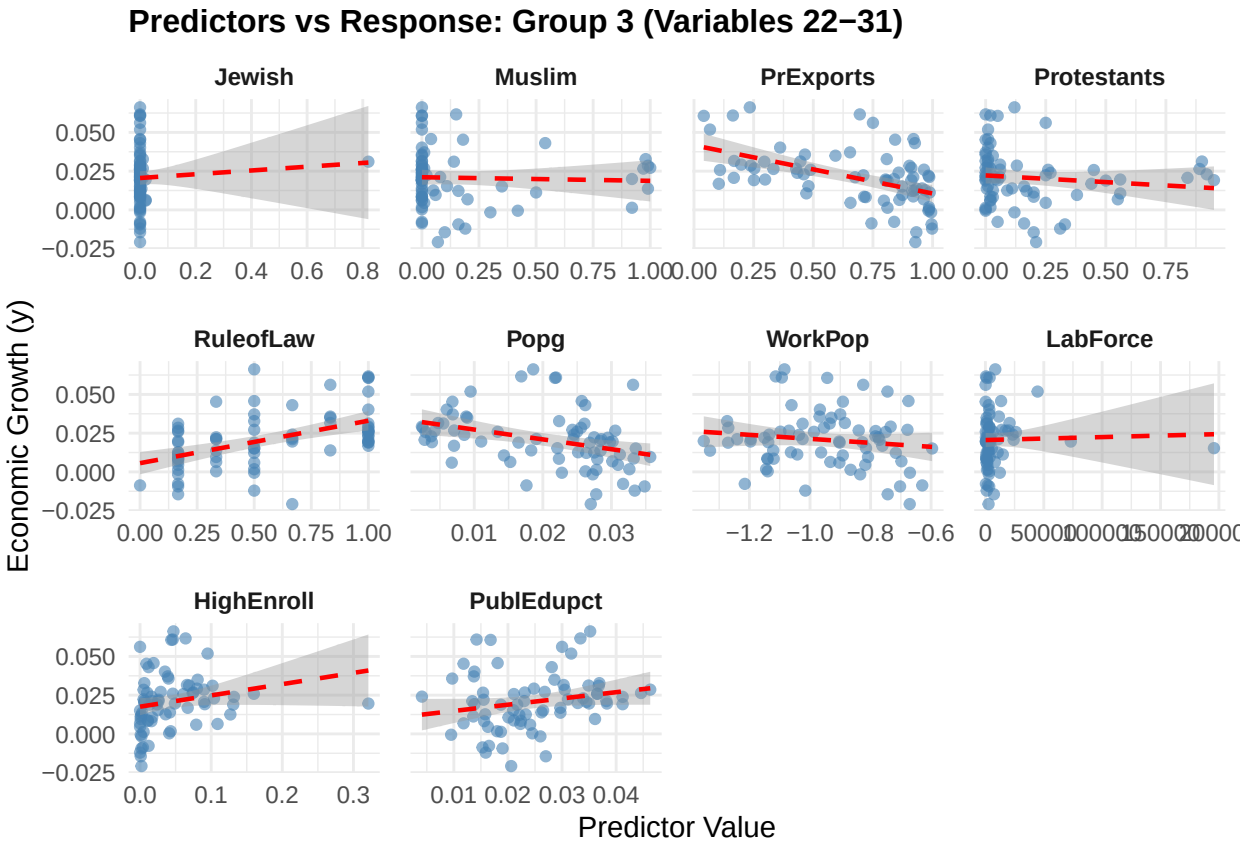
```
print(s2)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



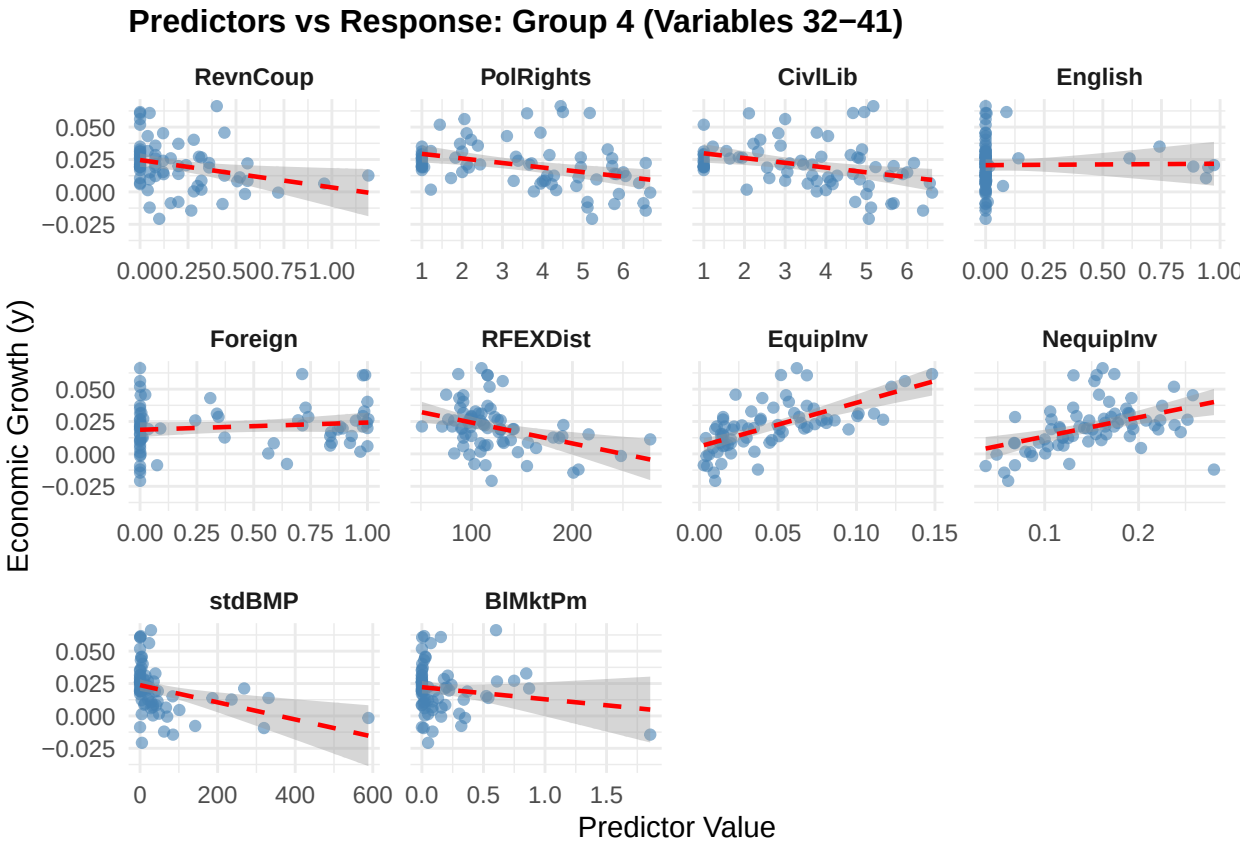

```
print(s3)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



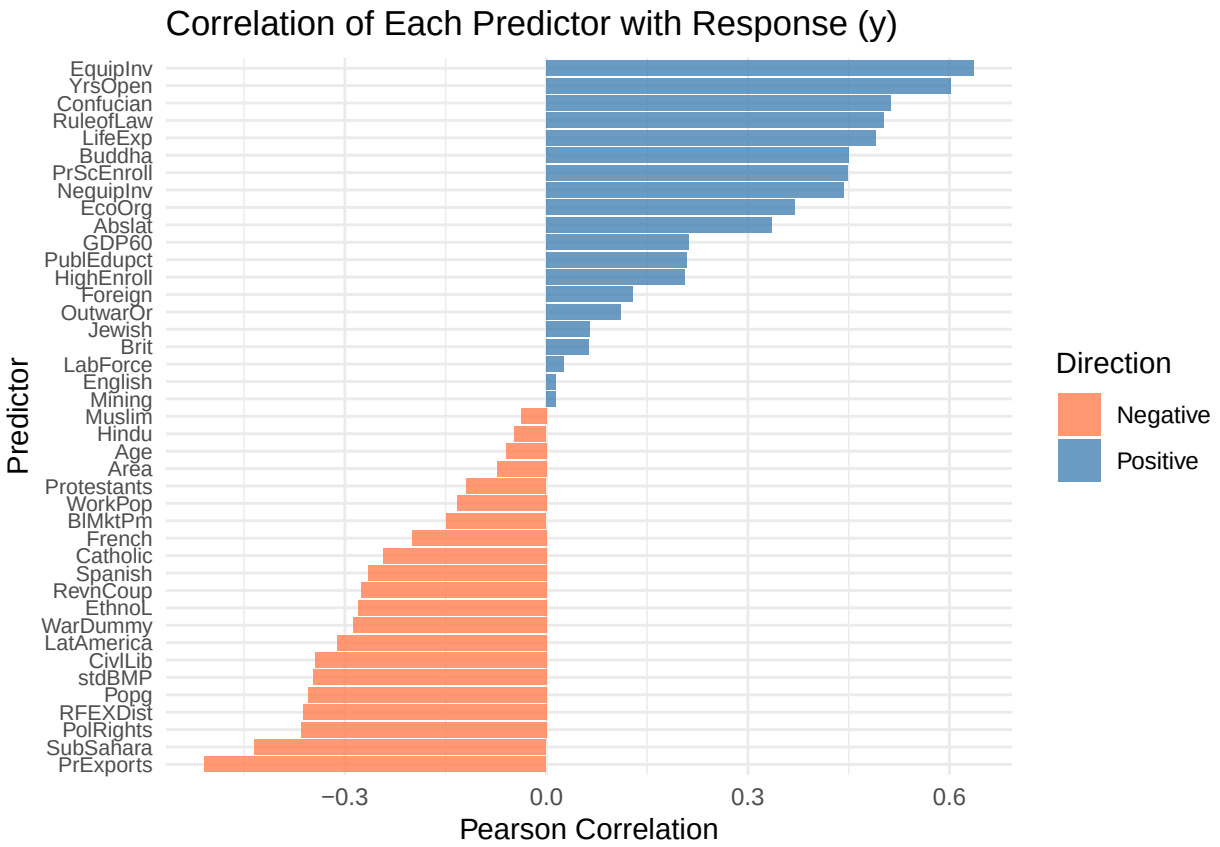

```
print(s4)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



Correlation Bar Chart for $Corr(x_i \in X, y)$

```r
# Optional: correlation bar chart - top correlated predictors with y
cors <- cor(x, y)
cor_df <- data.frame(
  Variable    = rownames(cors),
  Correlation = as.numeric(cors)
)
cor_df <- cor_df[order(abs(cor_df$Correlation), decreasing = TRUE), ]

ggplot(cor_df, aes(x = reorder(Variable, Correlation),
                   y = Correlation,
                   fill = Correlation > 0)) +
  geom_bar(stat = "identity", alpha = 0.8) +
  scale_fill_manual(values = c("TRUE" = "steelblue", "FALSE" = "coral"),
                    labels = c("TRUE" = "Positive", "FALSE" = "Negative"),
                    name   = "Direction") +
  coord_flip() +
  labs(title = "Correlation of Each Predictor with Response (y)",
       x = "Predictor", y = "Pearson Correlation") +
  theme_minimal() +
  theme(axis.text.y = element_text(size = 8))
```



E2.A.1c Check for missing values and outliers.

There are no missing values found in the dataset.

```
cat("=== Missing Value Summary ===\n")
```

```
## === Missing Value Summary ===
```

```
missing_counts <- colSums(is.na(datafls))
cat("Total missing values in dataset:", sum(missing_counts), "\n\n")
```

```
## Total missing values in dataset: 0
```

```
missing_df <- data.frame(
  Variable = names(missing_counts),
  Missing  = as.integer(missing_counts)
)
missing_df <- missing_df[missing_df$Missing > 0, ]

if (nrow(missing_df) == 0) {
  cat("No missing values found in any variable.\n")
} else {
  print(missing_df)
}
```

```
## No missing values found in any variable.
```

```r
cat("\n=== Outlier Summary (IQR Rule) ===\n")
```

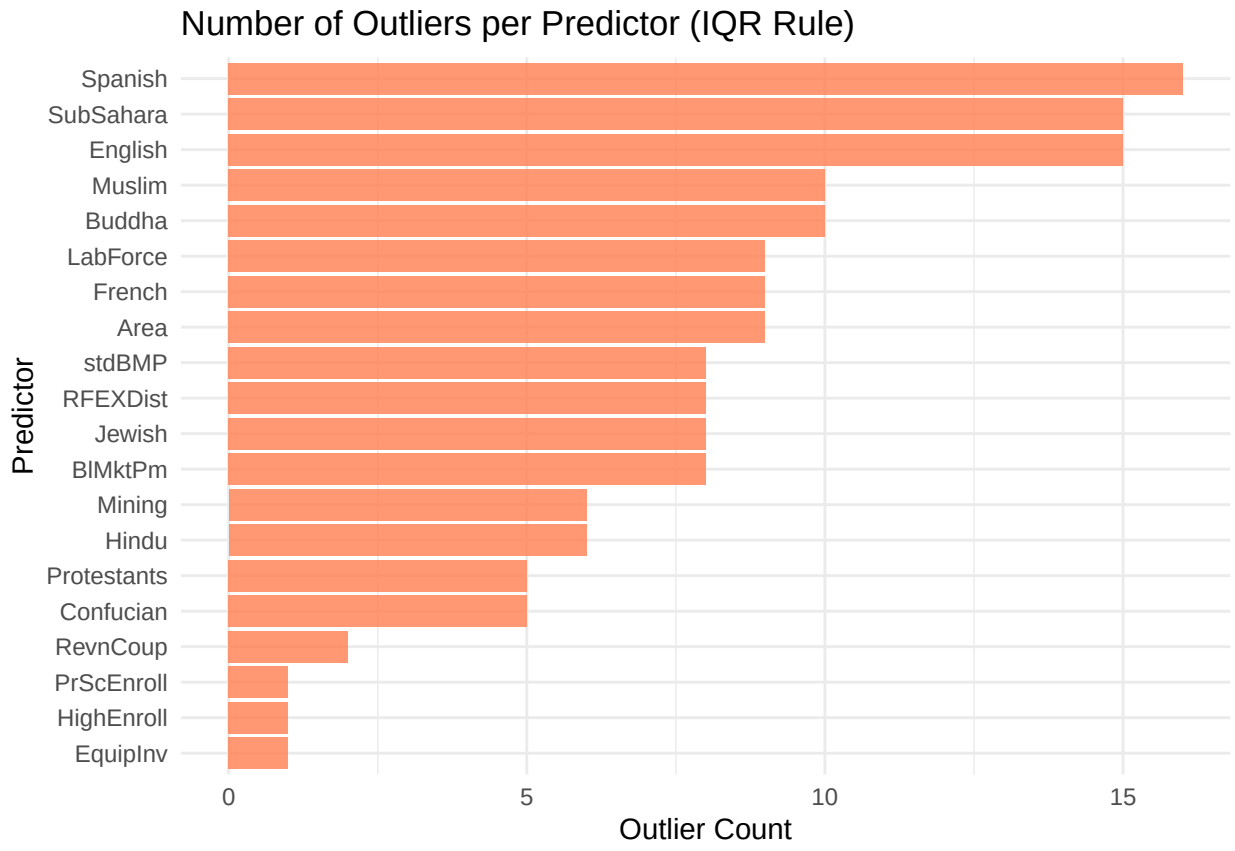
```
##
## === Outlier Summary (IQR Rule) ===
```

```r
outlier_counts <- sapply(x, function(col) {
  Q1  <- quantile(col, 0.25, na.rm = TRUE)
  Q3  <- quantile(col, 0.75, na.rm = TRUE)
  IQR <- Q3 - Q1
  sum(col < (Q1 - 1.5 * IQR) | col > (Q3 + 1.5 * IQR), na.rm = TRUE)
})

outlier_df <- data.frame(
  Variable      = names(outlier_counts),
  Outlier_Count = as.integer(outlier_counts)
)
outlier_df <- outlier_df[order(outlier_df$Outlier_Count, decreasing = TRUE), ]
print(outlier_df)
```

```
##       Variable Outlier_Count
## 2      Spanish            16
## 7    SubSahara            15
## 35     English            15
## 17      Buddha            10
## 23      Muslim            10
## 3       French             9
## 9         Area             9
## 29    LabForce             9
## 22      Jewish             8
## 37    RFEXDist             8
## 40      stdBMP             8
## 41     BlMktPm             8
## 13      Mining             6
## 21       Hindu             6
## 19   Confucian             5
## 25 Protestants             5
## 32    RevnCoup             2
## 10  PrScEnroll             1
## 30  HighEnroll             1
## 38    EquipInv             1
## 1       Abslat             0
## 4         Brit             0
## 5     WarDummy             0
## 6   LatAmerica             0
## 8     OutwarOr             0
## 11     LifeExp             0
## 12       GDP60             0
## 14      EcoOrg             0
## 15     YrsOpen             0
## 16         Age             0
## 18    Catholic             0
## 20      EthnoL             0
```

```
## 24   PrExports             0
## 26   RuleofLaw             0
## 27        Popg             0
## 28     WorkPop             0
## 31  PublEdupct             0
## 33   PolRights             0
## 34     CivlLib             0
## 36     Foreign             0
## 39   NequipInv             0
```

```r
# Visualise outlier counts
ggplot(outlier_df[outlier_df$Outlier_Count > 0, ],
       aes(x = reorder(Variable, Outlier_Count), y = Outlier_Count)) +
  geom_bar(stat = "identity", fill = "coral", alpha = 0.8) +
  coord_flip() +
  labs(title = "Number of Outliers per Predictor (IQR Rule)",
       x = "Predictor", y = "Outlier Count") +
  theme_minimal() +
  theme(axis.text.y = element_text(size = 9))
```




```r
Q1_y  <- quantile(y, 0.25)
Q3_y  <- quantile(y, 0.75)
IQR_y <- Q3_y - Q1_y
y_outliers <- which(y < (Q1_y - 1.5 * IQR_y) | y > (Q3_y + 1.5 * IQR_y))

cat("\n=== Outliers in Response Variable y ===\n")
```

```
##
## === Outliers in Response Variable y ===
```

```
cat("Number of outliers in y:", length(y_outliers), "\n")
```

```
## Number of outliers in y: 5
```

```
if (length(y_outliers) > 0) {
  cat("Observation indices:", y_outliers, "\n")
  cat("Outlier values:", y[y_outliers], "\n")
}
```

```
## Observation indices: 29 38 39 55 60
## Outlier values: 0.060658 0.066179 -0.020769 0.061622 0.060895
```

2. Feature Engineering

E2.A.2a Create any relevant interaction terms or polynomial features.

```
data(datafls)
y <- datafls$y
x <- datafls[, -which(names(datafls) == "y")]
```

From the correlation plots above, we take the variable with the top 5 absolute highest correlation with the response y.

```
top5_idx <- order(abs(cors), decreasing = TRUE)[1:5]
top5_vars <- rownames(cors)[top5_idx]

cat("Top 5 predictors by |correlation| with y:\n")
```

```
## Top 5 predictors by |correlation| with y:
```

```
print(data.frame(
  Variable    = top5_vars,
  Correlation = round(cors[top5_idx], 4)
))
```

```
##    Variable Correlation
## 1  EquipInv      0.6361
## 2   YrsOpen      0.6026
## 3 Confucian      0.5124
## 4 PrExports     -0.5091
## 5 RuleofLaw      0.5026
```

```
top5_vars <- c("EquipInv", "YrsOpen", "Confucian", "PrExports", "RuleofLaw")
```

Since we are taking the top 5 we will have a 72×56 data matrix with the extra 15 newly engineered variables.

```r
x_top5          <- x[, top5_vars]
interact_df     <- as.data.frame(model.matrix(~ .^2 - 1, data = x_top5))
interaction_cols <- interact_df[, !names(interact_df) %in% top5_vars]

cat("\nInteraction terms created:", ncol(interaction_cols), "\n")
```

```
##
## Interaction terms created: 10
```

```r
cat("Interaction term names:\n")
```

```
## Interaction term names:
```

```r
print(names(interaction_cols))
```

```
##  [1] "EquipInv:YrsOpen"    "EquipInv:Confucian"  "EquipInv:PrExports"
##  [4] "EquipInv:RuleofLaw"  "YrsOpen:Confucian"   "YrsOpen:PrExports"
##  [7] "YrsOpen:RuleofLaw"   "Confucian:PrExports" "Confucian:RuleofLaw"
## [10] "PrExports:RuleofLaw"
```

```r
poly_df <- as.data.frame(
  lapply(top5_vars, function(var) {
    df        <- data.frame(x[[var]]^2)
    names(df) <- paste0(var, "_sq")
    return(df)
  })
)

cat("\nPolynomial features created:", ncol(poly_df), "\n")
```

```
##
## Polynomial features created: 5
```

```r
cat("Polynomial feature names:\n")
```

```
## Polynomial feature names:
```

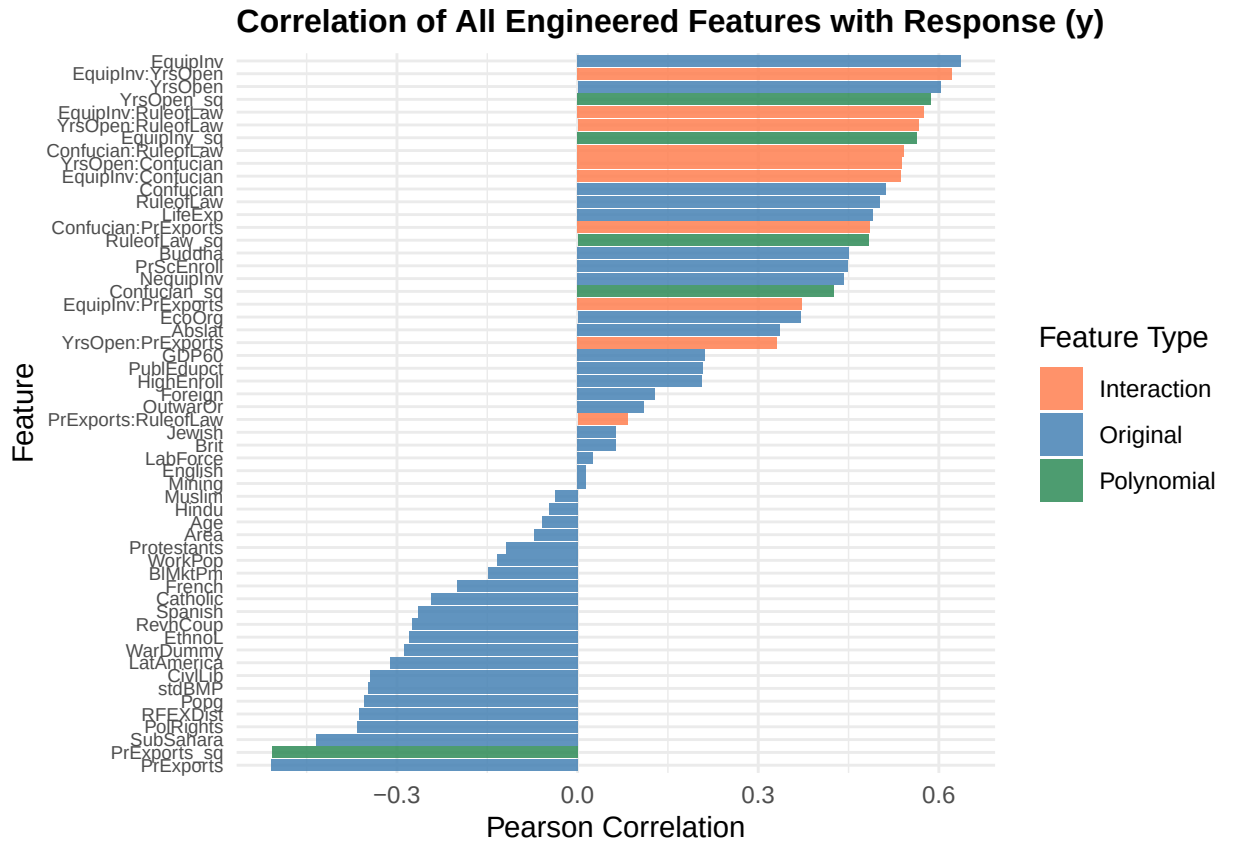
```r
print(names(poly_df))
```

```
## [1] "EquipInv_sq"  "YrsOpen_sq"   "Confucian_sq" "PrExports_sq" "RuleofLaw_sq"
```

```r
x_engineered <- cbind(x, interaction_cols, poly_df)
```

```r
cors_eng <- cor(x_engineered, y)
cor_df_eng <- data.frame(
  Variable    = rownames(cors_eng),
  Correlation = as.numeric(cors_eng)
)
```

```r
cor_df_eng <- cor_df_eng[order(cor_df_eng$Correlation, decreasing = TRUE), ]


original_vars    <- names(x)
interaction_vars <- names(interaction_cols)
poly_vars        <- names(poly_df)

cor_df_eng$Type <- ifelse(
  cor_df_eng$Variable %in% interaction_vars, "Interaction",
  ifelse(cor_df_eng$Variable %in% poly_vars, "Polynomial", "Original")
)

# Bar chart coloured by feature type
ggplot(cor_df_eng, aes(x = reorder(Variable, Correlation),
                       y = Correlation,
                       fill = Type)) +
  geom_bar(stat = "identity", alpha = 0.85) +
  scale_fill_manual(values = c(
    "Original"    = "steelblue",
    "Interaction" = "coral",
    "Polynomial"  = "seagreen"
  )) +
  coord_flip() +
  labs(title = "Correlation of All Engineered Features with Response (y)",
       x     = "Feature",
       y     = "Pearson Correlation",
       fill  = "Feature Type") +
  theme_minimal() +
  theme(
    axis.text.y = element_text(size = 7),
    plot.title  = element_text(size = 12, face = "bold")
  )
```




```r
#  (interactions + polynomials)
new_features <- cor_df_eng[cor_df_eng$Type != "Original", ]

ggplot(new_features, aes(x = reorder(Variable, Correlation),
                         y = Correlation,
                         fill = Type)) +
  geom_bar(stat = "identity", alpha = 0.85) +
  scale_fill_manual(values = c(
    "Interaction" = "coral",
    "Polynomial"  = "seagreen"
  )) +
  coord_flip() +
  labs(title = "Correlation of New Engineered Features with Response (y)",
       x     = "Feature",
       y     = "Pearson Correlation",
       fill  = "Feature Type") +
  theme_minimal() +
  theme(
    axis.text.y = element_text(size = 9),
    plot.title  = element_text(size = 12, face = "bold")
  )
```


```
#  Print summary table of new features only
cat("=== Correlation of New Engineered Features with y ===\n")
```

```
## === Correlation of New Engineered Features with y ===
```

```
print(new_features[, c("Variable", "Correlation", "Type")])
```

```
##              Variable Correlation        Type
## 42    EquipInv:YrsOpen   0.6214511 Interaction
## 53          YrsOpen_sq   0.5864558  Polynomial
## 45  EquipInv:RuleofLaw   0.5749434 Interaction
## 48   YrsOpen:RuleofLaw   0.5662395 Interaction
## 52         EquipInv_sq   0.5641047  Polynomial
## 50 Confucian:RuleofLaw   0.5421559 Interaction
## 46   YrsOpen:Confucian   0.5392198 Interaction
## 43  EquipInv:Confucian   0.5374860 Interaction
## 49 Confucian:PrExports   0.4861810 Interaction
## 56        RuleofLaw_sq   0.4831797  Polynomial
## 54        Confucian_sq   0.4253941  Polynomial
## 44  EquipInv:PrExports   0.3731315 Interaction
## 47   YrsOpen:PrExports   0.3306112 Interaction
## 51 PrExports:RuleofLaw   0.0830349 Interaction
## 55        PrExports_sq  -0.5077514  Polynomial
```

E2.A.2b Standardize features for SVR (explain why this is crucial).

Standardizing features for SVR is important because we have different measurements and possible ranges in the predictor variables. This could lead to distance being dominated by variables with larger ranges. As a example below, we see that `max(LabForce)`$= 195781.5 >>$ `max(Mining)` $= 0.533$.We can easily show that if the kernel is dominated by one feature the other features will have no influence on the predictions at all, hence we must standardize the variables before using SVR.

```
range(x$Mining)
```

```
## [1] 0.000 0.533
```

```
range(x$LabForce)
```

```
## [1]    210.9185 195781.5000
```

```
range(x$RuleofLaw)
```

```
## [1] 0 1
```

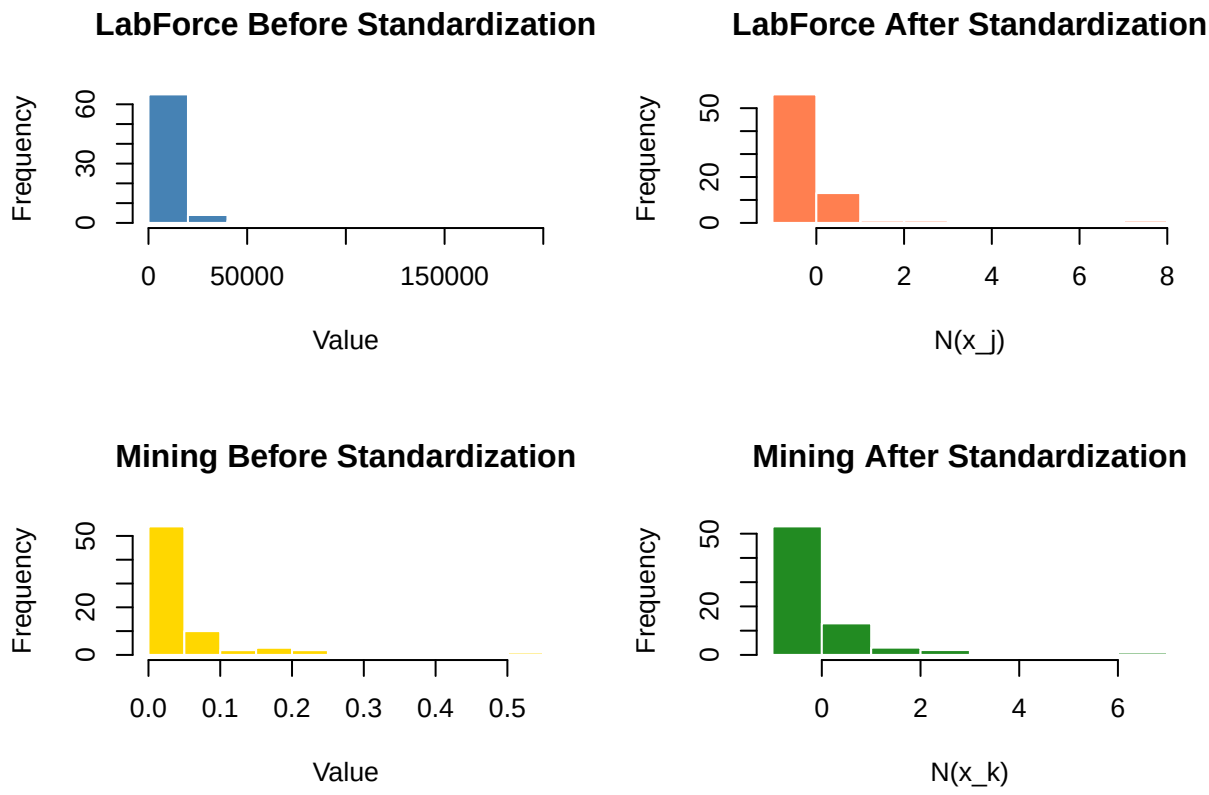
```
x_means <- colMeans(x_engineered)
x_sds   <- apply(x_engineered, 2, sd)
zero_sd <- x_sds == 0
if (any(zero_sd)) {
  cat(names(x_sds)[zero_sd], "\n")
  x_engineered <- x_engineered[, !zero_sd]
  x_means      <- x_means[!zero_sd]
  x_sds        <- x_sds[!zero_sd]
}


x_scaled <- as.data.frame(
  sweep(sweep(x_engineered, 2, x_means, "-"), 2, x_sds, "/")
)


y_mean   <- mean(y)
y_sd     <- sd(y)
y_scaled <- (y - y_mean) / y_sd
```

We plot the `LabForce` and `Mining` variable's distribution to check that the linear transformation did not change the distribution but just scales it accordingly.

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}$$

```
par(mfrow = c(2, 2))

hist(x_engineered$LabForce,
     main   = "LabForce Before Standardization",
     xlab   = "Value",
     col    = "steelblue",
     border = "white")
```

```r
hist(x_scaled$LabForce,
     main   = "LabForce After Standardization",
     xlab   = "N(x_j)",
     col    = "coral",
     border = "white")

hist(x_engineered$Mining,
     main   = "Mining Before Standardization",
     xlab   = "Value",
     col    = "gold",
     border = "white")

hist(x_scaled$Mining,
     main   = "Mining After Standardization",
     xlab   = "N(x_k)",
     col    = "forestgreen",
     border = "white")
```




```r
range(x_scaled$Mining)
```

```
## [1] -0.5824638  6.3357131
```

```r
range(x_scaled$LabForce)
```

```
## [1] -0.3651504  7.4871800
```

E2.A.2c Split data into training (70%), validation (15%), and test (15%) sets. If we wanted to interpret for the original value of y, we would perform the inverse transformation

$$y_{original} = y_{scaled} * 0.01825391 + 0.02072847$$

```r
set.seed(547)
n         <- nrow(x_scaled)
train_idx <- sample(1:n, floor(0.70 * n))
remain    <- setdiff(1:n, train_idx)
val_idx   <- sample(remain, floor(0.15 * n))
test_idx  <- setdiff(remain, val_idx)

x_train <- x_scaled[train_idx, ]
x_val   <- x_scaled[val_idx,   ]
x_test  <- x_scaled[test_idx,  ]

y_train <- y_scaled[train_idx]
y_val   <- y_scaled[val_idx]
y_test  <- y_scaled[test_idx]

cat("\n=== Split Summary ===\n")
```

```
##
## === Split Summary ===
```

```r
cat("Train:", length(train_idx), "observations (",
    round(length(train_idx)/n*100), "%)\n")
```

```
## Train: 50 observations ( 69 %)
```

```r
cat("Val  :", length(val_idx),   "observations (",
    round(length(val_idx)/n*100),   "%)\n")
```

```
## Val  : 10 observations ( 14 %)
```

```r
cat("Test :", length(test_idx),  "observations (",
    round(length(test_idx)/n*100),  "%)\n")
```

```
## Test : 12 observations ( 17 %)
```

```r
cat("\nFinal dimensions:\n")
```

```
##
## Final dimensions:
```

```r
cat("x_train:", dim(x_train), "\n")
```

```
## x_train: 50 56
```

```r
cat("x_val  :", dim(x_val),   "\n")
```

```
## x_val  : 10 56
```

```r
cat("x_test :", dim(x_test),  "\n")
```

```
## x_test : 12 56
```

```r
cat("\n=== Back-transformation ===\n")
```

```
##
## === Back-transformation ===
```

```r
cat("y_pred_original = y_pred_scaled *", y_sd,
    "+",y_mean, "\n")
```

```
## y_pred_original = y_pred_scaled * 0.01825391 + 0.02072847
```

3. Baseline Models

E2.A.3a Fit a linear regression baseline. We regress $y_{train} \subsetneq y_{scaled}$ on all the predictor variables $x_{train} \subsetneq x_{scaled}$. We perform Ridge Regression since we have $\kappa < 1$ problem and our data matrix is under-determined.

```r
dim(x_train)
```

```
## [1] 50 56
```

```r
(kappa_1 <- nrow(x_train)/ncol(x_train))
```

```
## [1] 0.8928571
```

```r
x_train_mat <- as.matrix(x_train)
x_val_mat   <- as.matrix(x_val)
x_test_mat  <- as.matrix(x_test)


y_train <- as.numeric(y_train)
y_val   <- as.numeric(y_val)
y_test  <- as.numeric(y_test)
```

```r
cv_ridge <- cv.glmnet(x_train_mat, y_train, alpha = 0, nfolds = 5)
lambda_min <- cv_ridge$lambda.min
ridge_model <- glmnet(x_train_mat, y_train, alpha = 0, lambda = lambda_min, scale= FALSE)


pred_val_ridge_scaled <- predict(ridge_model, newx = x_val_mat, s = lambda_min)

pred_val_ridge_orig <- pred_val_ridge_scaled * y_sd + y_mean
y_val_orig <- y_val * y_sd + y_mean
```

E2.A.3b Fit a single decision tree (no pruning).

```
tree_unpruned <- rpart(y_train ~ ., data = data.frame(x_train_mat, y_train),
                       method = "anova", control = rpart.control(cp = 0, minsplit = 2))
pred_val_tree_scaled <- predict(tree_unpruned, newdata = data.frame(x_val_mat))
pred_val_tree_orig <- pred_val_tree_scaled * y_sd + y_mean
```

E2.A.3c Report baseline performance on validation set.

```
cat("\n=== Baseline Models (Validation Set, original scale) ===\n")
```

```
##
## === Baseline Models (Validation Set, original scale) ===
```

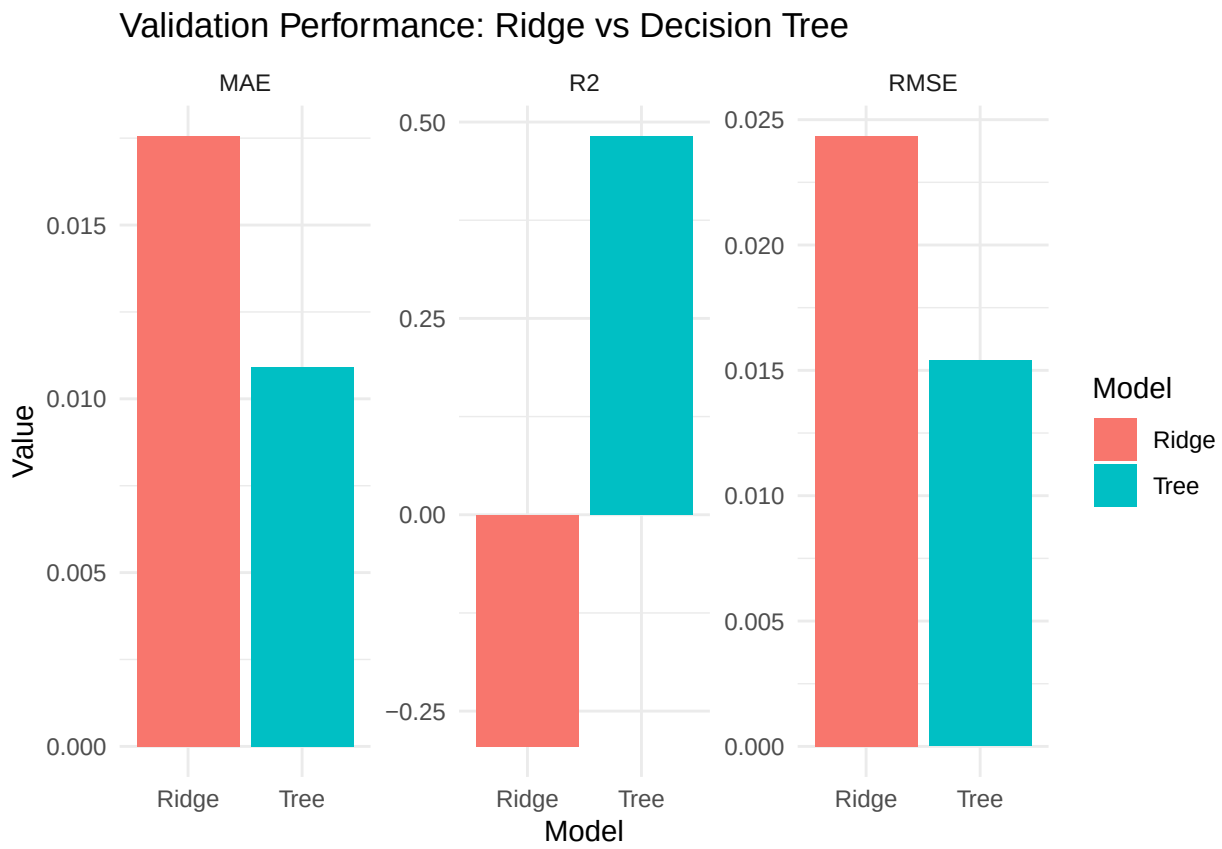
```
get_metrics(y_val_orig, pred_val_ridge_orig, label = "Ridge Regression")
```

```
## Ridge Regression
##   RMSE: 0.024377
##   MAE : 0.017581
##   R2  : -0.298413

##      RMSE       MAE        R2
##  0.024377  0.017581 -0.298413
```

```
get_metrics(y_val_orig, pred_val_tree_orig, label = "Unpruned Decision Tree")
```

```
## Unpruned Decision Tree
##   RMSE: 0.015396
##   MAE : 0.010922
##   R2  : 0.482047

##     RMSE      MAE       R2
## 0.015396 0.010922 0.482047
```

```
results <- data.frame(
  Model = c("Ridge", "Tree"),
  RMSE = c(0.024343, 0.015396),
  MAE  = c(0.017558, 0.010922),
  R2   = c(-0.294891, 0.482047)
)

results_long <- results %>%
  pivot_longer(-Model, names_to = "Metric", values_to = "Value")

ggplot(results_long, aes(Model, Value, fill = Model)) +
  geom_col(position = "dodge") +
  facet_wrap(~Metric, scales = "free_y") +
  theme_minimal() +
  ggtitle("Validation Performance: Ridge vs Decision Tree")
```



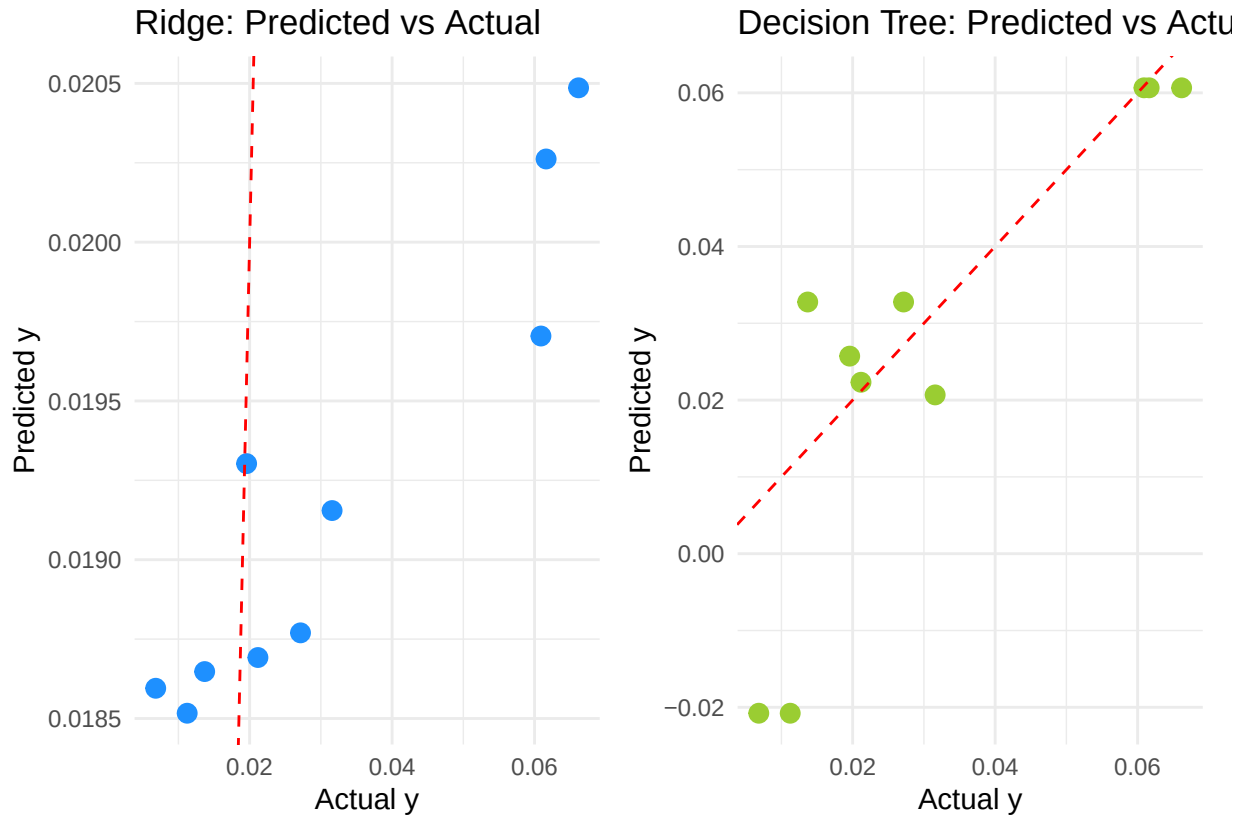
The $R^2_{Ridge} < 0$ because $SSR/SST \approx 1.29$, this is a high dimensional regression and it tells us that a linear model will fail as the model error is much greater than the mean model error. In essence, The ridge model predicts worse than the constant mean predictor on the validation set

More Visualizations *Ridge Sounds like Blue for me and Tree is Green by nature.*

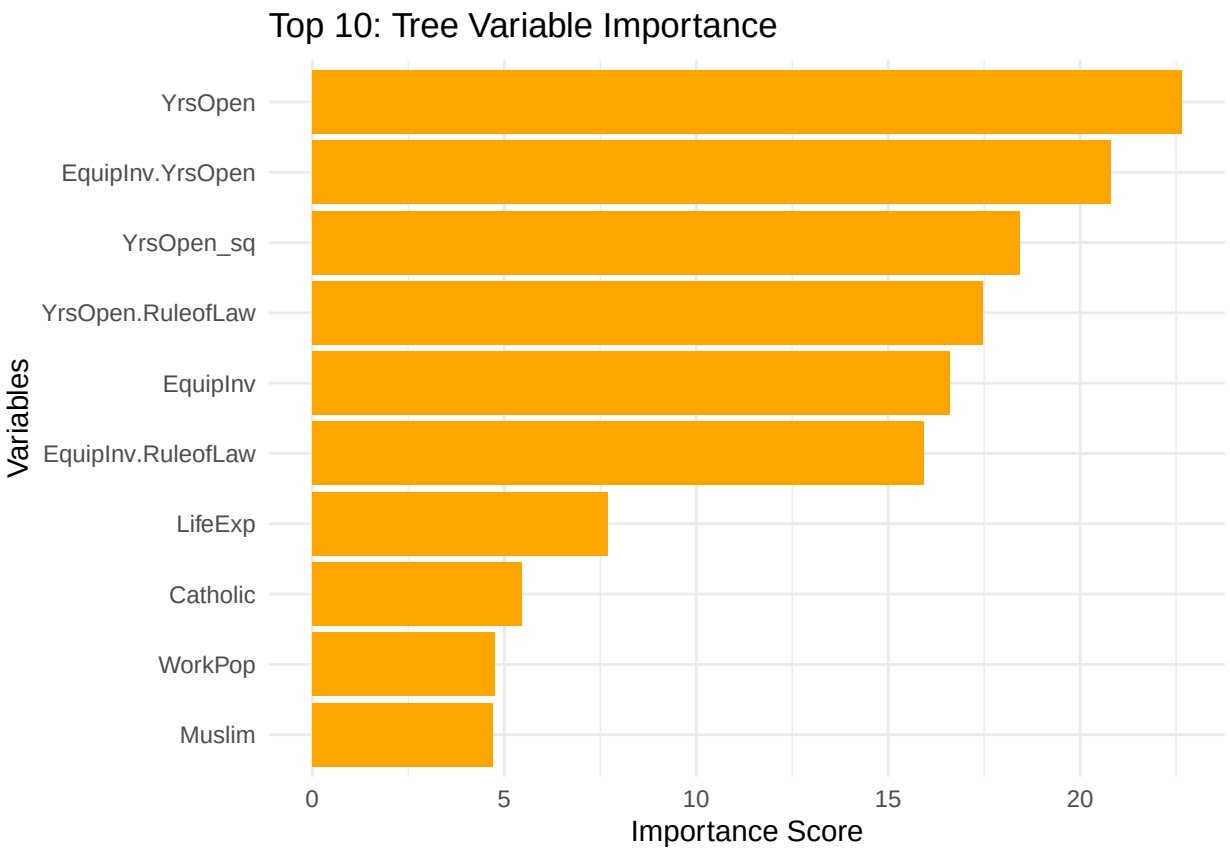
```
 plot_df <- data.frame(
  y_true = y_val_orig,
  Ridge  = as.numeric(pred_val_ridge_orig),
  Tree   = as.numeric(pred_val_tree_orig)
)
```

```
p1 <- ggplot(plot_df, aes(y_true, Ridge)) +
  geom_point(color = "dodgerblue", size = 3) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "red") +
  theme_minimal() +
  labs(title = "Ridge: Predicted vs Actual", x = "Actual y", y = "Predicted y")


p2 <- ggplot(plot_df, aes(y_true, Tree)) +
  geom_point(color = "yellowgreen", size = 3) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "red") +
  theme_minimal() +
  labs(title = "Decision Tree: Predicted vs Actual", x = "Actual y", y = "Predicted y")
```

```
p1 + p2
```




```
vip <- data.frame(
  Variable = names(tree_unpruned$variable.importance),
  Importance = as.numeric(tree_unpruned$variable.importance)
) %>%
  arrange(desc(Importance)) %>%
  head(10)


ggplot(vip, aes(x = reorder(Variable, Importance), y = Importance)) +
  geom_col(fill = "orange") + # Fixed: Use fill = "orange"
  coord_flip() +
  theme_minimal() +
  labs(
    title = "Top 10: Tree Variable Importance",
    x = "Variables",
    y = "Importance Score"
  )
```



Part B. Support Vector Regression

The following parts for 1 and 2 is done in a mixed order due to convenience and the limitation of my coding abilities.

1. Model Training E2.B.1a Train SVR models with linear, polynomial, and RBF kernels. **E2.B.1b** Use cross-validation to tune hyperparameters (C, ϵ, kernel parameters). **E2.B.1c** Plot validation performance vs. hyperparameter values.

```
tune_svr <- function(kernel, param_grid, x_tr, y_tr, x_v, y_v) {
  results <- list()
  for (i in 1:nrow(param_grid)) {
    if (kernel == "linear") {
      model <- svm(x_tr, y_tr, kernel = "linear",
                   cost = param_grid$C[i], epsilon = param_grid$epsilon[i],
                   scale = FALSE)
    } else if (kernel == "polynomial") {
      model <- svm(x_tr, y_tr, kernel = "polynomial",
                   cost = param_grid$C[i], epsilon = param_grid$epsilon[i],
                   degree = param_grid$degree[i], coef0 = param_grid$coef0[i],
                   scale = FALSE)
    } else if (kernel == "radial") {
```

```
      model <- svm(x_tr, y_tr, kernel = "radial",
                   cost = param_grid$C[i], epsilon = param_grid$epsilon[i],
                   gamma = param_grid$gamma[i],
                   scale = FALSE)
    }
    pred <- predict(model, x_v)
    rmse <- sqrt(mean((y_v - pred)^2))
    results[[i]] <- c(param_grid[i, ], RMSE = rmse)
  }
  return(do.call(rbind, results))
}
```

```
# Tune hyperparameters using validation set
# RBF kernel
grid_rbf <- expand.grid(C = c(0.1, 1, 10, 100),
                        epsilon = c(0.01, 0.05, 0.1, 0.5),
                        gamma = c(0.01, 0.1, 1, 10))
rbf_tune <- tune_svr("radial", grid_rbf, x_train_mat, y_train, x_val_mat, y_val)
best_rbf <- rbf_tune[which.min(rbf_tune[, "RMSE"]), ]

# Linear kernel
grid_lin <- expand.grid(C = c(0.1, 1, 10, 100),
                        epsilon = c(0.01, 0.05, 0.1, 0.5))
lin_tune <- tune_svr("linear", grid_lin, x_train_mat, y_train, x_val_mat, y_val)
best_lin <- lin_tune[which.min(lin_tune[, "RMSE"]), ]

# Polynomial kernel
grid_poly <- expand.grid(C = c(0.1, 1, 10),
                         epsilon = c(0.01, 0.1),
                         degree = c(2, 3),
                         coef0 = c(0, 1))
poly_tune <- tune_svr("polynomial", grid_poly, x_train_mat, y_train, x_val_mat, y_val)
best_poly <- poly_tune[which.min(poly_tune[, "RMSE"]), ]
```

```
# Train final models
svr_lin <- svm(x_train_mat, y_train, kernel = "linear",
               cost = best_lin$C, epsilon = best_lin$epsilon, scale = FALSE)
svr_poly <- svm(x_train_mat, y_train, kernel = "polynomial",
                cost = best_poly$C, epsilon = best_poly$epsilon,
                degree = best_poly$degree, coef0 = best_poly$coef0, scale = FALSE)
svr_rbf <- svm(x_train_mat, y_train, kernel = "radial",
               cost = best_rbf$C, epsilon = best_rbf$epsilon,
               gamma = best_rbf$gamma, scale = FALSE)

# Test set predictions (back-transformed)
pred_test_lin <- predict(svr_lin, x_test_mat) * y_sd + y_mean
pred_test_poly <- predict(svr_poly, x_test_mat) * y_sd + y_mean
pred_test_rbf <- predict(svr_rbf, x_test_mat) * y_sd + y_mean
y_test_orig <- y_test * y_sd + y_mean
```

2. Model Evaluation

```r
cat("\n=== SVR Test Performance (original scale) ===\n")
```

```
##
## === SVR Test Performance (original scale) ===
```

```r
get_metrics(y_test_orig, pred_test_lin, label = "Linear SVR")
```

```
## Linear SVR
##   RMSE: 0.007898
##   MAE : 0.00608
##   R2  : 0.690115

##     RMSE      MAE       R2
## 0.007898 0.006080 0.690115
```

```r
get_metrics(y_test_orig, pred_test_poly, label = "Polynomial SVR")
```

```
## Polynomial SVR
##   RMSE: 0.006776
##   MAE : 0.004933
##   R2  : 0.771937

##     RMSE      MAE       R2
## 0.006776 0.004933 0.771937
```

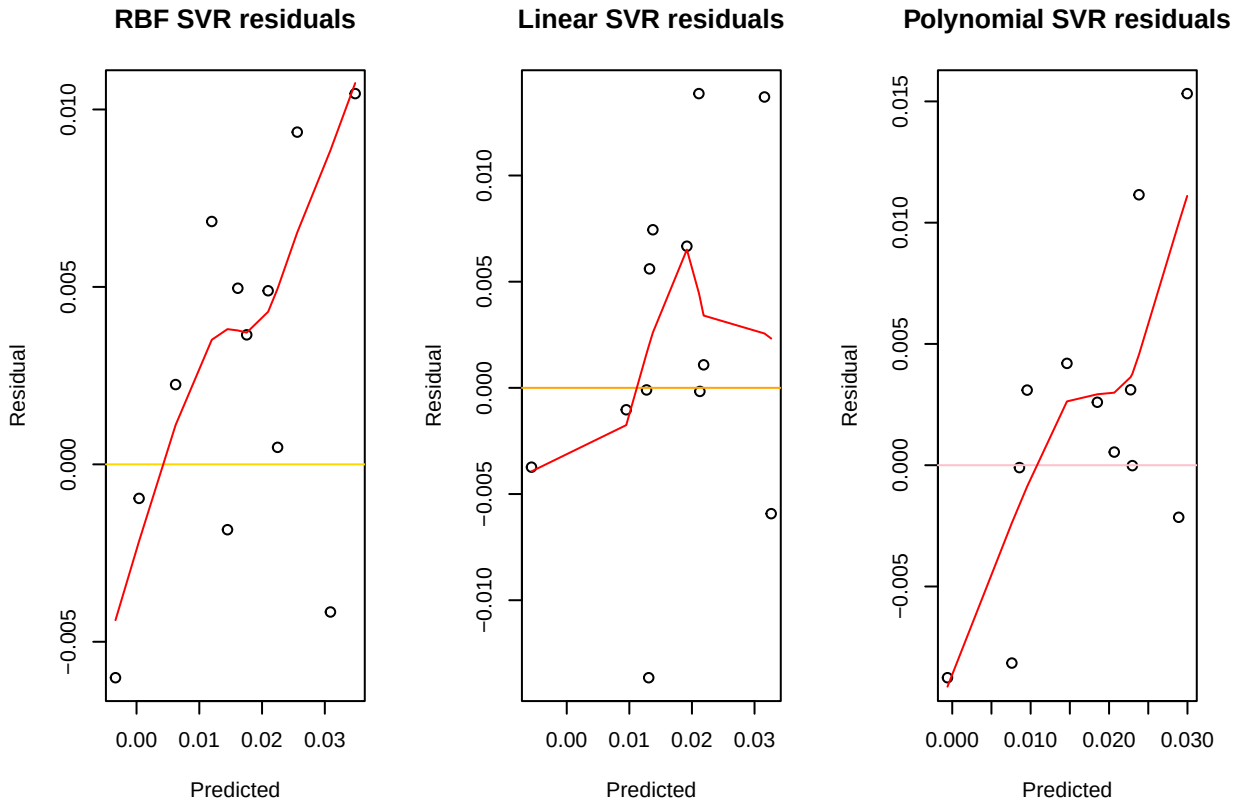
```r
get_metrics(y_test_orig, pred_test_rbf, label = "RBF SVR")
```

```
## RBF SVR
##   RMSE: 0.005542
##   MAE : 0.004655
##   R2  : 0.847425

##     RMSE      MAE       R2
## 0.005542 0.004655 0.847425
```

E2.B.2a Compare test performance of different kernels. **E2.B.2b** Analyze residuals and identify patterns. We first plot the residuals of each SVR model. We know that a well-fitted model should have residuals that look like random noise scattered evenly around the zero line, the strong upward linear trend in these plots indicates that none of these models are fully capturing the underlying relationship in the data.We further plotted the lowess lines for each graph to explicitly show that the residuals are not randomly scattered. It is possible the error is due to some systematic pattern that we are not aware at the moment.

```r
# Residual plots
#View(resids_all)
par(mfrow = c(1,3))
resid_rbf <- y_test_orig - pred_test_rbf
plot(pred_test_rbf, resid_rbf, main = "RBF SVR residuals",
     xlab = "Predicted", ylab = "Residual")
abline(h = 0, col = "gold")
```

```r
lines(lowess(pred_test_rbf, resid_rbf), col = "red")

resid_lin <- y_test_orig - pred_test_lin
plot(pred_test_lin, resid_lin, main = "Linear SVR residuals",
     xlab = "Predicted", ylab = "Residual")
abline(h = 0, col = "orange")
lines(lowess(pred_test_lin, resid_lin), col = "red")

resid_poly <- y_test_orig - pred_test_poly
plot(pred_test_poly, resid_poly, main = "Polynomial SVR residuals",
     xlab = "Predicted", ylab = "Residual")
abline(h = 0, col = "pink")
lines(lowess(pred_test_poly, resid_poly), col = "red")
```



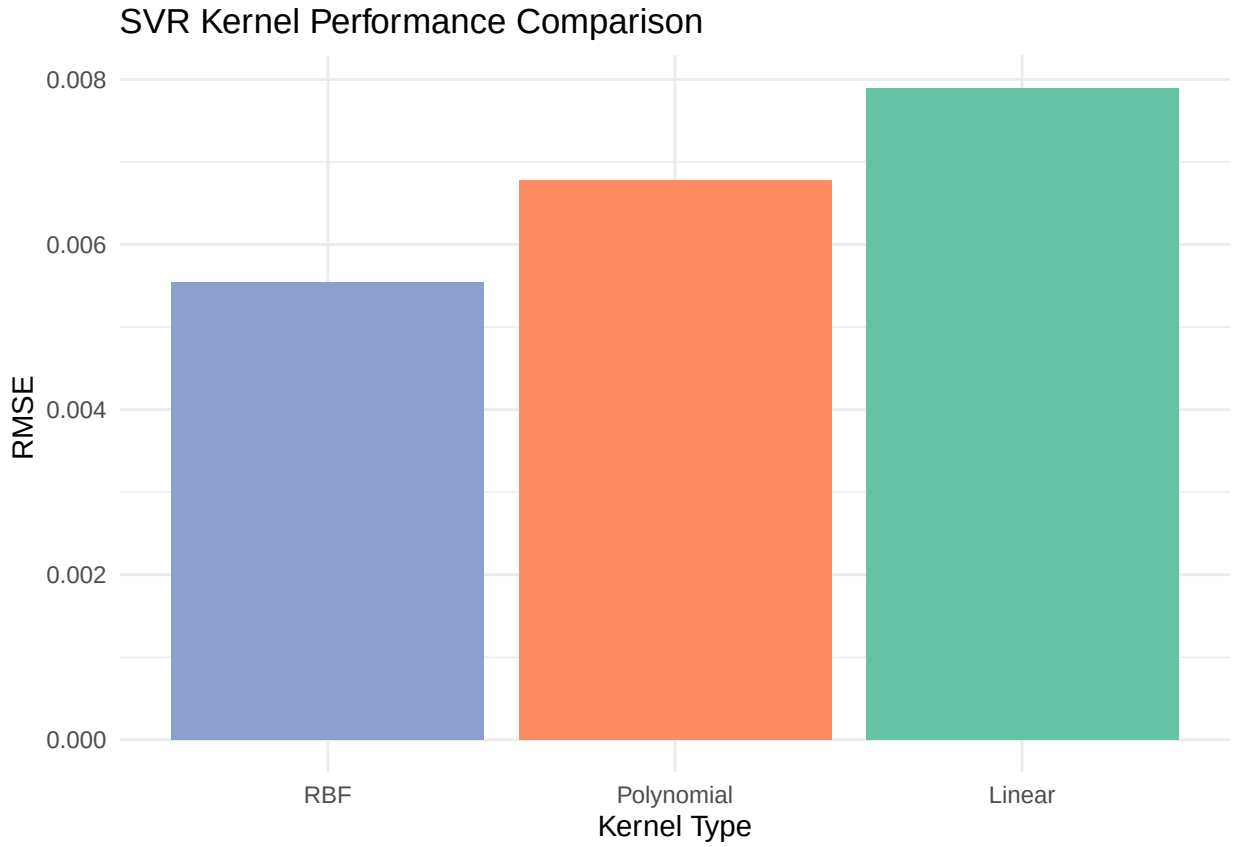
More Visuals for Understanding

$$RMSE = \sqrt{\frac{1}{n}\sum_{1\leq i\leq n}(y_i - \hat{f}_{SVR_i})^2}$$

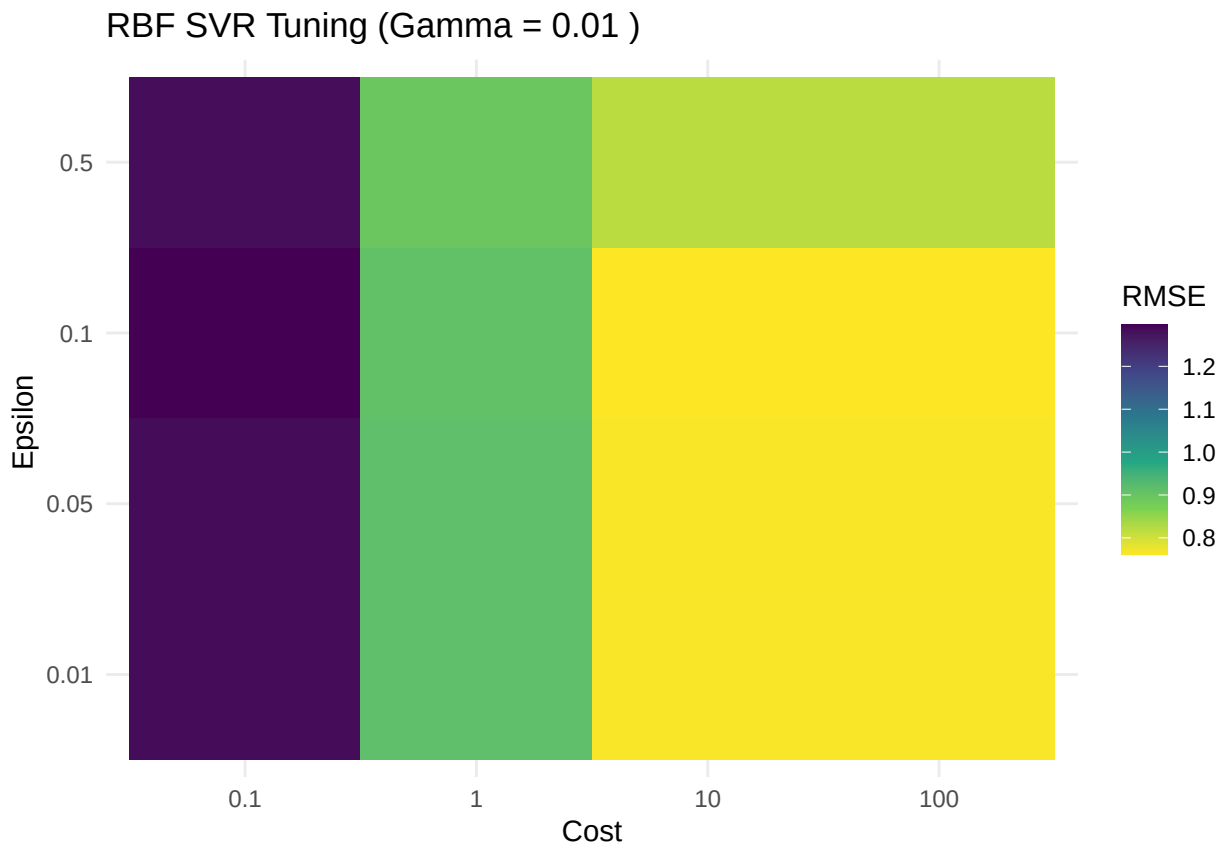
From the RMSE Metric RBF wins!

```r
metrics_df <- data.frame(
  Kernel = c("Linear", "Polynomial", "RBF"),
  RMSE = c(sqrt(mean((y_test_orig - pred_test_lin)^2)),
           sqrt(mean((y_test_orig - pred_test_poly)^2)),
```

```
          sqrt(mean((y_test_orig - pred_test_rbf)^2)))
)

# Plot the RMSE comparison
ggplot(metrics_df, aes(x = reorder(Kernel, RMSE), y = RMSE, fill = Kernel)) +
  geom_col(show.legend = FALSE) +
  scale_fill_brewer(palette = "Set2") +
  theme_minimal() +
  labs(title = "SVR Kernel Performance Comparison",
       x = "Kernel Type", y = "RMSE ")
```

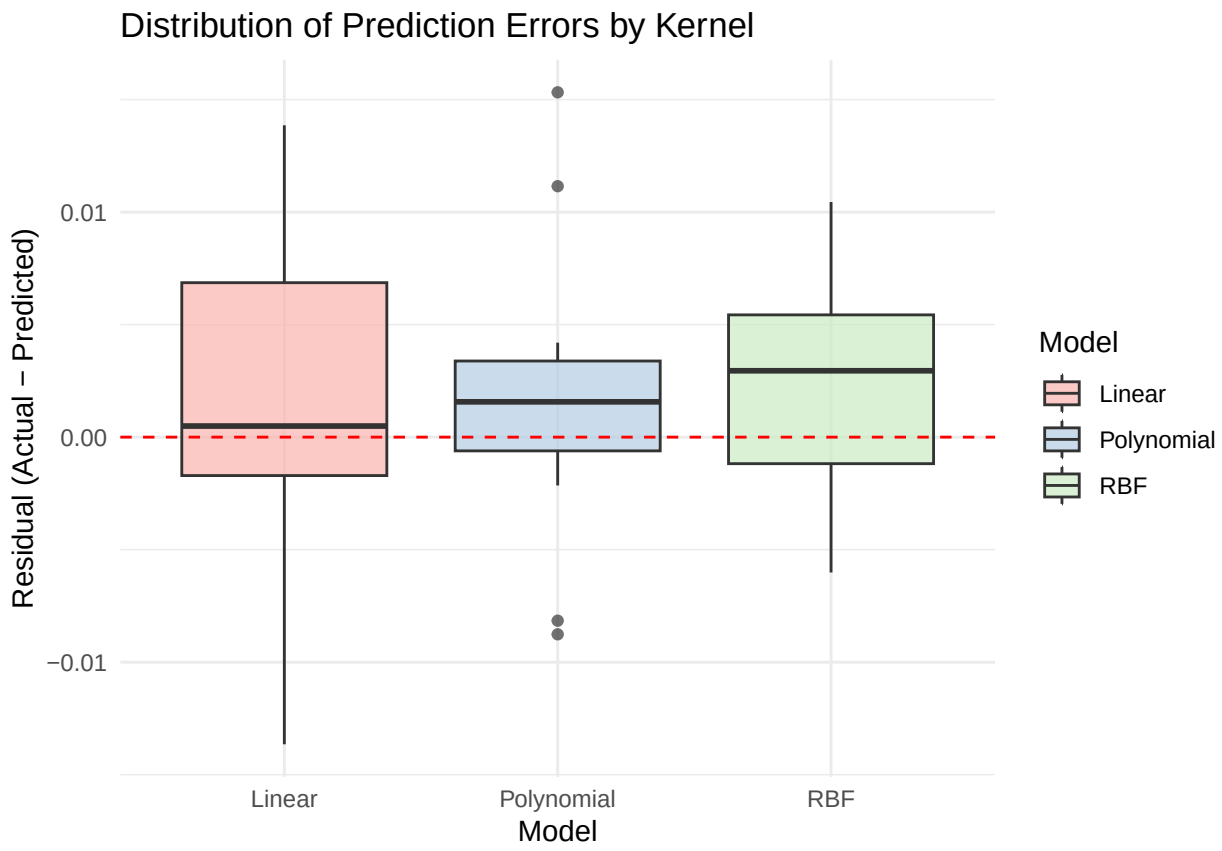



```
rbf_plot_df <- as.data.frame(rbf_tune)
rbf_plot_df[] <- lapply(rbf_plot_df, unlist)


best_gamma_val <- unlist(best_rbf$gamma)

ggplot(rbf_plot_df[rbf_plot_df$gamma == best_gamma_val, ],
       aes(x = factor(C), y = factor(epsilon), fill = RMSE)) +
  geom_tile() +
  scale_fill_viridis_c(direction = -1) +
  labs(title = paste("RBF SVR Tuning (Gamma =", best_gamma_val, ")"),
       x = "Cost", y = "Epsilon ",
       fill = "RMSE") +
  theme_minimal()
```

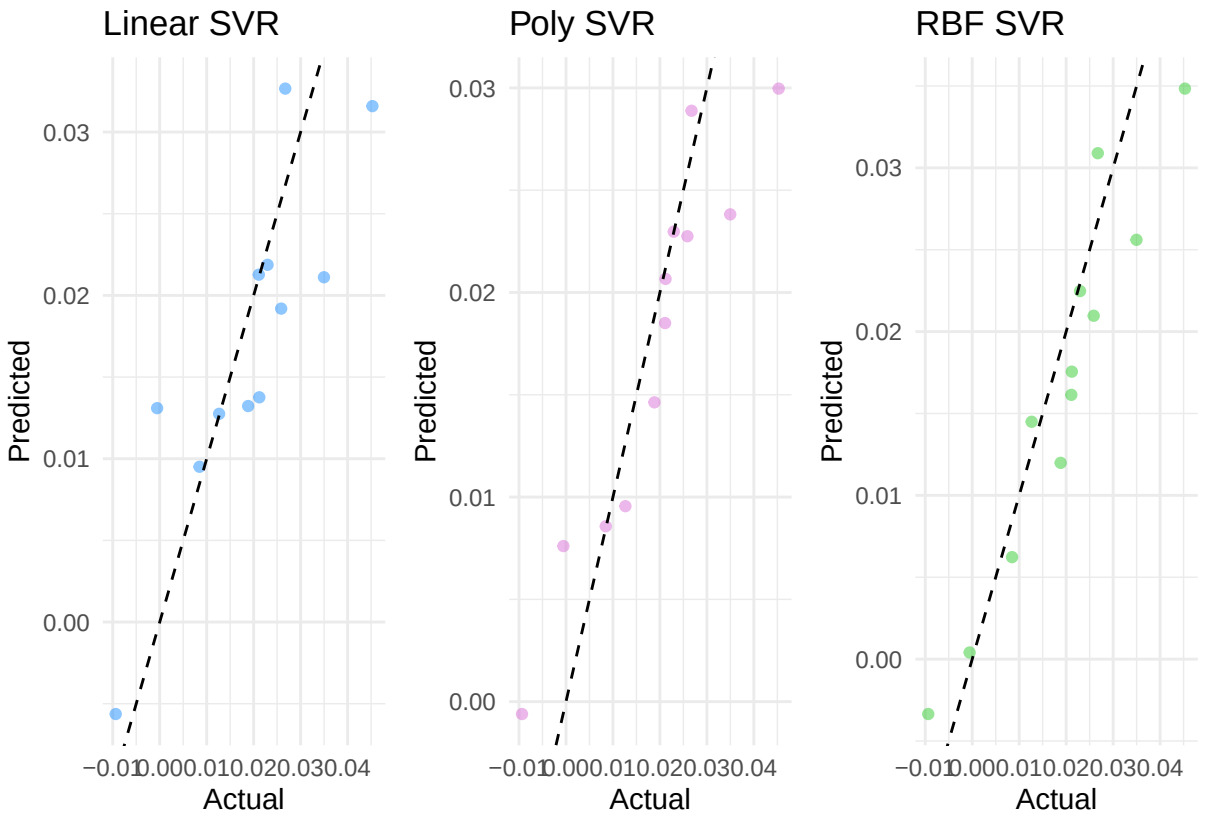


The yellow region is where the best models lies because it minimizes the RMSE.

```
resids_all <- data.frame(
  Model = rep(c("Linear", "Polynomial", "RBF"), each = length(y_test_orig)),
  Residual = c(y_test_orig - pred_test_lin,
               y_test_orig - pred_test_poly,
               y_test_orig - pred_test_rbf)
)

ggplot(resids_all, aes(x = Model, y = Residual, fill = Model)) +
  geom_boxplot(alpha = 0.7) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  scale_fill_brewer(palette = "Pastel1") +
  theme_minimal() +
  labs(title = "Distribution of Prediction Errors by Kernel",
       y = "Residual (Actual - Predicted)")
```




```
plot_svr_fit <- function(actual, pred, title, color) {
  ggplot(data.frame(actual, pred), aes(x = actual, y = pred)) +
    geom_point(color = color, alpha = 0.5) +
    geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "black") +
    theme_minimal() +
    labs(title = title, x = "Actual", y = "Predicted")
}

p_lin <- plot_svr_fit(y_test_orig, pred_test_lin, "Linear SVR", "dodgerblue")
p_poly <- plot_svr_fit(y_test_orig, pred_test_poly, "Poly SVR", "orchid")
p_rbf <- plot_svr_fit(y_test_orig, pred_test_rbf, "RBF SVR", "limegreen")

p_lin + p_poly + p_rbf
```



E2.B.2c Discuss computational requirements of different kernels.

Table 1: Summary Comparison of Computational Requirements by Kernel

Kernel	Time Complexity (Training)	Memory Complexity	Scalability (n)	Scalability (d)
Linear	$(O(n \times d))$	$(O(d))$	Excellent	Excellent
Polynomial	$(O(n^2 \times d))$	$(O(n^2))$	Poor	Moderate
RBF	$(O(n^2 \times d))$	$(O(n^2))$	Poor	Moderate

3. Interpretation

E2.B.3a How many support vectors were selected? Number of support vectors: Linear: 44 / 50 Polynomial: 50 / 50 RBF: 45 / 50

$$\hat{f}(x) = \sum_{i \in \text{SV}} (\alpha_i - \alpha_i^*) K(x_i, x) + b$$

```
cat("\nNumber of support vectors:\n")
```

```
##
## Number of support vectors:
```

```
cat("Linear:", nrow(svr_lin$SV), "/", nrow(x_train_mat), "\n")
```

```
## Linear: 44 / 50
```

```r
cat("Polynomial:", nrow(svr_poly$SV), "/", nrow(x_train_mat), "\n")
```

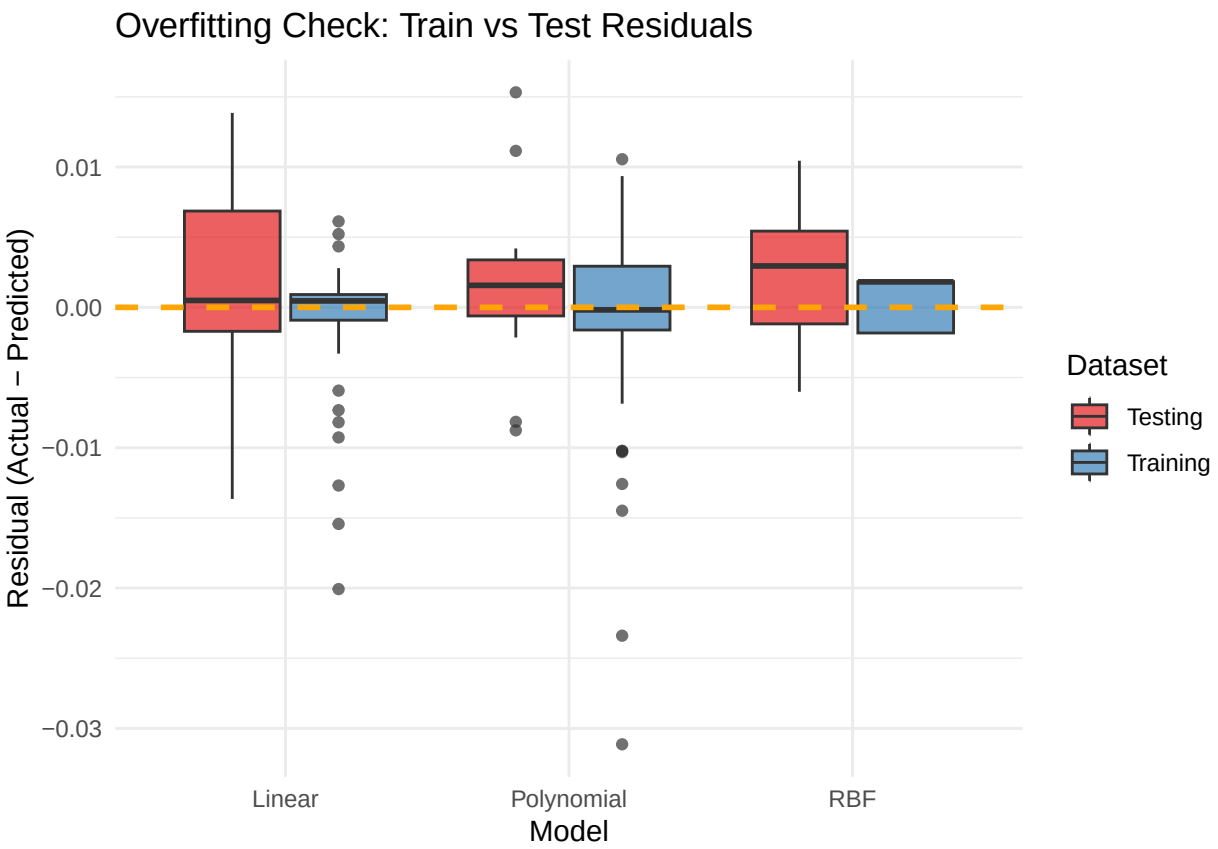
```
## Polynomial: 50 / 50
```

```r
cat("RBF:", nrow(svr_rbf$SV), "/", nrow(x_train_mat), "\n")
```

```
## RBF: 45 / 50
```

E2.B.3b What does this tell you about the complexity of the solution?

```r
pred_train_lin <- predict(svr_lin, x_train_mat) * y_sd + y_mean
pred_train_poly <- predict(svr_poly, x_train_mat) * y_sd + y_mean
pred_train_rbf <- predict(svr_rbf, x_train_mat) * y_sd + y_mean


y_train_orig <- y_train * y_sd + y_mean

resids_train <- data.frame(
  Model = rep(c("Linear", "Polynomial", "RBF"), each = length(y_train_orig)),
  Dataset = "Training",
  Residual = c(y_train_orig - pred_train_lin,
               y_train_orig - pred_train_poly,
               y_train_orig - pred_train_rbf)
)


resids_test <- data.frame(
  Model = rep(c("Linear", "Polynomial", "RBF"), each = length(y_test_orig)),
  Dataset = "Testing",
  Residual = c(y_test_orig - pred_test_lin,
               y_test_orig - pred_test_poly,
               y_test_orig - pred_test_rbf)
)


resids_combined <- rbind(resids_train, resids_test)


ggplot(resids_combined, aes(x = Model, y = Residual, fill = Dataset)) +
  geom_boxplot(alpha = 0.7) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "orange", linewidth = 1) +
  scale_fill_brewer(palette = "Set1") +
  theme_minimal() +
  labs(title = "Overfitting Check: Train vs Test Residuals",
       y = "Residual (Actual - Predicted)")
```



Seeing that we have so many non-zero support vectors, nearly every single data point and especially for `svr_polySV`, exactly 100% of the support vectors is non-zero. This means that the solution is overly complex, so the model is effectively connecting the dots and memorizing the training data set rather than discovering a generalization trend. If we plot the Training and Test Residuals we can see that the Testing box plots for the linear kernel yields the greatest variation even though it is less complex, before this we hypothesized the opposite, but if we think about the non linearity of the data set it makes sense, even though the linear kernel provided a less complexed model it was unable to capture the full behavior of the data. Look at the test residual box plot for the RBF kernel, the variation is moderately close to the training residuals. Lastly for the polynomial boxplot, it has a very close variance for both training and testing residuals, suggesting the data may be well described by a systematic polynomial behavior however the amount of outlier in the boxplot for such a small sample of residual raises some danger.

E2.B.3b Discuss challenges in interpreting SVR models. It is very clear that after implementing inner products of the data in infinite dimensional space will not yield a easy translation of the behaviors of the data set in real world. Take the net weight $\beta_i = (\alpha_i - \alpha_i^*)$ where $0 \leq \alpha_i, \alpha_i^* \leq C$, then

$$\hat{f}(\vec{x}_{\text{new}}) = \sum_{i \in SV} \beta_i \, K(\vec{x}_i, \vec{x}_{\text{new}}) + b$$

No matter the choice of the kernel $K(\cdot, \cdot)$ we loss the interpretation in the inner products.

Part C. Tree-Based Regression

1. Single Trees

E2.C.1a Train a pruned decision tree using cross-validation.

We add a complexity penalty `cp` to prune the tree, where $|T|$ is the number of leaves and R_m is the m-th region.

$$C_\alpha(T) = \sum_{1 \le m \le |T|} \sum_{i \in R_m} (Y_i - \bar{y}_{R_m})^2 + \alpha |T| = SSE(T) + \alpha |T|$$

Such α is found via

$$\alpha^* = \arg\min_\alpha \text{TestError}(T_\alpha)$$

Then in `tree_pruned` we compute

$$T_{\alpha^*} = \arg\min_T C_{\alpha^*}(T)$$

to reduce variance.

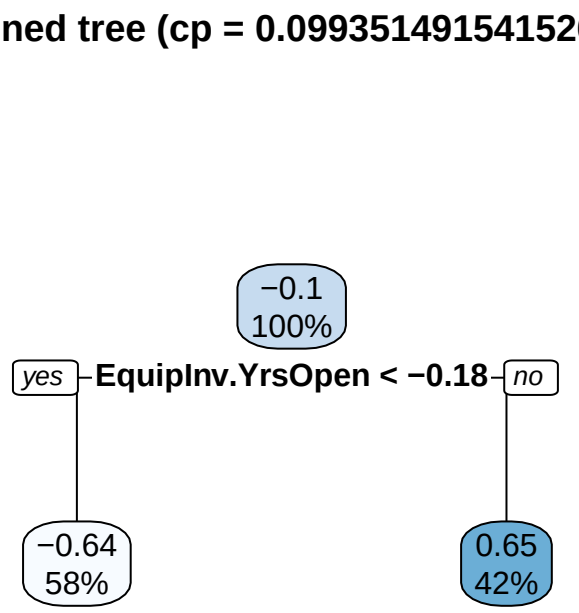
```
tree_full <- rpart(y_train ~ ., data = data.frame(x_train_mat, y_train),
                   method = "anova", control = rpart.control(cp = 0.01))  #This is Growing the Tree
best_cp <- tree_full$cptable[which.min(tree_full$cptable[, "xerror"]), "CP"]
tree_pruned <- prune(tree_full, cp = best_cp)
summary(tree_pruned)
```

```
## Call:
## rpart(formula = y_train ~ ., data = data.frame(x_train_mat, y_train),
##     method = "anova", control = rpart.control(cp = 0.01))
##   n= 50
##
##           CP nsplit rel error    xerror      xstd
## 1 0.44684777      0 1.0000000 1.0366673 0.2088546
## 2 0.09935149      1 0.5531522 0.7156887 0.1310321
##
## Variable importance
##   EquipInv.YrsOpen            YrsOpen         YrsOpen_sq  YrsOpen.RuleofLaw
##                 19                 18                 18                 17
## EquipInv.RuleofLaw           EquipInv
##                 15                 14
##
## Node number 1: 50 observations,    complexity param=0.4468478
##   mean=-0.1008361, MSE=0.9118794
##   left son=2 (29 obs) right son=3 (21 obs)
##   Primary splits:
##       EquipInv.YrsOpen   < -0.1760206 to the left,  improve=0.4468478, (0 missing)
##       YrsOpen.RuleofLaw  < -0.0905234 to the left,  improve=0.3977555, (0 missing)
##       EquipInv.RuleofLaw < -0.2307474 to the left,  improve=0.3556204, (0 missing)
##       EquipInv           < -0.1822604 to the left,  improve=0.3411148, (0 missing)
##       EquipInv_sq        < -0.3821696 to the left,  improve=0.3411148, (0 missing)
##   Surrogate splits:
##       YrsOpen            < 0.3903155  to the left,  agree=0.96, adj=0.905, (0 split)
##       YrsOpen_sq         < 0.07156788 to the left,  agree=0.96, adj=0.905, (0 split)
##       YrsOpen.RuleofLaw  < -0.0905234 to the left,  agree=0.94, adj=0.857, (0 split)
##       EquipInv.RuleofLaw < -0.3918171 to the left,  agree=0.90, adj=0.762, (0 split)
##       EquipInv           < 0.06525988 to the left,  agree=0.88, adj=0.714, (0 split)
##
## Node number 2: 29 observations
##   mean=-0.6440353, MSE=0.5273434
##
```

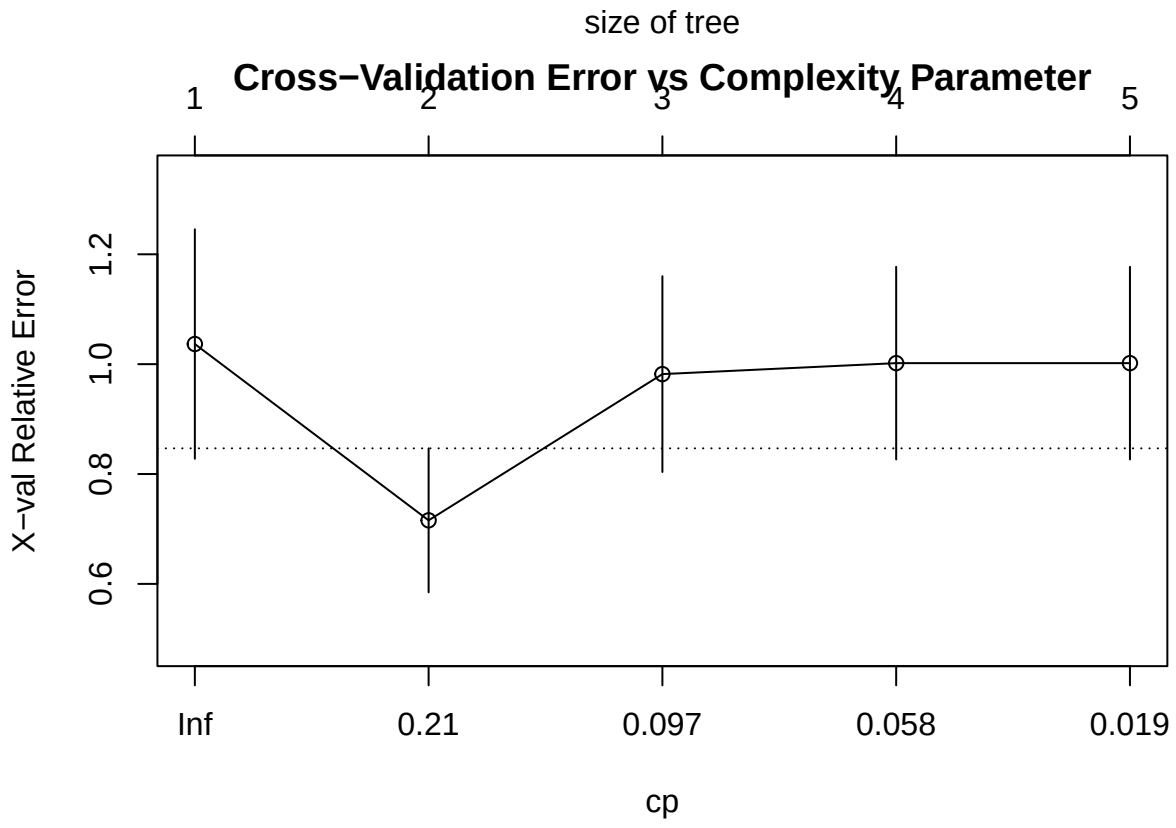
```
## Node number 3: 21 observations
##   mean=0.6492962, MSE=0.4727356
```

In essence, We first grow a maximal tree and then apply cost-complexity pruning, selecting the complexity parameter via cross-validated error minimization **E2.C.1b** Visualize the tree structure and important splits.

```
rpart.plot(tree_pruned, main = paste("Pruned tree (cp =", best_cp, ")"))
```




```
plotcp(tree_full,
       main = "Cross-Validation Error vs Complexity Parameter")
```

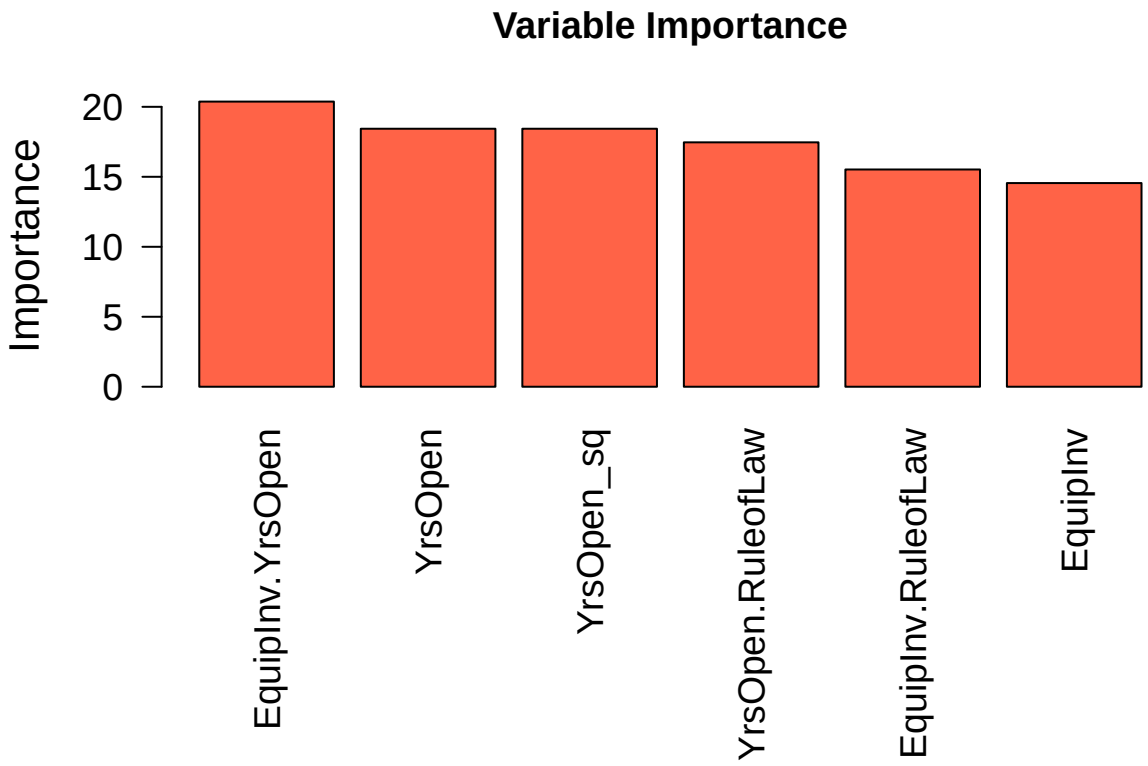


`Cross Validation` Show that a size of tree being 2 minimize the CV error. From the tree having terminal nodes $|T| = 2$ yields the piecewise constant regression model as

$$\hat{f} = \begin{cases} -0.64, & \text{EquipInv.YrsOpen} < -0.18 \\ 0.65, & \text{EquipInv.YrsOpen} \geq -0.18 \end{cases}$$

E2.C.1c Compute feature importance scores.

```
par(mar = c(11, 4, 4, 2))

barplot(tree_pruned$variable.importance,
        las = 2,
        cex.names = 1.2,
        cex.axis = 1.2,
        cex.lab  = 1.3,
        col = "tomato",
        ylab = "Importance",
        main = "Variable Importance")
```



2. Ensemble Methods

E2.C.2a Train a random forest and gradient boosting machine. We choose `mtry` ≈ 20 due to a result from Breiman's orginal Random Forest paper where `mtry` $\approx \frac{p}{3}$.

$$\mathcal{M} = \{2, 4, 6, \ldots, \min(20, p)\}$$

where $p = \text{ncol}(X_{\text{train}})$ is the total number of features.For each candidate $m \in \mathcal{M}$, train a Random Forest and we compute validation RMSE as

$$\text{RMSE}(m) = \sqrt{\frac{1}{n_{\text{val}}} \sum_{i=1}^{n_{\text{val}}} \left(y_i - \hat{f}_m(x_i)\right)^2}$$

```
mtry_grid <- seq(2, min(20, ncol(x_train_mat)), by = 2)
rf_val_rmse <- sapply(mtry_grid, function(m) {
  rf <- randomForest(x_train_mat, y_train, mtry = m, ntree = 200)
  pred <- predict(rf, x_val_mat)
  sqrt(mean((y_val - pred)^2))
})
best_mtry <- mtry_grid[which.min(rf_val_rmse)]
rf_final <- randomForest(x_train_mat, y_train, mtry = best_mtry, ntree = 500, importance = TRUE)
```

E2.C.2b Tune hyperparameters using validation set performance. The hyperparamter is tuned from

$$\theta^* = (n_t^*, d^*, \lambda^*) = \arg\min_{\theta \in \Theta} \text{RMSE}(\theta)$$

$$\Theta = \{(n_t, d, \lambda) : n_t \in \{100, 200\},\ d \in \{2, 3\},\ \lambda \in \{0.05, 0.1\}\}$$

where n_t is the number of trees, d is the interaction depth, and λ is the shrinkage rate.

```
gbm_val_rmse <- function(nt, depth, shr) {
  gbm_mod <- gbm(y_train ~ ., data = data.frame(x_train_mat, y_train),
                 distribution = "gaussian", n.trees = nt,
                 interaction.depth = depth, shrinkage = shr, verbose = FALSE)
  pred <- predict(gbm_mod, newdata = data.frame(x_val_mat), n.trees = nt)
  sqrt(mean((y_val - pred)^2))
}
params <- expand.grid(nt = c(100, 200), depth = c(2, 3), shr = c(0.05, 0.1))
params$rmse <- mapply(gbm_val_rmse, params$nt, params$depth, params$shr)
best_gbm <- params[which.min(params$rmse), ]
gbm_final <- gbm(y_train ~ ., data = data.frame(x_train_mat, y_train),
                 distribution = "gaussian", n.trees = best_gbm$nt,
                 interaction.depth = best_gbm$depth, shrinkage = best_gbm$shr)
```

```
cat("Random Forest OOB RMSE:", sqrt(tail(rf_final$mse, 1)), "\n")
```

```
## Random Forest OOB RMSE: 0.7530944
```

```
cat("Random Forest validation RMSE:", min(rf_val_rmse), "\n")
```

```
## Random Forest validation RMSE: 0.8161978
```

```
# Test set evaluation (original scale)
pred_test_rf <- predict(rf_final, x_test_mat) * y_sd + y_mean
pred_test_gbm <- predict(gbm_final, newdata = data.frame(x_test_mat), n.trees = best_gbm$nt) * y_sd + y
```

E2.C.2c Compare out-of-bag error with validation error The out of bag error is given

$$\hat{R}_{OOB} = \frac{1}{n} \sum_{1 \le i \le n} \left(y_i - \frac{1}{|\{b | i \neq \mathcal{D}_b^*\}|} \sum_{b: i \neq \mathcal{D}_b^*} T_b(\vec{x}_i) \right)^2$$

which uses the observation left out of each bootstrap sample. As we derived in class as

$$\lim_{n \to \infty} \left(1 - \frac{1}{n}\right)^n \approx 0.368$$

$$\text{RMSE}_{\text{OOB}} = \sqrt{\frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} \left(y_i - \hat{f}_{\text{OOB}}(x_i)\right)^2}$$

with $\hat{f}_{OOB}$ given as

$$\hat{f}_{\text{OOB}}(x_i) = \frac{1}{|\mathcal{B}_i|} \sum_{b \in \mathcal{B}_i} T_b(x_i)$$

and $\mathcal{B}_i$ is the set of trees for which observation i was not in the bootstrap sample.

$$\hat{y}_{\text{test}} = \hat{f}(x_{\text{test}}) \cdot \sigma_y + \mu_y$$

```r
cat("Random Forest OOB RMSE:", sqrt(tail(rf_final$mse, 1)), "\n")
```

```
## Random Forest OOB RMSE: 0.7530944
```

```r
cat("Random Forest validation RMSE:", min(rf_val_rmse), "\n")
```

```
## Random Forest validation RMSE: 0.8161978
```

```r
# Test set evaluation (original scale)
pred_test_rf <- predict(rf_final, x_test_mat) * y_sd + y_mean
pred_test_gbm <- predict(gbm_final, newdata = data.frame(x_test_mat), n.trees = best_gbm$nt) * y_sd + y
```

```r
get_metrics(y_test_orig, pred_test_rf, label = "Random Forest")
```

```
## Random Forest
##   RMSE: 0.00547
##   MAE : 0.004637
##   R2  : 0.851345

##     RMSE      MAE       R2
## 0.005470 0.004637 0.851345
```

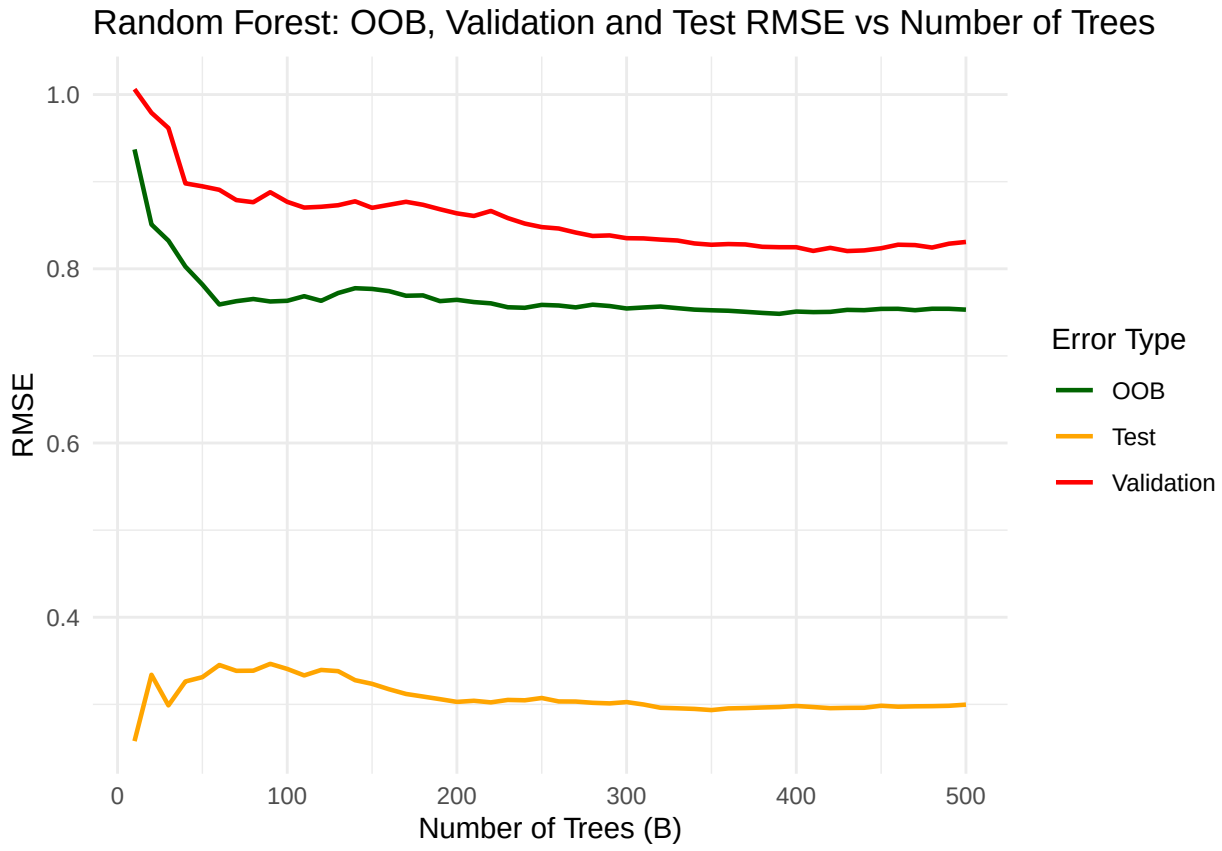
```r
get_metrics(y_test_orig, pred_test_gbm, label = "Gradient Boosting")
```

```
## Gradient Boosting
##   RMSE: 0.007477
##   MAE : 0.006259
##   R2  : 0.722295

##     RMSE      MAE       R2
## 0.007477 0.006259 0.722295
```

More Visuals

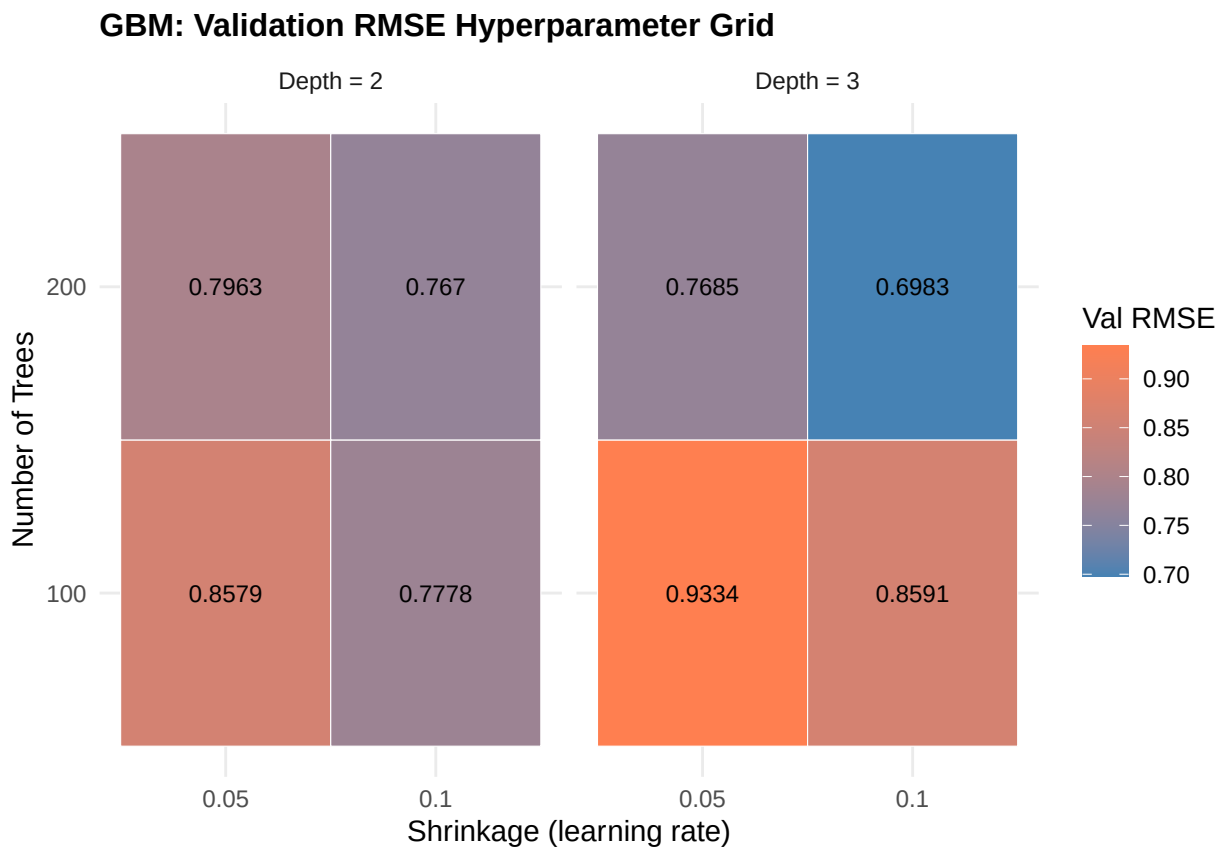
```r
tree_seq <- seq(10, 500, by = 10)

oob_rmse_vec <- sqrt(rf_final$mse[tree_seq])

val_rmse_vec <- sapply(tree_seq, function(nt) {
  pred_val <- predict(rf_final, x_val_mat, predict.all = TRUE)$individual[, 1:nt]
  pred_val_mean <- rowMeans(pred_val)
  sqrt(mean((y_val - pred_val_mean)^2))
})

test_rmse_vec <- sapply(tree_seq, function(nt) {
  pred_test <- predict(rf_final, x_test_mat, predict.all = TRUE)$individual[, 1:nt]
  pred_test_mean <- rowMeans(pred_test)
  sqrt(mean((y_test - pred_test_mean)^2))
})
```

```r
rmse_df <- data.frame(
  Trees = rep(tree_seq, 3),
  RMSE  = c(oob_rmse_vec, val_rmse_vec, test_rmse_vec),
  Type  = rep(c("OOB", "Validation", "Test"), each = length(tree_seq))
)

ggplot(rmse_df, aes(x = Trees, y = RMSE, colour = Type)) +
  geom_line(linewidth = 0.8) +
  scale_colour_manual(values = c(
    "OOB"        = "darkgreen",
    "Validation" = "red",
    "Test"       = "orange"
  )) +
  labs(title = "Random Forest: OOB, Validation and Test RMSE vs Number of Trees",
       x = "Number of Trees (B)",
       y = "RMSE",
       colour = "Error Type") +
  theme_minimal()
```

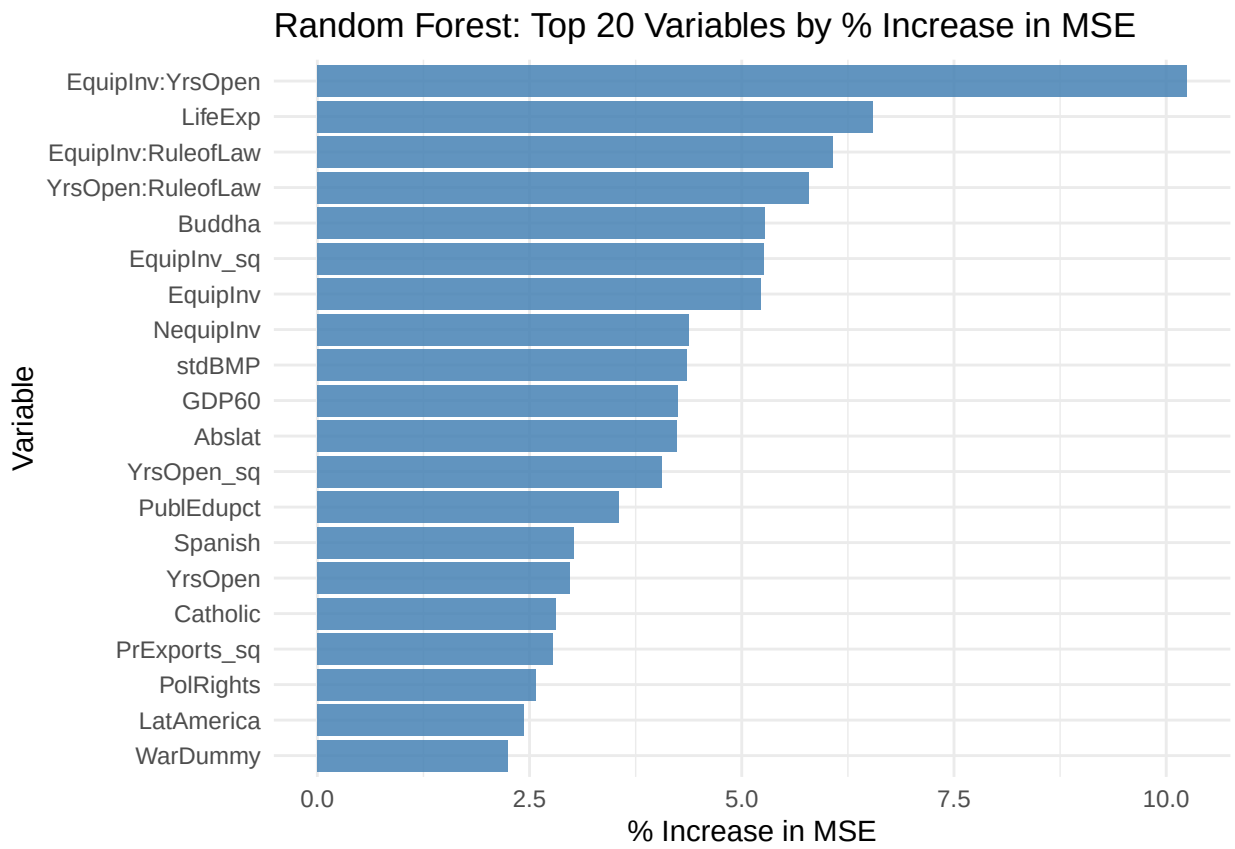



```r
params$depth <- as.factor(params$depth)
params$shr   <- as.factor(params$shr)

ggplot(params, aes(x = shr, y = as.factor(nt), fill = rmse)) +
  geom_tile(colour = "white") +
  facet_wrap(~ paste("Depth =", depth)) +
```

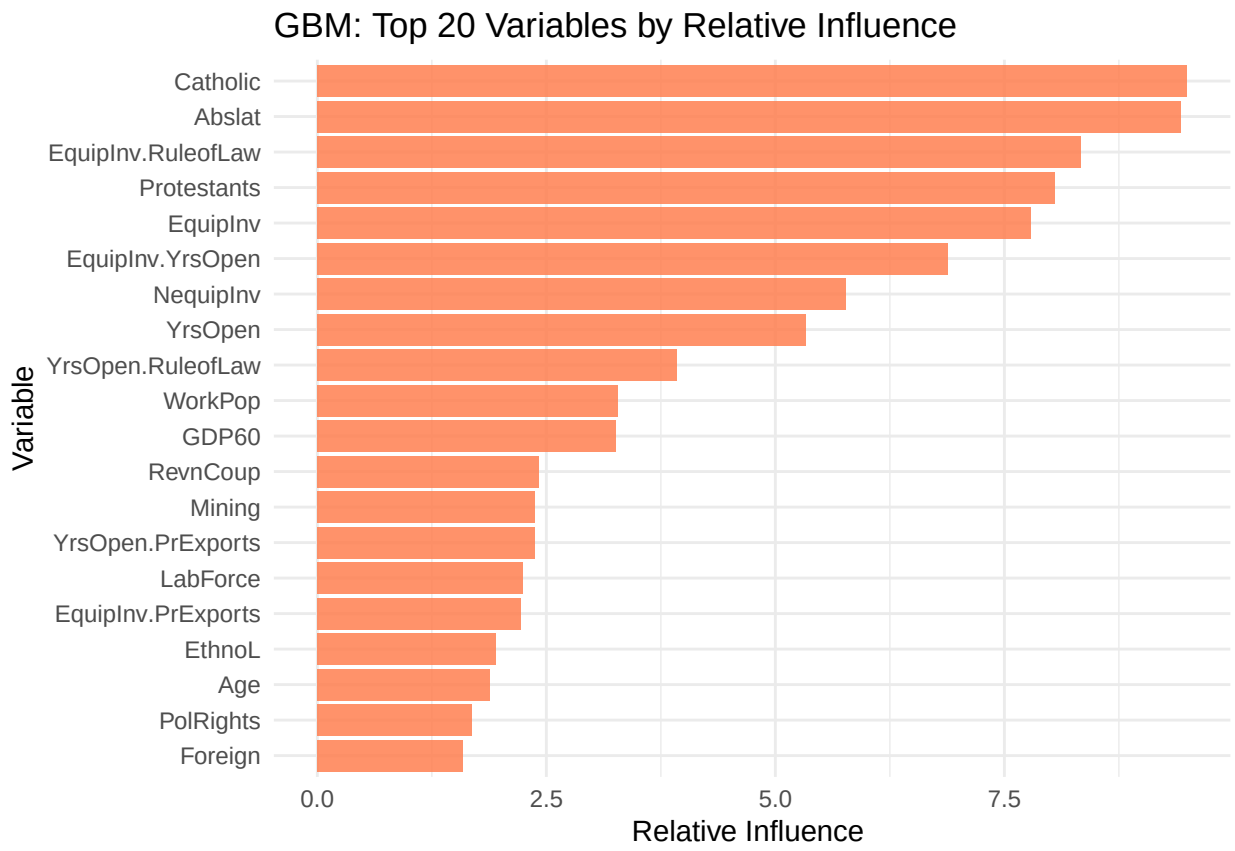
```
  scale_fill_gradient(low = "steelblue", high = "coral",
                      name = "Val RMSE") +
  geom_text(aes(label = round(rmse, 4)), size = 3) +
  labs(title = "GBM: Validation RMSE Hyperparameter Grid",
       x = "Shrinkage (learning rate)",
       y = "Number of Trees") +
  theme_minimal() +
  theme(plot.title = element_text(size = 12, face = "bold"))
```



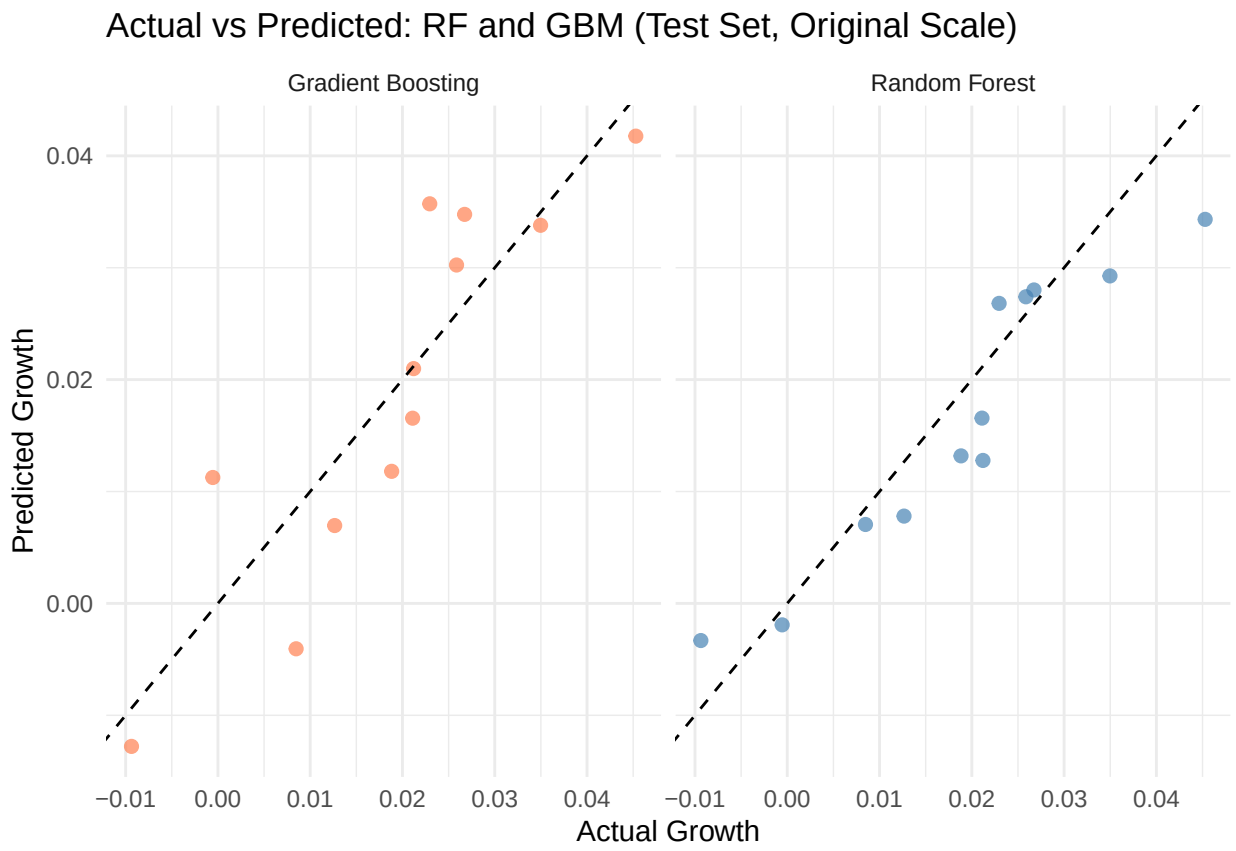

```
imp_df <- data.frame(
  Variable   = rownames(importance(rf_final)),
  IncMSE     = importance(rf_final)[, "%IncMSE"],
  IncNodePur = importance(rf_final)[, "IncNodePurity"]
)
imp_df <- imp_df[order(imp_df$IncMSE, decreasing = TRUE), ]

ggplot(imp_df[1:20, ],
       aes(x = reorder(Variable, IncMSE), y = IncMSE)) +
  geom_bar(stat = "identity", fill = "steelblue", alpha = 0.85) +
  coord_flip() +
  labs(title = "Random Forest: Top 20 Variables by % Increase in MSE",
       x = "Variable", y = "% Increase in MSE") +
  theme_minimal()
```




```r
gbm_imp <- summary(gbm_final, plotit = FALSE)
gbm_imp_df <- data.frame(
  Variable   = gbm_imp$var,
  Importance = gbm_imp$rel.inf
)

ggplot(gbm_imp_df[1:20, ],
       aes(x = reorder(Variable, Importance), y = Importance)) +
  geom_bar(stat = "identity", fill = "coral", alpha = 0.85) +
  coord_flip() +
  labs(title = "GBM: Top 20 Variables by Relative Influence",
       x = "Variable", y = "Relative Influence ") +
  theme_minimal()
```




```r
pred_df <- data.frame(
  Actual  = rep(y_test_orig, 2),
  Predicted = c(pred_test_rf, pred_test_gbm),
  Model   = rep(c("Random Forest", "Gradient Boosting"),
                each = length(y_test_orig))
)

ggplot(pred_df, aes(x = Actual, y = Predicted, colour = Model)) +
  geom_point(size = 2, alpha = 0.7) +
  geom_abline(slope = 1, intercept = 0,
              colour = "black", linetype = "dashed") +
  facet_wrap(~ Model) +
  scale_colour_manual(values = c(
    "Random Forest"    = "steelblue",
    "Gradient Boosting" = "coral"
  )) +
  labs(title = "Actual vs Predicted: RF and GBM (Test Set, Original Scale)",
       x = "Actual Growth", y = "Predicted Growth") +
  theme_minimal() +
  theme(legend.position = "none")
```



Note that in Gradient Boosting, each new tree is correcting the mistakes of all previous trees. The learning rate λ controls how aggressively each correction is applied so a smaller λ means more conservative updates and generally better generalization, but requires more trees T to converge.

```
oob_val_df <- data.frame(
  Metric = c("OOB RMSE", "Validation RMSE", "Test RMSE"),
  RF     = c(sqrt(tail(rf_final$mse, 1)) * y_sd,
             min(rf_val_rmse)            * y_sd,
             get_metrics(y_test_orig, pred_test_rf)["RMSE"]),
  GBM    = c(NA,
             min(params$rmse)            * y_sd,
             get_metrics(y_test_orig, pred_test_gbm)["RMSE"])
)
```

```
##   RMSE: 0.00547
##   MAE : 0.004637
##   R2  : 0.851345
##
##   RMSE: 0.007477
##   MAE : 0.006259
##   R2  : 0.722295
```

```
cat("=== OOB vs Validation vs Test Error ===\n")
```

```
## === OOB vs Validation vs Test Error ===
```

```r
print(oob_val_df)
```

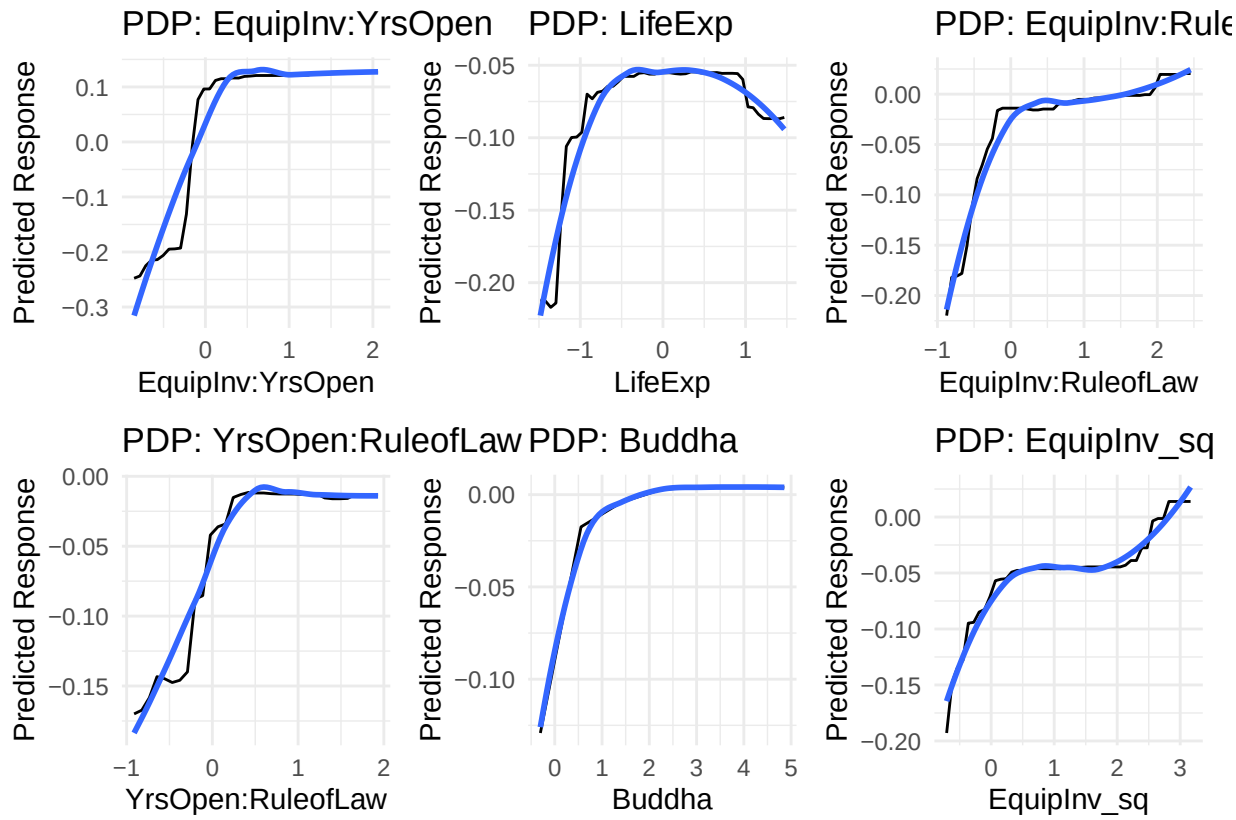
```
##            Metric         RF        GBM
## 1        OOB RMSE 0.01374692         NA
## 2 Validation RMSE 0.01489880 0.01274758
## 3       Test RMSE 0.00547000 0.00747700
```

3. Interpretation

E2.C.3a Create partial dependence plots for important features. We select the top features from the based on the importance measure. As we can see that random forest is indeed a great model for this dataset in comparison to a linear regression, it would not have been able to capture these nonlinear behaviors.

```r
importance_df <- data.frame(
  Variable = rownames(importance(rf_final)),
  IncMSE   = importance(rf_final)[, "%IncMSE"]
)
top_vars <- importance_df[order(importance_df$IncMSE, decreasing = TRUE), "Variable"][1:6] #Just select

pdp_plots <- lapply(top_vars, function(var) {
  pd <- partial(rf_final, pred.var = var,
                train = x_train_mat)
  autoplot(pd, smooth = TRUE) +
    labs(title = paste("PDP:", var),
         x = var, y = "Predicted Response") +
    theme_minimal()
})

wrap_plots(pdp_plots, ncol = 3)
```

```
## `geom_smooth()` using method = 'loess'
## `geom_smooth()` using method = 'loess'
## `geom_smooth()` using method = 'loess'
## `geom_smooth()` using method = 'loess'
## `geom_smooth()` using method = 'loess'
## `geom_smooth()` using method = 'loess'
```



E2.C.3b Compare feature importance across different tree methods. It appears that the feature importance across Random Forest and Gradient Boosting is significantly different. The top features in RF is the engineered artifical features and GBM prefers the original features more in general.

```
rf_imp <- sort(importance(rf_final)[, 1], decreasing = TRUE)[1:6]
gbm_imp <- summary(gbm_final, plotit = FALSE)
cat("\nTop RF features:\n"); print(head(rf_imp))
```

```
##
## Top RF features:

##   EquipInv:YrsOpen            LifeExp EquipInv:RuleofLaw  YrsOpen:RuleofLaw
##          10.243500           6.542367           6.075563           5.786505
##             Buddha        EquipInv_sq
##           5.269792           5.259627
```

```
cat("Top GBM features:\n"); print(head(gbm_imp))
```

```
## Top GBM features:

##                                  var  rel.inf
## Catholic                     Catholic 9.492567
## Abslat                         Abslat 9.423897
## EquipInv.RuleofLaw EquipInv.RuleofLaw 8.338805
```

```
## Protestants               Protestants 8.054638
## EquipInv                     EquipInv 7.792793
## EquipInv.YrsOpen     EquipInv.YrsOpen 6.887582
```

E2.C.3c Discuss the trade-off between interpretability and performance.

There is a fundamental tension between model interpretability and predictive performance. A linear model offers full transparency; each coefficient $\hat{\beta}_j$ directly quantifies the marginal effect of feature j on the response. However, this comes at the cost of flexibility. A classic example is that a linear model can only capture relationships of the form

$$\hat{y} = \beta_0 + \sum_{j=1}^{p} \beta_j x_j$$

and will systematically underfit when the true data generating process is non-linear.

Random Forest and GBM sacrifice transparency for expressive power. Their predictions arise from ensembles of hundreds of trees, making it impossible to write a simple closed-form expression for $\hat{f}(x)$. We partially recover interpretability through tools such as variable importance and partial dependence plots, which approximate the marginal effect of a single feature x_j:

$$\bar{f}_{x_j}(x_j) = \frac{1}{n} \sum_{i=1}^{n} \hat{f}(x_j, x_{-j}^{(i)})$$

Part D. Comprehensive Comparison

1. Performance Metrics

E2.D.1a Create a table comparing all methods on test set RMSE, MAE, and R^2.

```
pred_test_ridge <- predict(ridge_model, newx = x_test_mat, s = lambda_min) * y_sd + y_mean
pred_test_tree_unpruned <- predict(tree_unpruned, newdata = data.frame(x_test_mat)) * y_sd + y_mean
pred_test_tree_pruned <- predict(tree_pruned, newdata = data.frame(x_test_mat)) * y_sd + y_mean
```

```
models_list <- list( "Ridge" = pred_test_ridge, "Unpruned Tree" = pred_test_tree_unpruned, "Pruned Tree

build_comparison_table <- function(models_list, actual) {
  results <- lapply(names(models_list), function(model_name) {
    pred    <- as.numeric(models_list[[model_name]])
    metrics <- get_metrics(actual, pred)
    data.frame(
      Model = model_name,
      RMSE  = metrics["RMSE"],
      MAE   = metrics["MAE"],
      R2    = metrics["R2"],
      row.names = NULL
    )
  })
  df <- do.call(rbind, results)
  df <- df[order(df$RMSE), ]
  return(df)
}

comparison_df <- build_comparison_table(models_list, y_test_orig)
```

```
##   RMSE: 0.013952
##   MAE : 0.010634
##   R2  : 0.033116
##
##   RMSE: 0.009528
##   MAE : 0.006929
##   R2  : 0.549054
##
##   RMSE: 0.00989
##   MAE : 0.008629
##   R2  : 0.514105
##
##   RMSE: 0.007898
##   MAE : 0.00608
##   R2  : 0.690115
##
##   RMSE: 0.006776
##   MAE : 0.004933
##   R2  : 0.771937
##
##   RMSE: 0.005542
##   MAE : 0.004655
##   R2  : 0.847425
##
##   RMSE: 0.00547
##   MAE : 0.004637
##   R2  : 0.851345
##
##   RMSE: 0.007477
##   MAE : 0.006259
##   R2  : 0.722295
```

```
cat("=== Full Model Comparison (Test Set) ===\n")
```

```
## === Full Model Comparison (Test Set) ===
```

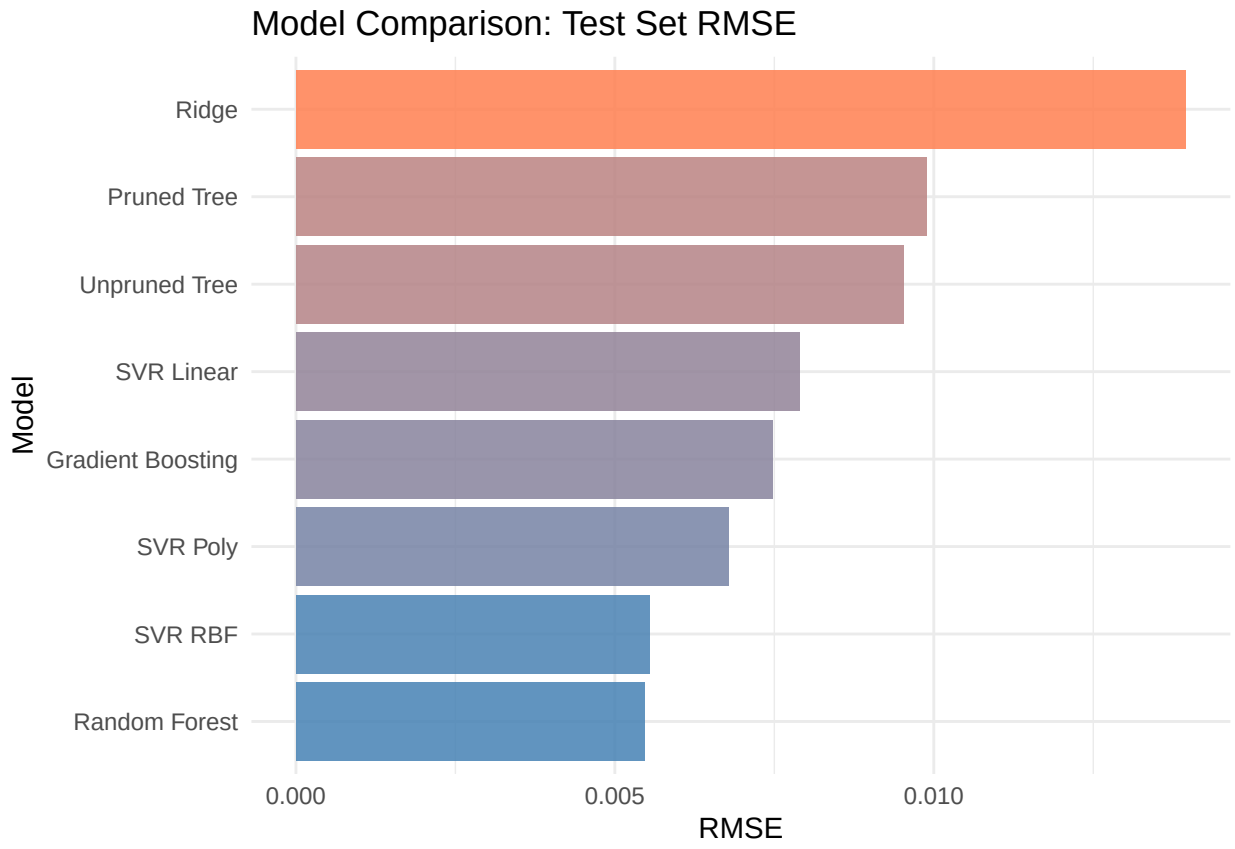
```
print(comparison_df)
```

```
##               Model     RMSE      MAE       R2
## 7     Random Forest 0.005470 0.004637 0.851345
## 6           SVR RBF 0.005542 0.004655 0.847425
## 5          SVR Poly 0.006776 0.004933 0.771937
## 8 Gradient Boosting 0.007477 0.006259 0.722295
## 4        SVR Linear 0.007898 0.006080 0.690115
## 2     Unpruned Tree 0.009528 0.006929 0.549054
## 3       Pruned Tree 0.009890 0.008629 0.514105
## 1             Ridge 0.013952 0.010634 0.033116
```

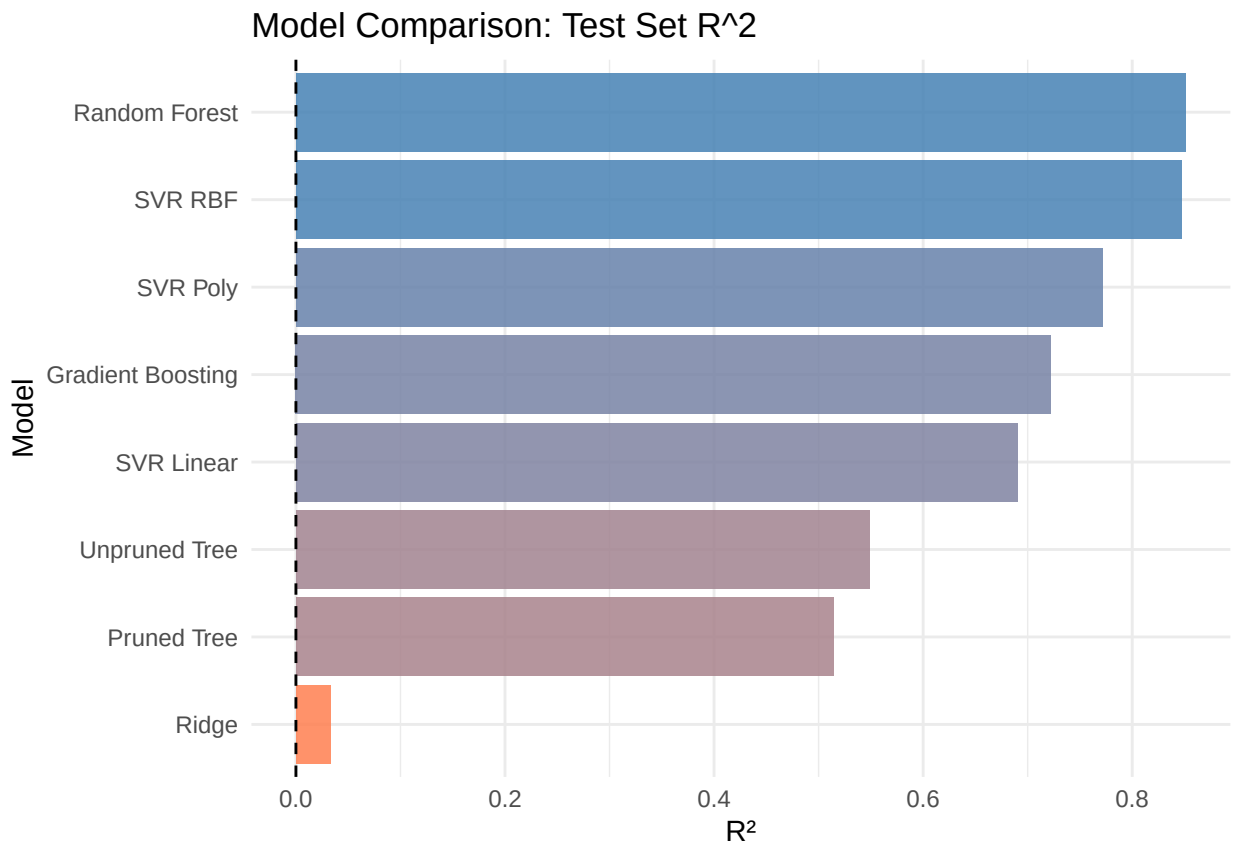
ggplot to visualize the metrics.

```
ggplot(comparison_df,
       aes(x = reorder(Model, RMSE), y = RMSE, fill = RMSE)) +
```

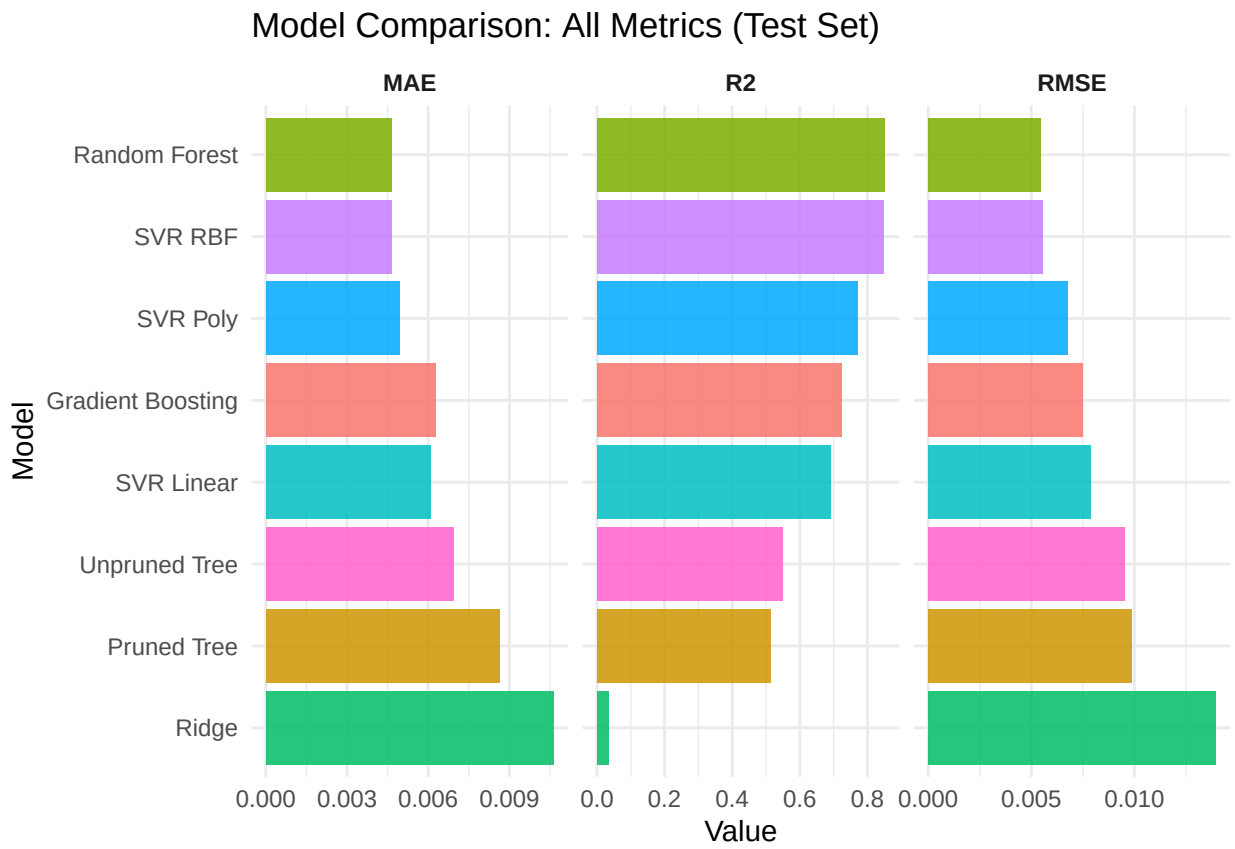
```
  geom_bar(stat = "identity", alpha = 0.85) +
  scale_fill_gradient(low = "steelblue", high = "coral") +
  coord_flip() +
  labs(title = "Model Comparison: Test Set RMSE",
       x = "Model", y = "RMSE") +
  theme_minimal() +
  theme(legend.position = "none")
```




```
# R2 bar chart
ggplot(comparison_df,
       aes(x = reorder(Model, R2), y = R2, fill = R2)) +
  geom_bar(stat = "identity", alpha = 0.85) +
  scale_fill_gradient(low = "coral", high = "steelblue") +
  geom_hline(yintercept = 0, colour = "black",
             linetype = "dashed", linewidth = 0.5) +
  coord_flip() +
  labs(title = "Model Comparison: Test Set R^2",
       x = "Model", y = "R²") +
  theme_minimal() +
  theme(legend.position = "none")
```




```
comparison_long <- tidyr::pivot_longer(
  comparison_df,
  cols      = c("RMSE", "MAE", "R2"),
  names_to  = "Metric",
  values_to = "Value"
)

ggplot(comparison_long,
       aes(x = reorder(Model, Value), y = Value, fill = Model)) +
  geom_bar(stat = "identity", alpha = 0.85) +
  facet_wrap(~ Metric, scales = "free_x") +
  coord_flip() +
  labs(title = "Model Comparison: All Metrics (Test Set)",
       x = "Model", y = "Value") +
  theme_minimal() +
  theme(legend.position = "none",
        strip.text      = element_text(face = "bold"))
```



E2.D.1b Perform statistical significance testing of performance differences. It is unclear what distribution to use for hypothesis testing, but with the power of bootstrap can test the hypothesis with empircal distributions of the metrics. Let $\delta = \text{Metric}(\hat{f}_{(1)}) - \text{Metric}(\hat{f}_{(2)})$. We can use hypothesize testing for

$$H_0 : \delta = 0, \qquad H_1 : \delta < 0$$

. Then the bootstrap p-value will be computed as the average

$$\hat{p} = \frac{1}{B} \sum_{1 \le b \le B} 1\left[\hat{\delta}^{(b)} > 0\right], \qquad \hat{\delta}^{(b)} = \text{Metric}^{(b)}(\hat{f}_{(1)}) - \text{Metric}^{(b)}(\hat{f}_{(2)}), \quad b = 1, \cdots, B$$

```
# Bootstrap Significance Testing: RMSE, MAE and R2


perf <- comparison_df[, c("RMSE", "MAE", "R2")]
rownames(perf) <- comparison_df$Model

cat("=== Performance Table (perf) ===\n")
```

```
## === Performance Table (perf) ===
```

```
print(perf)
```

```
##                       RMSE      MAE       R2
## Random Forest     0.005470 0.004637 0.851345
```

```
## SVR RBF           0.005542 0.004655 0.847425
## SVR Poly          0.006776 0.004933 0.771937
## Gradient Boosting 0.007477 0.006259 0.722295
## SVR Linear        0.007898 0.006080 0.690115
## Unpruned Tree     0.009528 0.006929 0.549054
## Pruned Tree       0.009890 0.008629 0.514105
## Ridge             0.013952 0.010634 0.033116
```

```r
best_rmse   <- rownames(perf)[which.min(perf[, "RMSE"])]
second_rmse <- rownames(perf)[order(perf[, "RMSE"])[2]]

best_mae    <- rownames(perf)[which.min(perf[, "MAE"])]
second_mae  <- rownames(perf)[order(perf[, "MAE"])[2]]

best_r2     <- rownames(perf)[which.max(perf[, "R2"])]
second_r2   <- rownames(perf)[order(perf[, "R2"], decreasing = TRUE)[2]]

cat("\n=== Best Models per Metric ===\n")
```

```
##
## === Best Models per Metric ===
```

```r
cat("RMSE - Best:", best_rmse,   "| Second:", second_rmse, "\n")
```

```
## RMSE - Best: Random Forest | Second: SVR RBF
```

```r
cat("MAE  - Best:", best_mae,    "| Second:", second_mae,  "\n")
```

```
## MAE  - Best: Random Forest | Second: SVR RBF
```

```r
cat("R2   - Best:", best_r2,     "| Second:", second_r2,   "\n")
```

```
## R2   - Best: Random Forest | Second: SVR RBF
```

```r
set.seed(547)
B      <- 999
n_test <- length(y_test_orig)


# RMSE Bootstrap Test

pred_best_rmse   <- as.numeric(models_list[[best_rmse]])
pred_second_rmse <- as.numeric(models_list[[second_rmse]])

boot_rmse <- replicate(B, {
  idx   <- sample(1:n_test, replace = TRUE)
  rmse1 <- sqrt(mean((y_test_orig[idx] - pred_best_rmse[idx])^2))
  rmse2 <- sqrt(mean((y_test_orig[idx] - pred_second_rmse[idx])^2))
  rmse1 - rmse2   # negative means best wins
})
```

```r
p_val_rmse <- mean(boot_rmse > 0)

cat("\n=== RMSE Bootstrap Test ===\n")
```

```
##
## === RMSE Bootstrap Test ===
```

```r
cat("Observed Delta        :",
    round(sqrt(mean((y_test_orig - pred_best_rmse)^2)) -
          sqrt(mean((y_test_orig - pred_second_rmse)^2)), 6), "\n")
```

```
## Observed Delta        : -7.2e-05
```

```r
cat("Bootstrap mean Delta  :", round(mean(boot_rmse), 6), "\n")
```

```
## Bootstrap mean Delta  : -6.3e-05
```

```r
cat("95% CI                : [",
    round(quantile(boot_rmse, 0.025), 6), ",",
    round(quantile(boot_rmse, 0.975), 6), "]\n")
```

```
## 95% CI                : [ -0.001423 , 0.001356 ]
```

```r
cat("p-value (", best_rmse, "better than", second_rmse, "):",
    p_val_rmse, "\n")
```

```
## p-value ( Random Forest better than SVR RBF ): 0.4384384
```

```r
# MAE Bootstrap Test

pred_best_mae   <- as.numeric(models_list[[best_mae]])
pred_second_mae <- as.numeric(models_list[[second_mae]])

boot_mae <- replicate(B, {
  idx  <- sample(1:n_test, replace = TRUE)
  mae1 <- mean(abs(y_test_orig[idx] - pred_best_mae[idx]))
  mae2 <- mean(abs(y_test_orig[idx] - pred_second_mae[idx]))
  mae1 - mae2   # negative means best wins
})

p_val_mae <- mean(boot_mae > 0)

cat("\n=== MAE Bootstrap Test ===\n")
```

```
##
## === MAE Bootstrap Test ===
```

```r
cat("Observed Delta        :",
    round(mean(abs(y_test_orig - pred_best_mae)) -
          mean(abs(y_test_orig - pred_second_mae)), 6), "\n")
```

```
## Observed Delta        : -1.8e-05
```

```r
cat("Bootstrap mean Delta  :", round(mean(boot_mae), 6), "\n")
```

```
## Bootstrap mean Delta  : -2e-05
```

```r
cat("95% CI                : [",
    round(quantile(boot_mae, 0.025), 6), ",",
    round(quantile(boot_mae, 0.975), 6), "]\n")
```

```
## 95% CI                : [ -0.001427 , 0.00144 ]
```

```r
cat("p-value (", best_mae, "better than", second_mae, "):",
    p_val_mae, "\n")
```

```
## p-value ( Random Forest better than SVR RBF ): 0.4904905
```

```r
# R2 Bootstrap Test

pred_best_r2   <- as.numeric(models_list[[best_r2]])
pred_second_r2 <- as.numeric(models_list[[second_r2]])

boot_r2 <- replicate(B, {
  idx     <- sample(1:n_test, replace = TRUE)
  ss_tot  <- sum((y_test_orig[idx] - mean(y_test_orig))^2)
  ss_res1 <- sum((y_test_orig[idx] - pred_best_r2[idx])^2)
  ss_res2 <- sum((y_test_orig[idx] - pred_second_r2[idx])^2)
  r2_1    <- 1 - ss_res1 / ss_tot
  r2_2    <- 1 - ss_res2 / ss_tot
  r2_1 - r2_2
})

p_val_r2 <- mean(boot_r2 < 0)

cat("\n=== R2 Bootstrap Test ===\n")
```

```
##
## === R2 Bootstrap Test ===
```

```r
cat("Observed Delta        :",
    round((1 - sum((y_test_orig - pred_best_r2)^2) /
               sum((y_test_orig - mean(y_test_orig))^2)) -
          (1 - sum((y_test_orig - pred_second_r2)^2) /
               sum((y_test_orig - mean(y_test_orig))^2)), 6), "\n")
```

```
## Observed Delta        : 0.00392
```

```r
cat("Bootstrap mean Delta  :", round(mean(boot_r2), 6), "\n")
```

```
## Bootstrap mean Delta  : 0.003689
```

```r
cat("95% CI                : [",
    round(quantile(boot_r2, 0.025), 6), ",",
    round(quantile(boot_r2, 0.975), 6), "]\n")
```

```
## 95% CI                : [ -0.103632 , 0.103744 ]
```

```r
cat("p-value (", best_r2, "better than", second_r2, "):",
    p_val_r2, "\n")
```

```
## p-value ( Random Forest better than SVR RBF ): 0.4434434
```

```r
# Summary Table

summary_boot <- data.frame(
  Metric      = c("RMSE", "MAE", "R2"),
  Best_Model  = c(best_rmse, best_mae, best_r2),
  Second_Model= c(second_rmse, second_mae, second_r2),
  p_value     = round(c(p_val_rmse, p_val_mae, p_val_r2), 4),
  Significant = c(p_val_rmse < 0.05,
                  p_val_mae  < 0.05,
                  p_val_r2   < 0.05)
)

cat("\n=== Bootstrap Significance Summary ===\n")
```

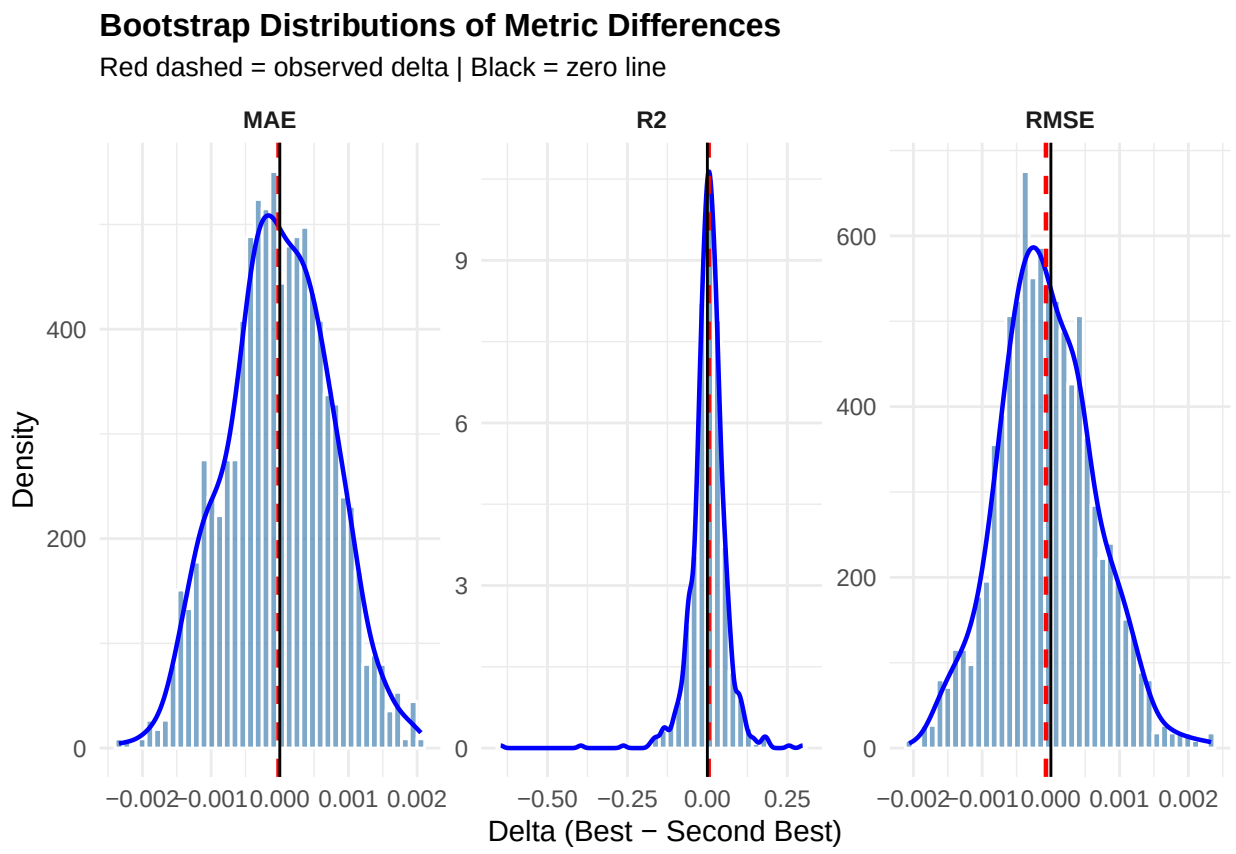
```
##
## === Bootstrap Significance Summary ===
```

```r
print(summary_boot)
```

```
##   Metric    Best_Model Second_Model p_value Significant
## 1   RMSE Random Forest      SVR RBF  0.4384       FALSE
## 2    MAE Random Forest      SVR RBF  0.4905       FALSE
## 3     R2 Random Forest      SVR RBF  0.4434       FALSE
```

```r
# Visualise all three bootstrap distributions

boot_plot_df <- data.frame(
  Delta  = c(boot_rmse, boot_mae, boot_r2),
  Metric = rep(c("RMSE", "MAE", "R2"), each = B)
)

obs_deltas <- data.frame(
  Metric = c("RMSE", "MAE", "R2"),
  Delta  = c(
```

```
    sqrt(mean((y_test_orig - pred_best_rmse)^2)) -
    sqrt(mean((y_test_orig - pred_second_rmse)^2)),
    mean(abs(y_test_orig - pred_best_mae)) -
    mean(abs(y_test_orig - pred_second_mae)),
    (1 - sum((y_test_orig - pred_best_r2)^2) /
         sum((y_test_orig - mean(y_test_orig))^2)) -
    (1 - sum((y_test_orig - pred_second_r2)^2) /
         sum((y_test_orig - mean(y_test_orig))^2))
  )
)

ggplot(boot_plot_df, aes(x = Delta)) +
  geom_histogram(aes(y = after_stat(density)),
                 bins   = 40,
                 fill   = "steelblue",
                 alpha  = 0.7,
                 colour = "white") +
  geom_density(colour = "blue", linewidth = 0.8) +
  geom_vline(data = obs_deltas,
             aes(xintercept = Delta),
             colour    = "red",
             linetype  = "dashed",
             linewidth = 0.8) +
  geom_vline(xintercept = 0,
             colour    = "black",
             linewidth = 0.5) +
  facet_wrap(~ Metric, scales = "free") +
  labs(
    title    = "Bootstrap Distributions of Metric Differences",
    subtitle = "Red dashed = observed delta | Black = zero line",
    x        = "Delta (Best - Second Best)",
    y        = "Density"
  ) +
  theme_minimal() +
  theme(
    plot.title    = element_text(size = 12, face = "bold"),
    plot.subtitle = element_text(size = 10),
    strip.text    = element_text(face = "bold")
  )
```



Since we know for certain that the random forest regression outperforms ridge regression for this data set, if we test the difference we should get small p-values which follows that we should be able to reject the H_0 for which the differences are definitely not zero. As we can see for the following three metrics. Metric RF_Value Ridge_Value Obs_Delta p_value Significant Winner 1 RMSE 0.005470 0.013952 -0.008481 0.004 TRUE Random Forest 2 MAE 0.004637 0.010634 -0.005997 0.006 TRUE Random Forest 3 R2 0.851345 0.033116 0.818229 0.001 TRUE Random Forest

```
rf_name    <- "Random Forest"
ridge_name <- "Ridge"

pred_rf    <- as.numeric(models_list[[rf_name]])
pred_ridge <- as.numeric(models_list[[ridge_name]])

cat("=== Observed Metrics: RF vs Ridge ===\n")
```

```
## === Observed Metrics: RF vs Ridge ===
```

```
cat("Random Forest  - RMSE:", round(perf[rf_name,    "RMSE"], 6),
                    "| MAE:", round(perf[rf_name,    "MAE"],  6),
                    "| R2 :", round(perf[rf_name,    "R2"],   6), "\n")
```

```
## Random Forest  - RMSE: 0.00547 | MAE: 0.004637 | R2 : 0.851345
```

```r
cat("Ridge          - RMSE:", round(perf[ridge_name, "RMSE"], 6),
                   "| MAE:", round(perf[ridge_name, "MAE"],  6),
                   "| R2 :", round(perf[ridge_name, "R2"],   6), "\n")
```

```
## Ridge          - RMSE: 0.013952 | MAE: 0.010634 | R2 : 0.033116
```

```r
set.seed(547)
B      <- 999
n_test <- length(y_test_orig)


# RMSE Bootstrap: RF vs Ridge

boot_rmse_rfr <- replicate(B, {
  idx   <- sample(1:n_test, replace = TRUE)
  rmse_rf    <- sqrt(mean((y_test_orig[idx] - pred_rf[idx])^2))
  rmse_ridge <- sqrt(mean((y_test_orig[idx] - pred_ridge[idx])^2))
  rmse_rf - rmse_ridge   # negative means RF wins
})

p_val_rmse_rfr <- mean(boot_rmse_rfr > 0)

obs_rmse_rf    <- sqrt(mean((y_test_orig - pred_rf)^2))
obs_rmse_ridge <- sqrt(mean((y_test_orig - pred_ridge)^2))
obs_delta_rmse <- obs_rmse_rf - obs_rmse_ridge

cat("\n=== RMSE Bootstrap: RF vs Ridge ===\n")
```

```
##
## === RMSE Bootstrap: RF vs Ridge ===
```

```r
cat("Observed RF RMSE    :", round(obs_rmse_rf,    6), "\n")
```

```
## Observed RF RMSE    : 0.00547
```

```r
cat("Observed Ridge RMSE :", round(obs_rmse_ridge, 6), "\n")
```

```
## Observed Ridge RMSE : 0.013952
```

```r
cat("Observed Delta      :", round(obs_delta_rmse, 6), "\n")
```

```
## Observed Delta      : -0.008481
```

```r
cat("Bootstrap mean Delta:", round(mean(boot_rmse_rfr), 6), "\n")
```

```
## Bootstrap mean Delta: -0.008238
```

```r
cat("95% CI              : [",
    round(quantile(boot_rmse_rfr, 0.025), 6), ",",
    round(quantile(boot_rmse_rfr, 0.975), 6), "]\n")
```

```
## 95% CI              : [ -0.012936 , -0.00301 ]
```

```r
cat("p-value (RF better than Ridge):", p_val_rmse_rfr, "\n")
```

```
## p-value (RF better than Ridge): 0.004004004
```

```r
# MAE Bootstrap: RF vs Ridge

boot_mae_rfr <- replicate(B, {
  idx      <- sample(1:n_test, replace = TRUE)
  mae_rf    <- mean(abs(y_test_orig[idx] - pred_rf[idx]))
  mae_ridge <- mean(abs(y_test_orig[idx] - pred_ridge[idx]))
  mae_rf - mae_ridge   # negative means RF wins
})

p_val_mae_rfr <- mean(boot_mae_rfr > 0)

obs_mae_rf    <- mean(abs(y_test_orig - pred_rf))
obs_mae_ridge <- mean(abs(y_test_orig - pred_ridge))
obs_delta_mae <- obs_mae_rf - obs_mae_ridge

cat("\n=== MAE Bootstrap: RF vs Ridge ===\n")
```

```
##
## === MAE Bootstrap: RF vs Ridge ===
```

```r
cat("Observed RF MAE     :", round(obs_mae_rf,    6), "\n")
```

```
## Observed RF MAE     : 0.004637
```

```r
cat("Observed Ridge MAE  :", round(obs_mae_ridge, 6), "\n")
```

```
## Observed Ridge MAE  : 0.010634
```

```r
cat("Observed Delta      :", round(obs_delta_mae, 6), "\n")
```

```
## Observed Delta      : -0.005997
```

```r
cat("Bootstrap mean Delta:", round(mean(boot_mae_rfr), 6), "\n")
```

```
## Bootstrap mean Delta: -0.006119
```

```r
cat("95% CI              : [",
    round(quantile(boot_mae_rfr, 0.025), 6), ",",
    round(quantile(boot_mae_rfr, 0.975), 6), "]\n")
```

```
## 95% CI              : [ -0.011243 , -0.001337 ]
```

```r
cat("p-value (RF better than Ridge):", p_val_mae_rfr, "\n")
```

```
## p-value (RF better than Ridge): 0.006006006
```

```r
# R2 Bootstrap: RF vs Ridge

boot_r2_rfr <- replicate(B, {
  idx     <- sample(1:n_test, replace = TRUE)
  ss_tot  <- sum((y_test_orig[idx] - mean(y_test_orig))^2)
  r2_rf   <- 1 - sum((y_test_orig[idx] - pred_rf[idx])^2)    / ss_tot
  r2_ridge<- 1 - sum((y_test_orig[idx] - pred_ridge[idx])^2) / ss_tot
  r2_rf - r2_ridge
})

p_val_r2_rfr <- mean(boot_r2_rfr < 0)

obs_r2_rf    <- 1 - sum((y_test_orig - pred_rf)^2)    /
                    sum((y_test_orig - mean(y_test_orig))^2)
obs_r2_ridge <- 1 - sum((y_test_orig - pred_ridge)^2) /
                    sum((y_test_orig - mean(y_test_orig))^2)
obs_delta_r2 <- obs_r2_rf - obs_r2_ridge

cat("\n=== R2 Bootstrap: RF vs Ridge ===\n")
```

```
##
## === R2 Bootstrap: RF vs Ridge ===
```

```r
cat("Observed RF R2      :", round(obs_r2_rf,    6), "\n")
```

```
## Observed RF R2      : 0.851345
```

```r
cat("Observed Ridge R2   :", round(obs_r2_ridge, 6), "\n")
```

```
## Observed Ridge R2   : 0.033116
```

```r
cat("Observed Delta      :", round(obs_delta_r2, 6), "\n")
```

```
## Observed Delta      : 0.818229
```

```r
cat("Bootstrap mean Delta:", round(mean(boot_r2_rfr), 6), "\n")
```

```
## Bootstrap mean Delta: 0.798927
```

```
cat("95% CI              : [",
    round(quantile(boot_r2_rfr, 0.025), 6), ",",
    round(quantile(boot_r2_rfr, 0.975), 6), "]\n")
```

```
## 95% CI              : [ 0.58812 , 0.888597 ]
```

```
cat("p-value (RF better than Ridge):", p_val_r2_rfr, "\n")
```

```
## p-value (RF better than Ridge): 0.001001001
```

```
summary_rfr <- data.frame(
  Metric       = c("RMSE", "MAE", "R2"),
  RF_Value     = round(c(obs_rmse_rf,    obs_mae_rf,    obs_r2_rf),    6),
  Ridge_Value  = round(c(obs_rmse_ridge, obs_mae_ridge, obs_r2_ridge), 6),
  Obs_Delta    = round(c(obs_delta_rmse, obs_delta_mae, obs_delta_r2), 6),
  p_value      = round(c(p_val_rmse_rfr, p_val_mae_rfr, p_val_r2_rfr), 4),
  Significant  = c(p_val_rmse_rfr < 0.05,
                   p_val_mae_rfr  < 0.05,
                   p_val_r2_rfr   < 0.05),
  Winner       = ifelse(c(obs_delta_rmse < 0,
                          obs_delta_mae  < 0,
                          obs_delta_r2   > 0),
                        "Random Forest", "Ridge")
)

cat("\n=== RF vs Ridge Bootstrap Summary ===\n")
```

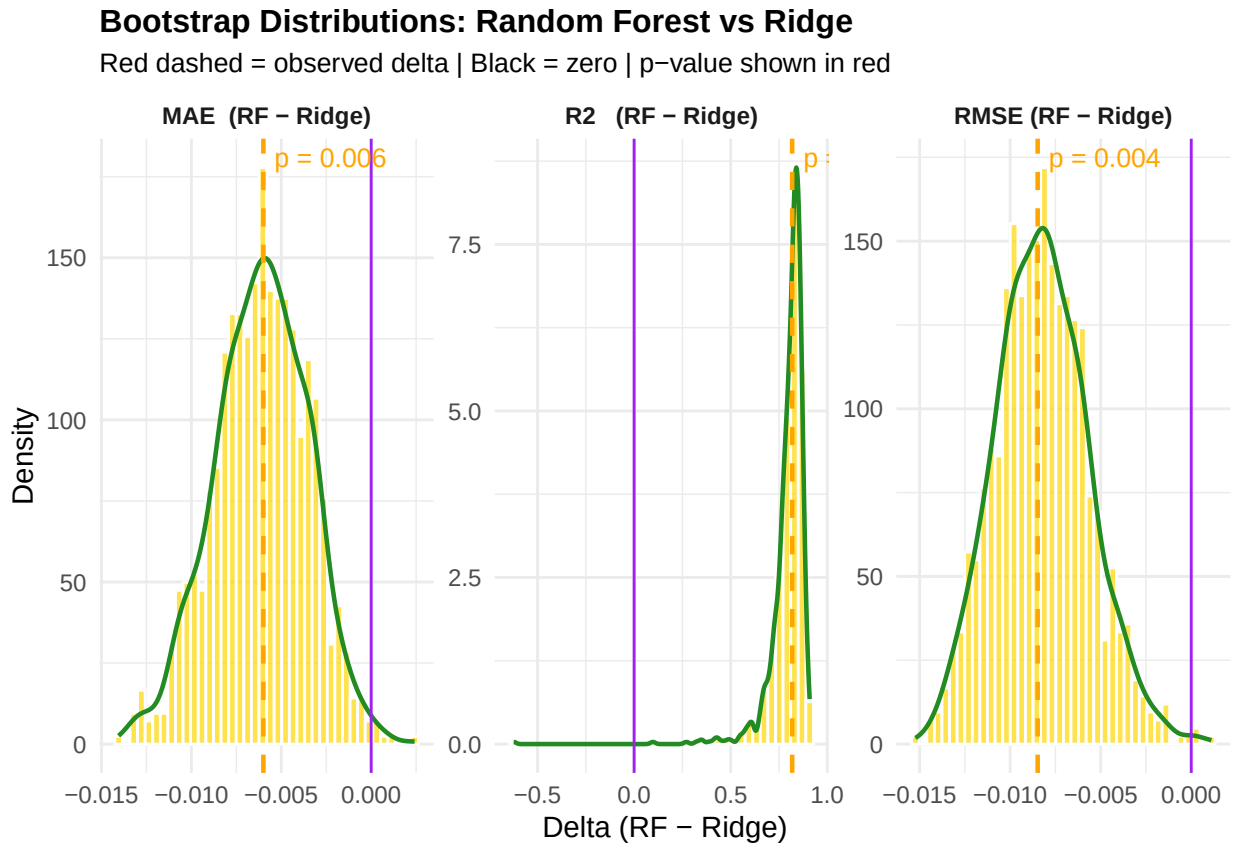
```
##
## === RF vs Ridge Bootstrap Summary ===
```

```
print(summary_rfr)
```

```
##   Metric RF_Value Ridge_Value Obs_Delta p_value Significant        Winner
## 1   RMSE 0.005470    0.013952 -0.008481   0.004        TRUE Random Forest
## 2    MAE 0.004637    0.010634 -0.005997   0.006        TRUE Random Forest
## 3     R2 0.851345    0.033116  0.818229   0.001        TRUE Random Forest
```

```
# Visualise all three bootstrap distributions

boot_plot_rfr <- data.frame(
  Delta  = c(boot_rmse_rfr, boot_mae_rfr, boot_r2_rfr),
  Metric = rep(c("RMSE (RF - Ridge)",
                 "MAE  (RF - Ridge)",
                 "R2   (RF - Ridge)"), each = B)
)

obs_deltas_rfr <- data.frame(
  Metric = c("RMSE (RF - Ridge)",
             "MAE  (RF - Ridge)",
             "R2   (RF - Ridge)"),
```

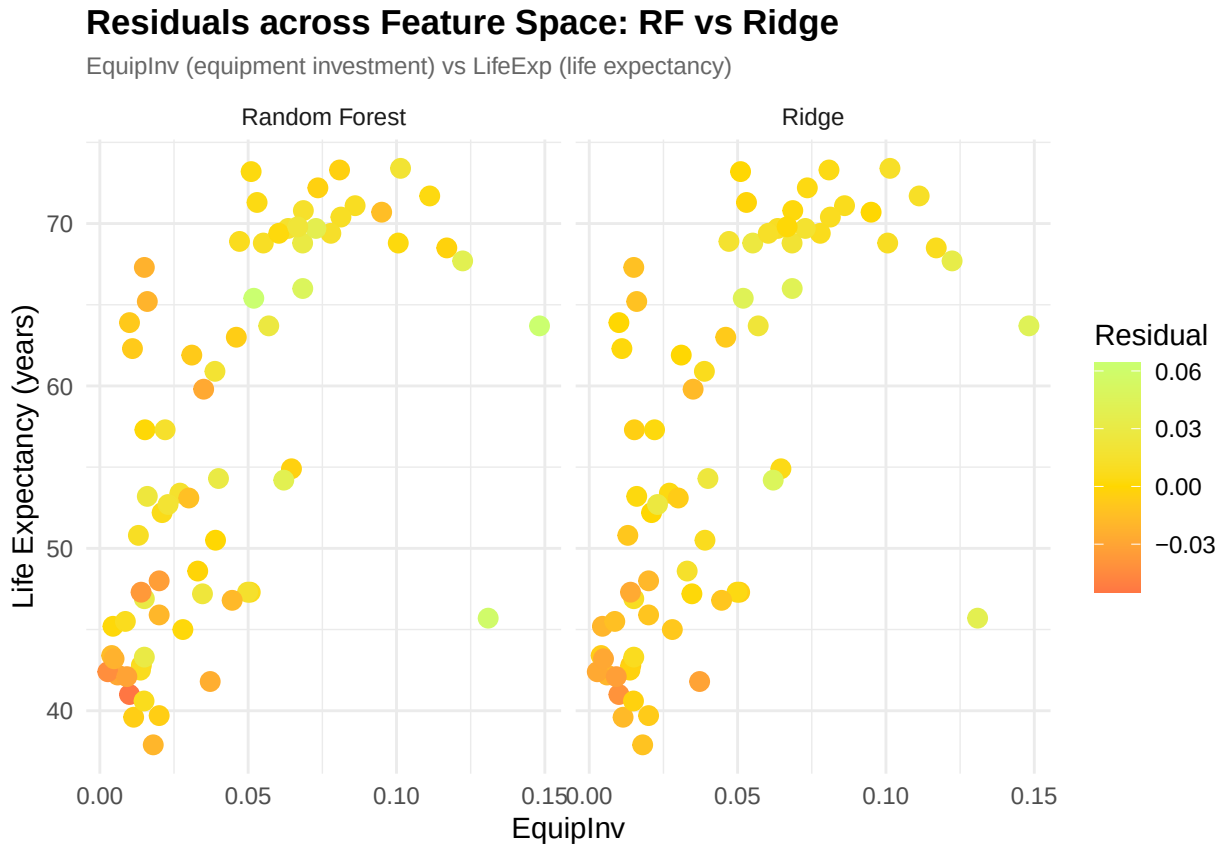
```
  Delta  = c(obs_delta_rmse, obs_delta_mae, obs_delta_r2),
  p_val  = round(c(p_val_rmse_rfr, p_val_mae_rfr, p_val_r2_rfr), 4)
)

ggplot(boot_plot_rfr, aes(x = Delta)) +
  geom_histogram(aes(y = after_stat(density)),
                 bins   = 40,
                 fill   = "gold",
                 alpha  = 0.7,
                 colour = "white") +
  geom_density(colour = "forestgreen", linewidth = 0.8) +
  geom_vline(data      = obs_deltas_rfr,
             aes(xintercept = Delta),
             colour    = "orange",
             linetype  = "dashed",
             linewidth = 0.8) +
  geom_vline(xintercept = 0,
             colour     = "purple",
             linewidth  = 0.5) +
  geom_text(data = obs_deltas_rfr,
            aes(x = Delta, y = Inf,
                label = paste("p =", p_val)),
            colour = "orange",
            vjust  = 1.5,
            hjust  = -0.1,
            size   = 3.5) +
  facet_wrap(~ Metric, scales = "free") +
  labs(
    title    = "Bootstrap Distributions: Random Forest vs Ridge",
    subtitle = paste("Red dashed = observed delta | Black = zero",
                     "| p-value shown in red"),
    x        = "Delta (RF - Ridge)",
    y        = "Density"
  ) +
  theme_minimal() +
  theme(
    plot.title    = element_text(size = 12, face = "bold"),
    plot.subtitle = element_text(size = 10),
    strip.text    = element_text(face = "bold")
  )
```



E2.D.1c Analyze which method works best for different regions of feature space.

```
residual_df <- data.frame(
  EquipInv    = x[["EquipInv"]],
  LifeExp     = x[["LifeExp"]],
  RF_Resid    = y - as.numeric(models_list[["Random Forest"]]),
  Ridge_Resid = y - as.numeric(models_list[["Ridge"]])
)

# Side by side comparison
residual_long <- tidyr::pivot_longer(
  residual_df,
  cols      = c("RF_Resid", "Ridge_Resid"),
  names_to  = "Model",
  values_to = "Residual"
)
residual_long$Model <- ifelse(residual_long$Model == "RF_Resid",
                              "Random Forest", "Ridge")

ggplot(residual_long, aes(x = EquipInv, y = LifeExp,
                          colour = Residual)) +
  geom_point(size = 3) +
  scale_colour_gradient2(low      = "tomato",
                         mid      = "gold",
                         high     = "darkolivegreen1",
                         midpoint = 0,
```

```
                        name     = "Residual") +
facet_wrap(~ Model) +
labs(title = "Residuals across Feature Space: RF vs Ridge",
     subtitle = "EquipInv (equipment investment) vs LifeExp (life expectancy)",
     x     = "EquipInv",
     y     = "Life Expectancy (years)") +
theme_minimal() +
theme(
  plot.title    = element_text(face = "bold"),
  plot.subtitle = element_text(size = 9,
                               colour = "grey40")
)
```



Although the Residuals is similar for two models, but if we take a closer look at the left bottom cluster, Random forest actually outperforms Ridge Regression by a large margin. There is a dense cluster of over approximating the real y but there far more near 0 residuals in Random forest model in that region than the Ridge Regression Model.

2. Practical Considerations

E2.D.2a Compare training time and prediction speed.

```
best_row    <- which.min(rbf_tune[, "RMSE"])
best_C      <- rbf_tune[best_row, "C"]
best_epsilon <- rbf_tune[best_row, "epsilon"]
best_gamma  <- rbf_tune[best_row, "gamma"]
```

```r
set.seed(547)
tic("Ridge")
ridge_timed <- glmnet(x_train_mat, y_train, alpha = 0,
                      lambda = lambda_min, standardize = FALSE,
                      intercept = FALSE)
t_ridge <- toc(quiet = TRUE)

tic("Random Forest")
rf_timed <- randomForest(x_train_mat, y_train,
                         mtry = best_mtry, ntree = 500)
t_rf <- toc(quiet = TRUE)

tic("GBM")
gbm_timed <- gbm(y_train ~ .,
                 data         = data.frame(x_train_mat, y_train),
                 distribution = "gaussian",
                 n.trees      = best_gbm$nt,
                 interaction.depth = best_gbm$depth,
                 shrinkage    = best_gbm$shr,
                 verbose      = FALSE)
t_gbm <- toc(quiet = TRUE)

tic("SVR RBF")
svr_timed <- svm(x_train_mat, y_train,
                 kernel  = "radial",
                 type    = "eps-regression",
                 cost    = best_C,
                 epsilon = best_epsilon,
                 gamma   = best_gamma)
t_svr <- toc(quiet = TRUE)

# Prediction speed
n_rep <- 100

pred_time_ridge <- system.time(
  replicate(n_rep, predict(ridge_timed, newx = x_test_mat,
                           s = lambda_min))
)["elapsed"] / n_rep

pred_time_rf <- system.time(
  replicate(n_rep, predict(rf_timed, x_test_mat))
)["elapsed"] / n_rep

pred_time_gbm <- system.time(
  replicate(n_rep, predict(gbm_timed,
                           newdata = data.frame(x_test_mat),
                           n.trees = best_gbm$nt))
)["elapsed"] / n_rep

pred_time_svr <- system.time(
  replicate(n_rep, predict(svr_timed, x_test_mat))
)["elapsed"] / n_rep
```

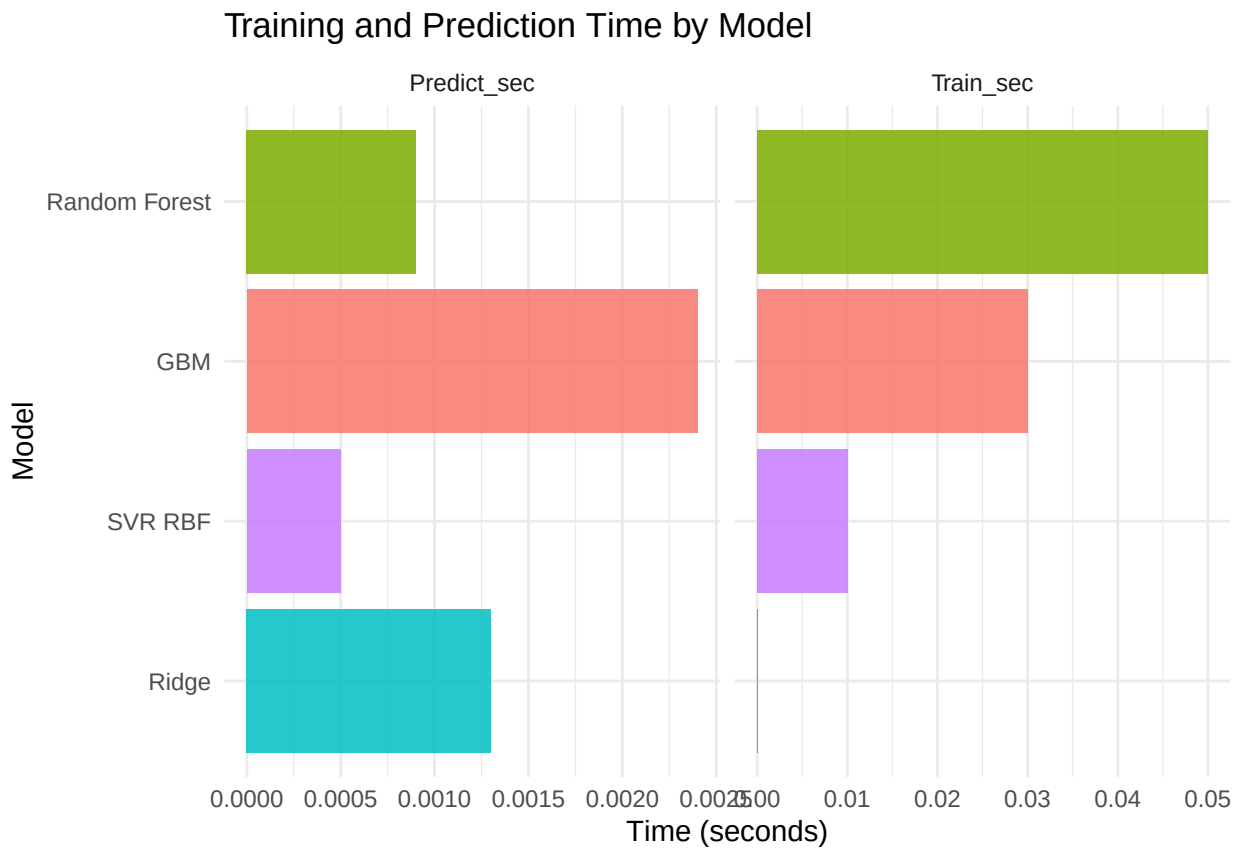
```r
# Summary table
time_df <- data.frame(
  Model       = c("Ridge", "Random Forest", "GBM", "SVR RBF"),
  Train_sec   = round(c(t_ridge$toc - t_ridge$tic,
                        t_rf$toc    - t_rf$tic,
                        t_gbm$toc   - t_gbm$tic,
                        t_svr$toc   - t_svr$tic), 4),
  Predict_sec = round(c(pred_time_ridge, pred_time_rf,
                        pred_time_gbm,  pred_time_svr), 6)
)
cat("=== Training and Prediction Time ===\n")
```

```
## === Training and Prediction Time ===
```

```r
print(time_df)
```

```
##           Model Train_sec Predict_sec
## 1         Ridge      0.00      0.0013
## 2 Random Forest      0.05      0.0009
## 3           GBM      0.03      0.0024
## 4       SVR RBF      0.01      0.0005
```

```r
# Plot
time_long <- tidyr::pivot_longer(time_df,
                                 cols     = c("Train_sec", "Predict_sec"),
                                 names_to = "Phase", values_to = "Time")
ggplot(time_long, aes(x = reorder(Model, Time), y = Time, fill = Model)) +
  geom_bar(stat = "identity", alpha = 0.85) +
  facet_wrap(~ Phase, scales = "free_x") +
  coord_flip() +
  labs(title = "Training and Prediction Time by Model",
       x = "Model", y = "Time (seconds)") +
  theme_minimal() +
  theme(legend.position = "none")
```



We see that SVR turns out to be the fastest in this small data set of $n = 76$ due to the fact that the quadratic programming problem is well studied. Gradient Boosting is slowest as expected because it builds the trees sequentially as each tree must wait for the previous trees to finish growing, compared to Random Forest, which grows tree in parallel hence it would be faster.

E2.D.2b Discuss robustness to outliers and noise.

Ridge regression is highly sensitive to outliers due to its squared loss function $(y - \hat{y})^2$, which disproportionately amplifies large errors. In contrast, Support Vector Regression (SVR) achieves inherent robustness by employing an ϵ-insensitive loss function, where errors outside the margin are penalized linearly rather than quadratically. Among ensemble methods, Random Forests effectively dilute outlier influence by averaging predictions across multiple bootstrapped trees; however, Gradient Boosting Machines (GBM) remain highly vulnerable, as their sequential nature continuously targets and amplifies the large residuals produced by outliers across iterations.

E2.D.2c Analyze sensitivity to hyperparameter choices.

```
# Show how RF R2 changes with mtry
sensitivity_df <- data.frame(
  mtry     = mtry_grid,
  Val_RMSE = rf_val_rmse
)

cat("=== RF Sensitivity to mtry ===\n")
```

```
## === RF Sensitivity to mtry ===
```

```r
print(sensitivity_df)
```

```
##    mtry  Val_RMSE
## 1     2 0.8887153
## 2     4 0.8483449
## 3     6 0.8880937
## 4     8 0.8532881
## 5    10 0.8171785
## 6    12 0.8499442
## 7    14 0.8698542
## 8    16 0.8635025
## 9    18 0.8242427
## 10   20 0.8161978
```

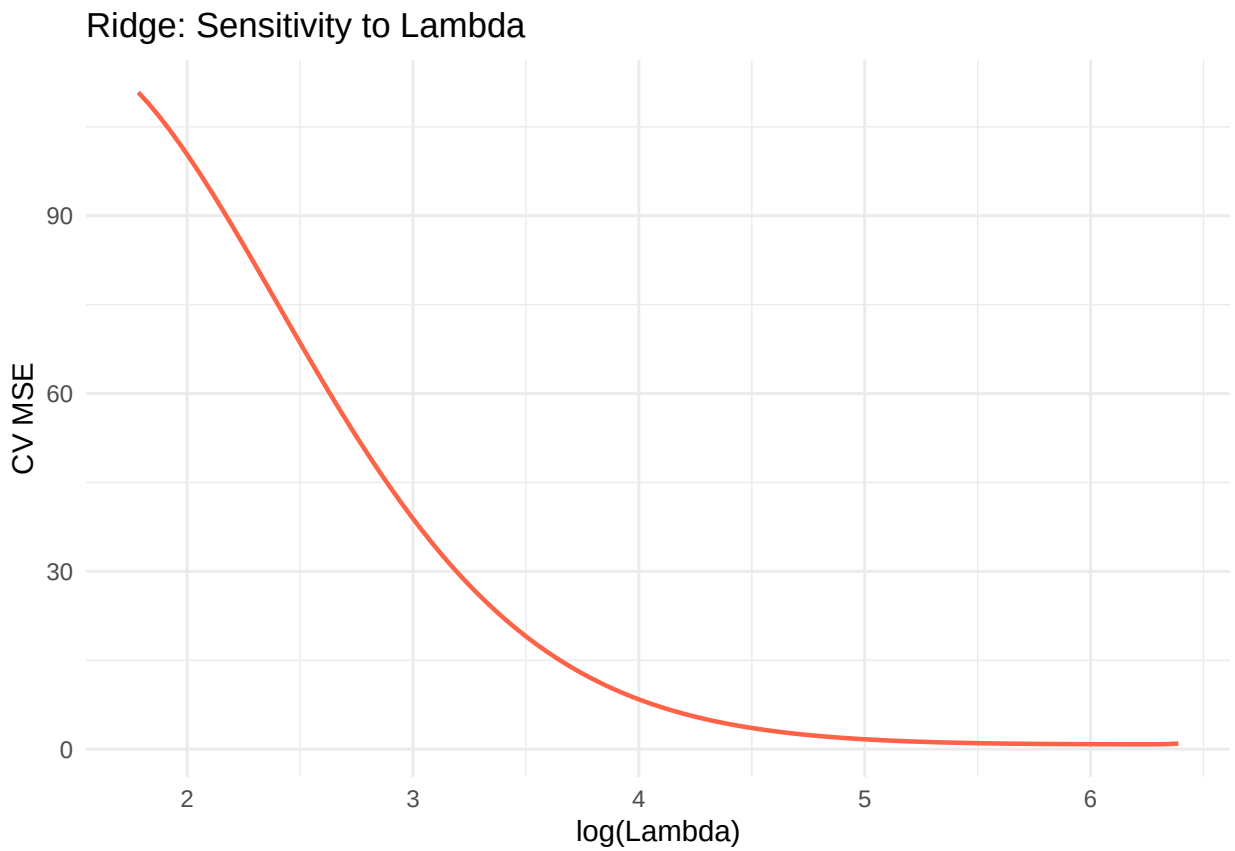
```r
cat("Range of val RMSE:", round(diff(range(rf_val_rmse)), 6), "\n")
```

```
## Range of val RMSE: 0.072518
```

```r
cat("Best mtry:", best_mtry, "\n")
```

```
## Best mtry: 20
```

```r
# Show how Ridge R2 changes with lambda
lambdas    <- cv_ridge$lambda
ridge_errs <- cv_ridge$cvm


lambda_df <- data.frame(
  Lambda = lambdas,
  CV_MSE = ridge_errs
)

#lambda_1se <- cv_ridge$lambda.1se

ggplot(lambda_df, aes(x = log(Lambda), y = CV_MSE)) +
  geom_line(colour = "tomato", linewidth = 0.8) +
  labs(title = "Ridge: Sensitivity to Lambda",
       x = "log(Lambda)", y = "CV MSE") +
  theme_minimal()
```



The Random Forest model demonstrates relatively low sensitivity to the `mtry` hyperparameter, with the validation RMSE fluctuating within a narrow band of roughly 0.07 before finding its optimal performance at mtry = 20. In contrast, the Ridge regression model is highly sensitive to its penalty parameter (λ), as the cross-validation MSE experiences a massive and steep decline before stabilizing at higher values of $\log(\lambda)$. This indicates that while Random Forest is generally robust to minor hyperparameter misspecifications, failing to properly tune the Ridge penalty will result in severe model underperformance.

3. Recommendations

E2.D.3a Based on your analysis, which method would you recommend for this dataset? I would recommend Random forest, given the highest $R^2 \approx 0.851345$ value provides us with a model that can explain the change in y purly from the predictor variables. **E2.D.3b** Justify your recommendation considering accuracy, interpretability, and computational cost. Note that we have a moderately acceptable interpretability with random forest regression as we are able to see that the top three variables found in `imp_df` is `EquipInv:YrsOpen`, `EquipInv:RuleofLaw` and `GDP60`. Suggesting that the interaction term of equipment investment with number of years having an open economic and rule of law with the inital GDP in 1960 is significantly important to in understanding the value of y (Economic growth 1960-1992 as from the Penn World Tables). **E2.D.3c** What would be your next steps for improvement? We can perform a weight average ensemble using the R^2 metric as R^2 is representative of the explanation power.

$$w_{RF} = \frac{R^2_{RF}}{R^2_{RF} + R^2_{SVR}}, \quad w_{SVR} = 1 - w_{RF}$$

So our improved ensemble prediction would be in the form of

$$\hat{y}_{ens} = w_{RF} \cdot \hat{y}_{RF} + w_{SVR} \cdot \hat{y}_{SVR}$$

As you may see below our R^2 slightly improved from $0.851345 \rightarrow 0.865688$. This is a $\approx 1.684\%$ increase.

```r
imp_sorted <- sort(importance(rf_final)[, "%IncMSE"],
                   decreasing = TRUE)
cat("Top 10 important features:\n")
```

```
## Top 10 important features:
```

```r
print(round(imp_sorted[1:10], 4))
```

```
##   EquipInv:YrsOpen            LifeExp EquipInv:RuleofLaw  YrsOpen:RuleofLaw
##            10.2435             6.5424             6.0756             5.7865
##             Buddha       EquipInv_sq           EquipInv          NequipInv
##             5.2698             5.2596             5.2213             4.3719
##             stdBMP              GDP60
##             4.3570             4.2434
```

```r
w_rf  <- perf["Random Forest", "R2"] /
         (perf["Random Forest", "R2"] + perf["SVR RBF", "R2"])
w_svr <- 1 - w_rf

pred_ensemble <- w_rf  * as.numeric(models_list[["Random Forest"]]) +
                 w_svr * as.numeric(models_list[["SVR RBF"]])

cat("=== Simple Ensemble (RF + SVR RBF) ===\n")
```

```
## === Simple Ensemble (RF + SVR RBF) ===
```

```r
cat("RF weight :", round(w_rf,  4), "\n")
```

```
## RF weight : 0.5012
```

```r
cat("SVR weight:", round(w_svr, 4), "\n")
```

```
## SVR weight: 0.4988
```

```r
get_metrics(y_test_orig, pred_ensemble)
```

```
##   RMSE: 0.0052
##   MAE : 0.004244
##   R2  : 0.865688
```

```
##     RMSE      MAE       R2
## 0.005200 0.004244 0.865688
```

Exercise 3: Applied Classification Learning

Dataset: MNIST Handwritten Digits \ 10-class classification problem: digits 0–9

STAT547-MNIST-HW3

Ducheng Tan

2026-04-10

```
#install.packages("BMS")
#install.packages("ggplot2")
#install.packages("latex2exp")
#install.packages("gbm")
#install.packages("yardstick")
library(ggplot2)
library(BMS)
library(forcats)
library(dplyr)
library(tidyr)
library(caret)
library(e1071)
library(rpart)
library(rpart.plot)
library(glmnet)
library(latex2exp)
library(randomForest)
library(gbm)
library(tictoc)
library(patchwork)
library(pdp)
library(yardstick)
```

Exercise 3: Applied Classification Learning

Dataset: MNIST Handwritten Digits \ 10-class classification problem: digits 0–9

Part A. Data Preparation and Exploration

1. Data Loading and Inspection

```
#install.packages("dslabs")
library(dslabs)
#View(data.frame(mnist))
```

E3.A.1a Load MNIST data (training: 60,000 samples, test: 10,000 samples).

```r
# This downloads the data directly into an R list
mnist <- read_mnist()
df_train <- data.frame(mnist$train$images, label = mnist$train$labels)
df_test  <- data.frame(mnist$test$images,  label = mnist$test$labels)

df_full <- rbind(df_train, df_test)
```

Checking the dimensions

```r
dim(df_full)
```

```
## [1] 70000   785
```

```r
dim(df_train)
```

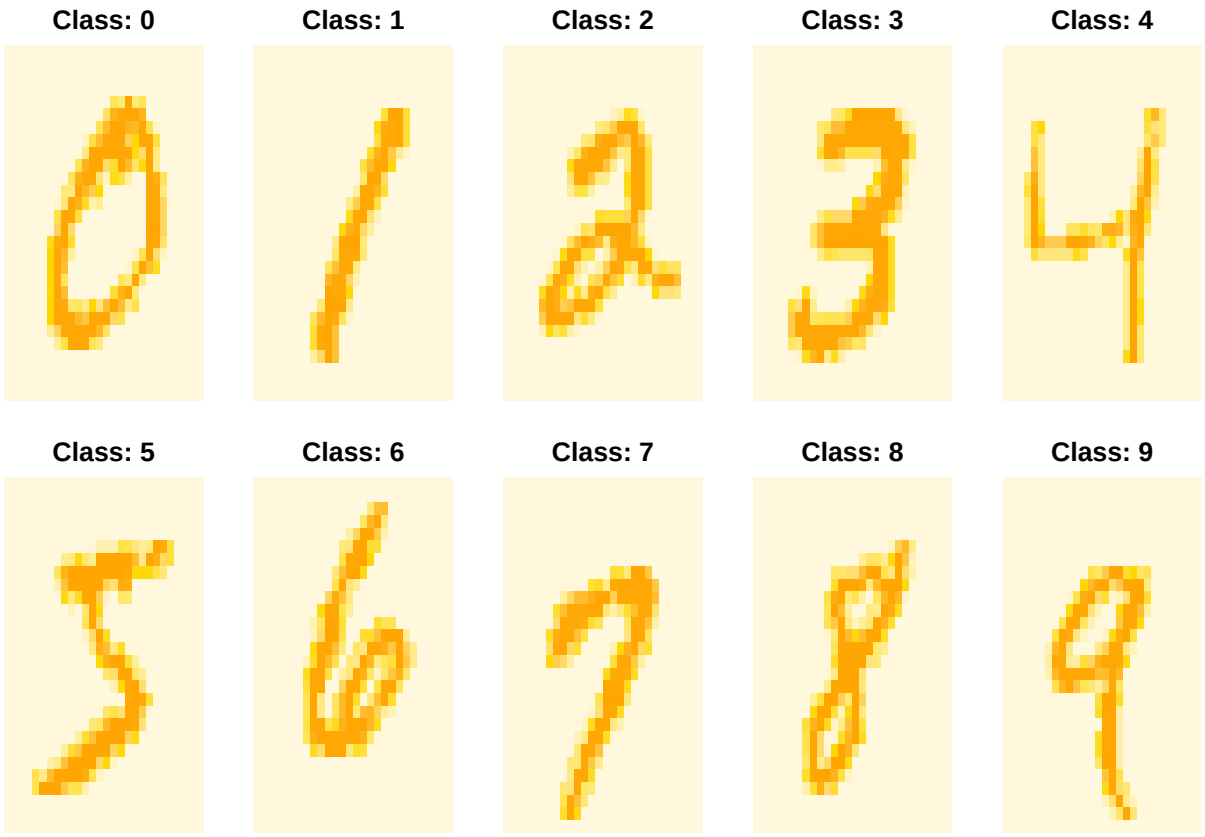
```
## [1] 60000   785
```

```r
dim(df_test)
```

```
## [1] 10000   785
```

E3.A.1b Visualize sample images from each class.

```r
par(mfrow = c(2, 5), mar = c(1, 1, 2, 1))
orange_palette <- colorRampPalette(c("cornsilk", "gold", "lightgoldenrod1", "orange"))(256)
# we can just plot the first 0-9 digits we see in df_train
for (digit in 0:9) {
  row_idx <- match(digit, df_train$label)

  pixel_vector <- as.numeric(df_train[row_idx, 1:784])
  img_matrix <- matrix(pixel_vector, nrow = 28, byrow = TRUE)
  img_matrix <- apply(img_matrix, 2, rev)

  image(1:28, 1:28, t(img_matrix),
        col = orange_palette,
        axes = FALSE,
        main = paste("Class:", digit))
}
```

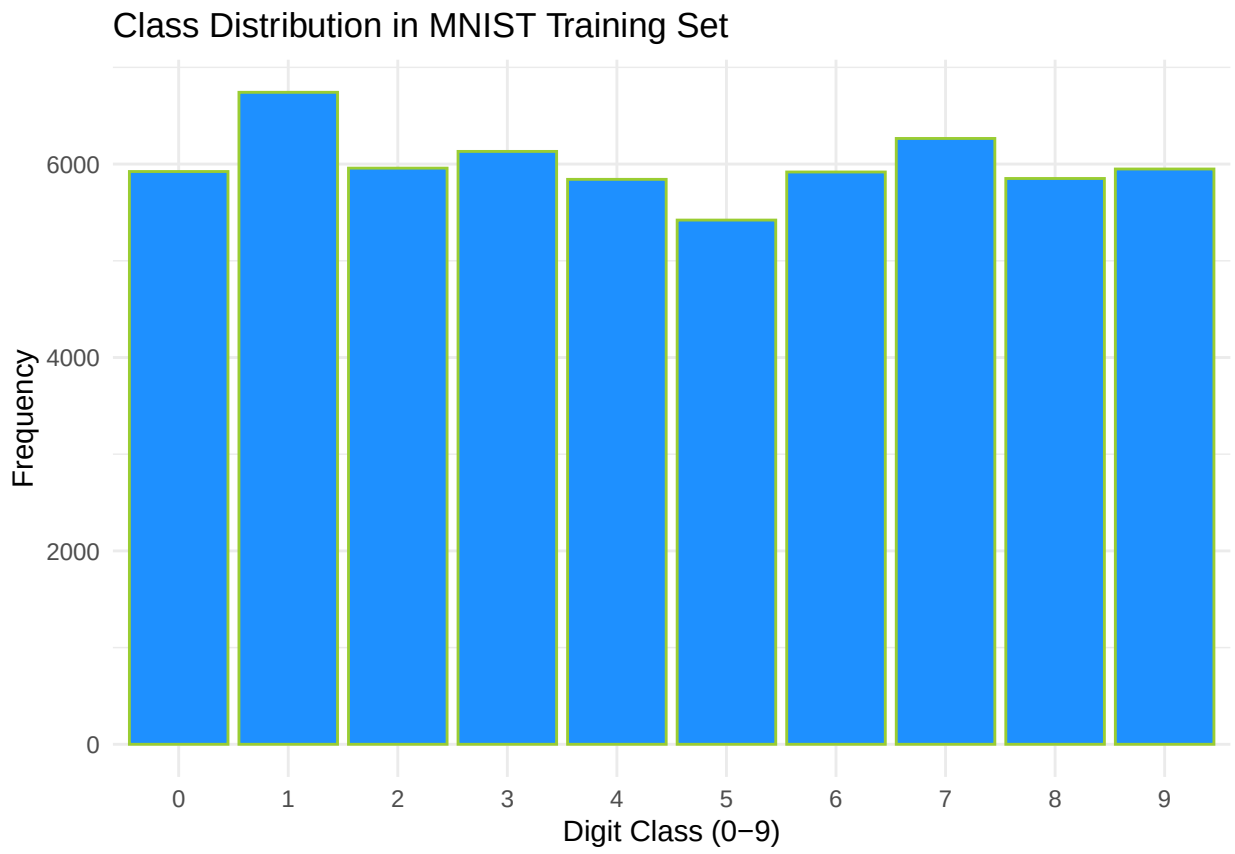


E3.A.1c Analyze class distribution and pixel value statistics. We can see that the class distribution is approximately uniform.

```
print(table(df_train$label))
```

```
##
##    0    1    2    3    4    5    6    7    8    9
## 5923 6742 5958 6131 5842 5421 5918 6265 5851 5949
```

```
ggplot(df_train, aes(x = factor(label))) +
  geom_bar(fill = "dodgerblue", color = "yellowgreen") +
  labs(title = "Class Distribution in MNIST Training Set",
       x = "Digit Class (0-9)",
       y = "Frequency") +
  theme_minimal()
```



2. Feature Engineering

E3.A.2a Implement dimensionality reduction (PCA) and visualize in 2D.

```
# the class label is in column
pc <- prcomp(df_train[, -785], center = TRUE, scale. = FALSE)
```

Checking the data after PCA

```
names(pc)
```

```
## [1] "sdev"     "rotation" "center"   "scale"    "x"
```

```
ncol(pc$x)
```

```
## [1] 784
```

```
dim(pc$x)
```

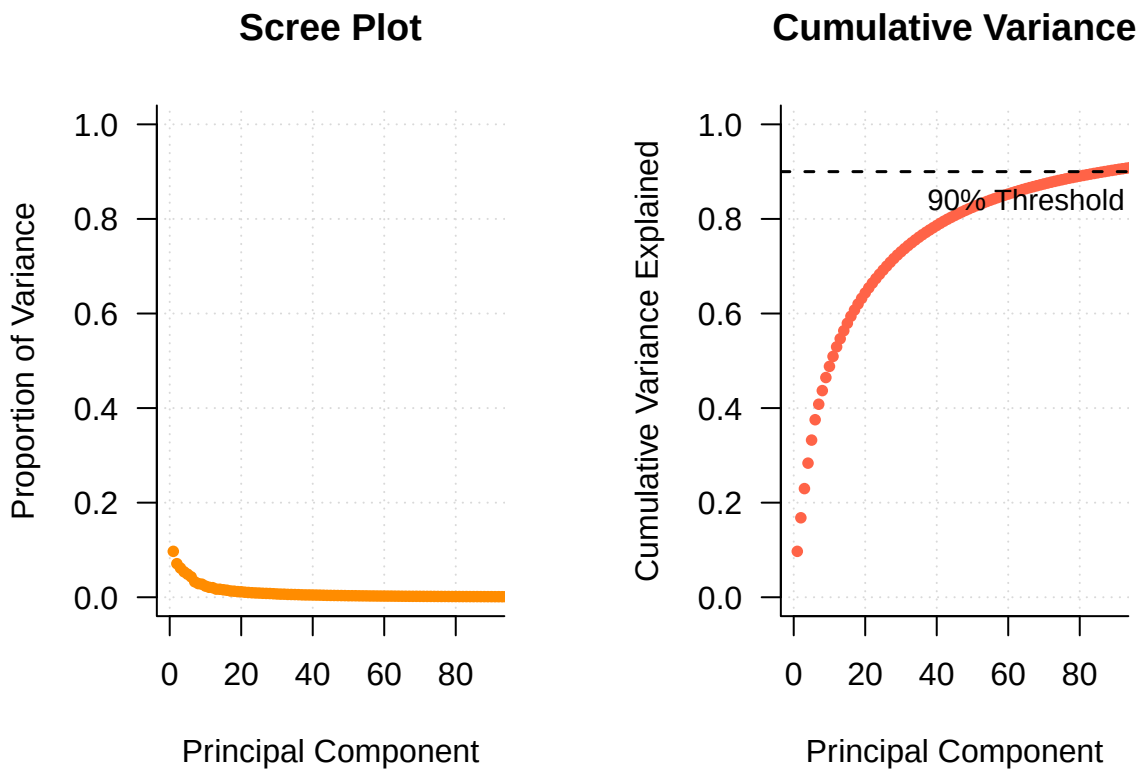
```
## [1] 60000   784
```

```
summary_pc <- summary(pc)
summary_pc$importance[, 1:10]
```

```
##                             PC1       PC2       PC3       PC4       PC5
## Standard deviation     576.82291 493.23822 459.89930 429.85624 408.56680
## Proportion of Variance   0.09705   0.07096   0.06169   0.05389   0.04869
## Cumulative Proportion    0.09705   0.16801   0.22970   0.28359   0.33228
##                             PC6       PC7       PC8       PC9      PC10
## Standard deviation     384.50613 334.93015 314.44305 307.72756 284.27069
## Proportion of Variance   0.04312   0.03272   0.02884   0.02762   0.02357
## Cumulative Proportion    0.37540   0.40812   0.43696   0.46458   0.48815
```

Plot the scree plot to see visualize the PC importance.

```
pc.var<- pc$sdev^2
propve <- pc.var/sum(pc.var)

par(mfrow = c(1, 2), mar = c(5, 5, 4, 2) + 0.1, las = 1)

plot(propve,
     xlab = "Principal Component",
     ylab = "Proportion of Variance",
     ylim = c(0, 1),
     xlim = c(0, 90),
     type = "b",
     pch = 16,
     col = "darkorange",
     lwd = 2,
     cex = 0.8,
     main = "Scree Plot",
     bty = "l",
     panel.first = grid(col = "gray85", lty = "dotted"))


plot(cumsum(propve),
     xlab = "Principal Component",
     ylab = "Cumulative Variance Explained",
     ylim = c(0, 1),
     xlim = c(0, 90),
     type = "b",
     pch = 16,
     col = "tomato",
     lwd = 2,
     cex = 0.8,
     main = "Cumulative Variance",
     bty = "l",
     panel.first = grid(col = "gray85", lty = "dotted"))

abline(h = 0.9, col = "black", lty = 2, lwd = 1.5)
text(x = 65, y = 0.84, labels = "90% Threshold", col = "black", cex = 0.9)
```



From this we know we need at least 87 Components to explain at least 90% of the variance.

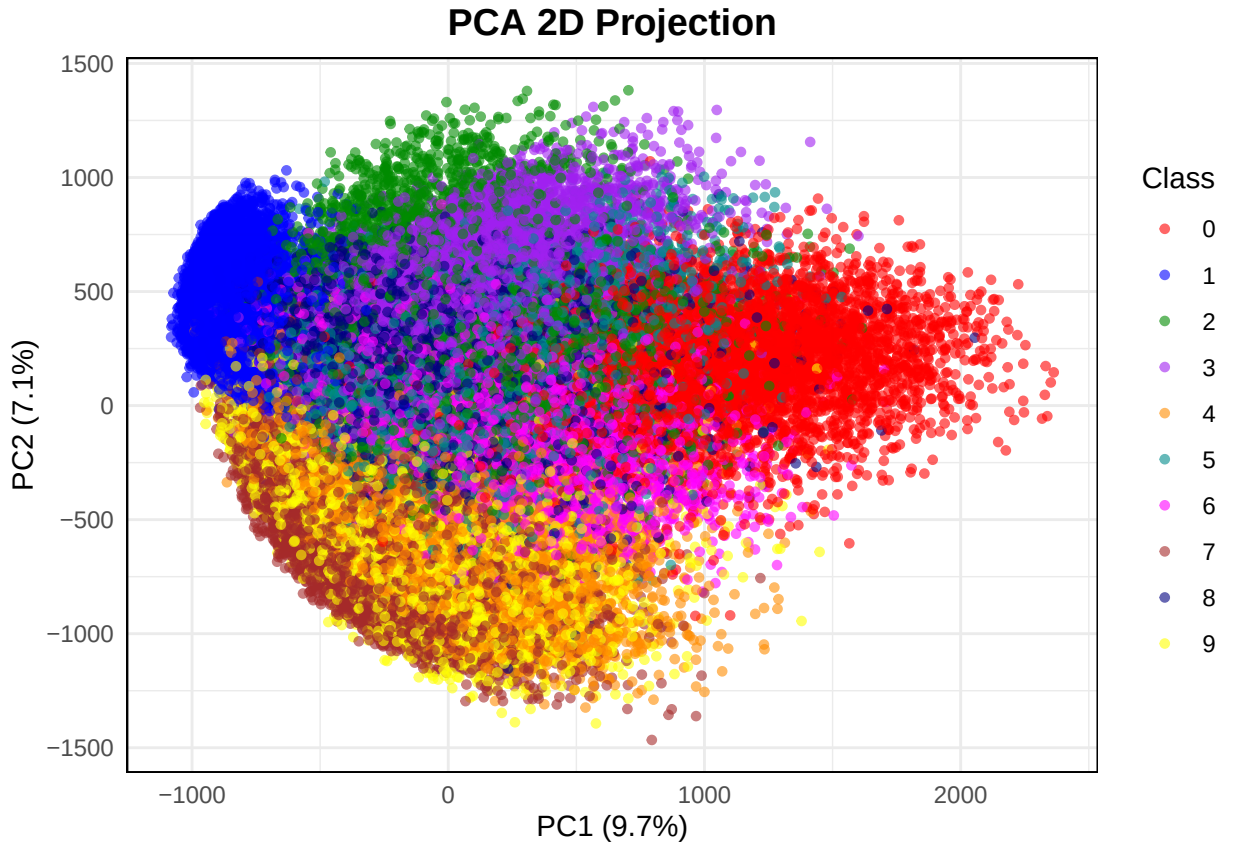
```
which(cumsum(propve) >= 0.9)[1]
```

```
## [1] 87
```

To visualize this in 2D we will take the top two PC.

```
pc_variance <- summary_pc$importance[2, 1:2] * 100
pc1_lab <- paste0("PC1 (", round(pc_variance[1], 1), "%)")
pc2_lab <- paste0("PC2 (", round(pc_variance[2], 1), "%)")

plot_data <- data.frame(
  PC1 = pc$x[, 1],
  PC2 = pc$x[, 2],
  Label = as.factor(df_train[, 785])
)


ggplot(plot_data, aes(x = PC1, y = PC2, color = Label)) +
  geom_point(alpha = 0.6, size = 1.2) +
  scale_color_manual(values = c("red", "blue", "green4", "purple", "darkorange",
                                "cyan4", "magenta", "brown", "navy", "yellow" )) +
  labs(
    title = "PCA 2D Projection",
```

```
    x = pc1_lab,
    y = pc2_lab,
    color = "Class"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold", size = 14),
    legend.position = "right",
    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5)
  )
```



E3.A.2b Create derived features (e.g., pixel intensity statistics, geometric moments).

The ij-th geometric Moment of an image is computed as

$$M_{ij} = \sum_x \sum_y x^i y^j I(x, y)$$

Where $I(x, y)$ is the pixel intersity at the coordinates (x, y).

```
X_raw <- as.matrix(df_train[, -785])

derive_features <- function(X) {
  t(apply(X, 1, function(px) {
    m       <- matrix(px, 28, 28)
    total   <- sum(px)
    cx      <- sum(col(m) * m) / (total + 1e-6)
```

```
    cy      <- sum(row(m) * m) / (total + 1e-6)
    c(mean     = mean(px),
      sd       = sd(px),
      max_px   = max(px),
      min_px   = min(px),
      skew     = mean((px - mean(px))^3) / (sd(px)^3 + 1e-6),
      kurt     = mean((px - mean(px))^4) / (sd(px)^4 + 1e-6),
      centroid_x = cx,
      centroid_y = cy,
      total_ink  = total)
  }))
}

#feeding the pixel as a matrix into the function
feat_train <- derive_features(as.matrix(df_train[, -785]))
feat_test  <- derive_features(as.matrix(df_test[,  -785]))
cat("Derived feature dimensions (train):", dim(feat_train), "\n")
```

```
## Derived feature dimensions (train): 60000 9
```

```
df_der <- data.frame(feat_train)
#df_der
```

E3.A.2c Compare raw pixels vs. engineered features in preliminary models.

```
set.seed(547)

# Sorry, computer lagged so I used a sample only but the results illustrate the point that the PCA feat
idx_small <- sample(1:nrow(df_train), 5000)

#Project the test data into the training PCA space
pc_test <- predict(pc, newdata = df_test[, -785])

num_pcs <- 87 # Recall this was the minimum PC needed for >= 90% variance explained


svm_raw  <- svm(pc$x[idx_small, 1:num_pcs],
                as.factor(df_train[idx_small, 785]),
                kernel = "radial", cost = 1, gamma = 0.01)

acc_raw  <- mean(predict(svm_raw,
                         pc_test[1:500, 1:num_pcs]) ==
                 as.factor(df_test[1:500, 785]))

# Engineered features model
svm_feat <- svm(feat_train[idx_small, ],
                as.factor(df_train[idx_small, 785]),
                kernel = "radial", cost = 1, gamma = 0.1)
```

```
## Warning in svm.default(feat_train[idx_small, ], as.factor(df_train[idx_small, :
## Variable(s) 'min_px' constant. Cannot scale data.
```

```
acc_feat <- mean(predict(svm_feat,
                         feat_test[1:500, ]) ==
                 as.factor(df_test[1:500, 785]))

cat("Accuracy - PCA pixels (Top 87) :",acc_raw, "\n")
```

```
## Accuracy - PCA pixels (Top 87) : 0.958
```

```
cat("Accuracy - Derived feats       :",acc_feat,  "\n")
```

```
## Accuracy - Derived feats       : 0.14
```

Further we can check out the confusion matrix, the SVM on Derived Features is almost worse than guessing.

```
pred_raw <- predict(svm_raw, pc_test[1:500, 1:num_pcs])


pred_feat <- predict(svm_feat, feat_test[1:500, ])

true_labels <- as.factor(df_test[1:500, 785])

confusionMatrix(pred_raw, true_labels)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1  2  3  4  5  6  7  8  9
##          0 42  0  0  0  0  1  1  0  0  0
##          1  0 67  0  0  0  0  0  0  0  0
##          2  0  0 52  0  1  0  0  0  0  1
##          3  0  0  0 41  0  1  0  0  0  1
##          4  0  0  0  0 53  0  0  0  0  1
##          5  0  0  0  1  1 48  1  0  0  0
##          6  0  0  0  1  0  0 41  0  0  0
##          7  0  0  1  1  0  0  0 48  0  2
##          8  0  0  1  1  0  0  0  0 40  2
##          9  0  0  1  0  0  0  0  1  0 47
##
## Overall Statistics
##
##                Accuracy : 0.958
##                  95% CI : (0.9365, 0.9738)
##     No Information Rate : 0.134
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.9532
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
```

```
##                      Class: 0 Class: 1 Class: 2 Class: 3 Class: 4 Class: 5
## Sensitivity            1.0000    1.000   0.9455   0.9111   0.9636   0.9600
## Specificity            0.9956    1.000   0.9955   0.9956   0.9978   0.9933
## Pos Pred Value         0.9545    1.000   0.9630   0.9535   0.9815   0.9412
## Neg Pred Value         1.0000    1.000   0.9933   0.9912   0.9955   0.9955
## Prevalence             0.0840    0.134   0.1100   0.0900   0.1100   0.1000
## Detection Rate         0.0840    0.134   0.1040   0.0820   0.1060   0.0960
## Detection Prevalence   0.0880    0.134   0.1080   0.0860   0.1080   0.1020
## Balanced Accuracy      0.9978    1.000   0.9705   0.9534   0.9807   0.9767
##                      Class: 6 Class: 7 Class: 8 Class: 9
## Sensitivity            0.9535   0.9796   1.0000   0.8704
## Specificity            0.9978   0.9911   0.9913   0.9955
## Pos Pred Value         0.9762   0.9231   0.9091   0.9592
## Neg Pred Value         0.9956   0.9978   1.0000   0.9845
## Prevalence             0.0860   0.0980   0.0800   0.1080
## Detection Rate         0.0820   0.0960   0.0800   0.0940
## Detection Prevalence   0.0840   0.1040   0.0880   0.0980
## Balanced Accuracy      0.9757   0.9854   0.9957   0.9329
```

```
confusionMatrix(pred_feat, true_labels)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1  2  3  4  5  6  7  8  9
##          0  4  0  4  2  2  4  1  0  2  3
##          1  1 22  2  2  3  1  1  7  1  2
##          2  6  2  3  5  4  2  5  2  7  3
##          3  2  2  4  4  4  4  3  0  3  5
##          4  4  0  6  5  3  3  3  5  2  3
##          5  3  2  3  2  1  6  2  3  2  6
##          6  6  1  4  4  8  5  6  6  6  4
##          7 12 34 21 13 18 21 14 15 10 20
##          8  3  1  3  4  4  2  3  2  4  5
##          9  1  3  5  4  8  2  5  9  3  3
##
## Overall Statistics
##
##                Accuracy : 0.14
##                  95% CI : (0.1108, 0.1735)
##     No Information Rate : 0.134
##     P-Value [Acc > NIR] : 0.366
##
##                   Kappa : 0.0441
##
##  Mcnemar's Test P-Value : 1.989e-08
##
## Statistics by Class:
##
##                      Class: 0 Class: 1 Class: 2 Class: 3 Class: 4 Class: 5
## Sensitivity           0.09524   0.3284  0.05455  0.08889  0.05455   0.1200
## Specificity           0.96070   0.9538  0.91910  0.94066  0.93034   0.9467
## Pos Pred Value        0.18182   0.5238  0.07692  0.12903  0.08824   0.2000
## Neg Pred Value        0.92050   0.9017  0.88720  0.91258  0.88841   0.9064
```

```
## Prevalence            0.08400   0.1340  0.11000  0.09000  0.11000   0.1000
## Detection Rate        0.00800   0.0440  0.00600  0.00800  0.00600   0.0120
## Detection Prevalence  0.04400   0.0840  0.07800  0.06200  0.06800   0.0600
## Balanced Accuracy     0.52797   0.6411  0.48682  0.51477  0.49244   0.5333
##                      Class: 6 Class: 7 Class: 8 Class: 9
## Sensitivity            0.1395  0.30612   0.1000  0.05556
## Specificity            0.9037  0.63858   0.9413  0.91031
## Pos Pred Value         0.1200  0.08427   0.1290  0.06977
## Neg Pred Value         0.9178  0.89441   0.9232  0.88840
## Prevalence             0.0860  0.09800   0.0800  0.10800
## Detection Rate         0.0120  0.03000   0.0080  0.00600
## Detection Prevalence   0.1000  0.35600   0.0620  0.08600
## Balanced Accuracy      0.5216  0.47235   0.5207  0.48293
```

3. Data Subsetting

E3.A.3a Create balanced subsets for efficient experimentation.

For this we create a subset data set

$$\mathcal{D}_s \subset \mathbb{R}^{10000 \times 785}$$

that is randomly sampled from `df_train`such that each class $\sum 1(y_i = g) = 1000, \quad g = 0, 1, 2, \cdots, 9$ which is random sampled from `df_train`.

```r
set.seed(547)
n_per_class <- 1000   # 1000 per digit → 10,000 total

idx_balanced <- unlist(lapply(0:9, function(d) {
  which_d <- which(df_train$label == d)
  sample(which_d, n_per_class)
}))

df_balanced <- df_train[idx_balanced, ]
table(df_balanced$label)
```

```
##
##    0    1    2    3    4    5    6    7    8    9
## 1000 1000 1000 1000 1000 1000 1000 1000 1000 1000
```

E3.A.3b For computational efficiency, work with 10,000 training samples initially.

```r
X_sub   <- as.matrix(df_balanced[, -785])
y_sub   <- as.factor(df_balanced$label)

pc_sub  <- pc$x[idx_balanced, 1:num_pcs]
dim(pc_sub)
```

```
## [1] 10000    87
```

```r
dim(X_sub)
```

```
## [1] 10000   784
```

E3.A.3c Create validation split for hyperparameter tuning.

```r
set.seed(547)
val_frac   <- 0.2
val_idx    <- sample(1:nrow(df_balanced), size = floor(val_frac * nrow(df_balanced)))
train_idx  <- setdiff(1:nrow(df_balanced), val_idx)

# Raw pixel splits
X_tr  <- X_sub[train_idx, ];  y_tr  <- y_sub[train_idx]
X_val <- X_sub[val_idx,   ];  y_val <- y_sub[val_idx]

# PCA splits
pc_tr  <- pc_sub[train_idx, ];  pc_val <- pc_sub[val_idx, ]

# Derived-feature splits
ft_tr  <- feat_train[idx_balanced[train_idx], ]
ft_val <- feat_train[idx_balanced[val_idx],   ]

cat("Train:", nrow(X_tr), "| Val:", nrow(X_val), "\n")
```

```
## Train: 8000 | Val: 2000
```

```r
cat("Train:", nrow(pc_tr), "| Val:", nrow(pc_val), "\n")
```

```
## Train: 8000 | Val: 2000
```

```r
cat("Train:", nrow(ft_tr), "| Val:", nrow(ft_val), "\n")
```

```
## Train: 8000 | Val: 2000
```

Part B. Support Vector Classification

1. Binary Classification E3.B.1a Implement one-vs-rest strategy for multi-class classification. It is arguable that handwritten 3s are frequently confused with 8, 5, and sometimes 9, making it a non-trivial binary problem worth studying. **E3.B.1b** Train SVM with RBF kernel for digit "3" vs. all others. **E3.B.1c** Analyze decision boundary and support vectors.

```r
# One-vs-rest labels
y_bin_tr  <- as.factor(ifelse(y_tr  == 3, "three", "other"))
y_bin_val <- as.factor(ifelse(y_val == 3, "three", "other"))

tic("Binary SVM fit")
svm_bin <- svm(pc_tr, y_bin_tr,
               kernel = "radial", cost = 1, gamma = 0.01,
               probability = TRUE)
toc()
```

```
## Binary SVM fit: 14.7 sec elapsed
```

```r
pred_bin <- predict(svm_bin, pc_val)
cat("Binary accuracy (3 vs rest):", mean(pred_bin == y_bin_val), "\n")
```

```
## Binary accuracy (3 vs rest): 0.9905
```

```
confusionMatrix(pred_bin, y_bin_val)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction other three
##      other  1802    17
##      three     2   179
##
##                Accuracy : 0.9905
##                  95% CI : (0.9852, 0.9943)
##     No Information Rate : 0.902
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.9444
##
##  Mcnemar's Test P-Value : 0.001319
##
##             Sensitivity : 0.9989
##             Specificity : 0.9133
##          Pos Pred Value : 0.9907
##          Neg Pred Value : 0.9890
##              Prevalence : 0.9020
##          Detection Rate : 0.9010
##    Detection Prevalence : 0.9095
##       Balanced Accuracy : 0.9561
##
##        'Positive' Class : other
##
```

```
# Support vector count
cat("Number of support vectors:", nrow(svm_bin$SV), "\n")
```

```
## Number of support vectors: 1462
```

```
cat("Support vectors per class:\n"); print(svm_bin$nSV)
```

```
## Support vectors per class:
```

```
## [1] 943 519
```

The binary SVM with an RBF kernel (trained on the top 87 principal components) achieves an overall accuracy of 99.05% on the validation set, well above the no-information rate of 90.2% (the rate achieved by predicting `other` for every sample).

2. Multi-class Classification

E3.B.2a Train multi-class SVM with RBF kernel.

```
tic("Multiclass SVM fit")
svm_multi <- svm(pc_tr, y_tr,
                 kernel = "radial", cost = 1, gamma = 0.01,
                 decision.values = TRUE)
toc()
```

```
## Multiclass SVM fit: 16.85 sec elapsed
```

```
pred_multi <- predict(svm_multi, pc_val)
cat("Multi-class accuracy:", mean(pred_multi == y_val), "\n")
```

```
## Multi-class accuracy: 0.958
```

E3.B.2b Use cross-validation to tune C and γ.

```
set.seed(547)
tune_grid <- tune(svm,
                  train.x = pc_tr,
                  train.y = y_tr,
                  kernel   = "radial",
                  ranges   = list(cost  = c(0.1, 1, 10),
                                  gamma = c(0.001, 0.01, 0.1)),
                  tunecontrol = tune.control(cross = 2))

summary(tune_grid)
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 2-fold cross validation
##
## - best parameters:
##  cost gamma
##    10  0.01
##
## - best performance: 0.0545
##
## - Detailed performance results:
##   cost gamma    error  dispersion
## 1  0.1 0.001 0.566625 0.076190756
## 2  1.0 0.001 0.103875 0.004065864
## 3 10.0 0.001 0.078750 0.001414214
## 4  0.1 0.010 0.126625 0.022097087
## 5  1.0 0.010 0.057625 0.005480078
## 6 10.0 0.010 0.054500 0.002474874
## 7  0.1 0.100 0.886750 0.016263456
## 8  1.0 0.100 0.640875 0.021743534
## 9 10.0 0.100 0.606000 0.026516504
```

```
best_cost  <- tune_grid$best.parameters$cost
best_gamma <- tune_grid$best.parameters$gamma
cat("Best cost:", best_cost, "| Best gamma:", best_gamma, "\n")
```

```
## Best cost: 10 | Best gamma: 0.01
```

```
svm_tuned <- svm(pc_tr, y_tr,
                 kernel = "radial",
                 cost   = best_cost,
                 gamma  = best_gamma)
pred_tuned <- predict(svm_tuned, pc_val)
cat("Tuned SVM accuracy:", mean(pred_tuned == y_val), "\n")
```

```
## Tuned SVM accuracy: 0.963
```

E3.B.2c Analyze confusion matrix and per-class performance. From the confusion matrix shows that the multi-classed SVM performs very well, only 2-3 specific number is even a notable error. In particularly, 6 instances of predicting 3 when the true class is 5, and 5 instances of predicting 4 when the true class is 9. It can been seen that 3 and 5 can be written quite similary in those 6 samples and same goes for 3 and 5.

```
cm_multi <- confusionMatrix(pred_tuned, y_val)
print(cm_multi)
```

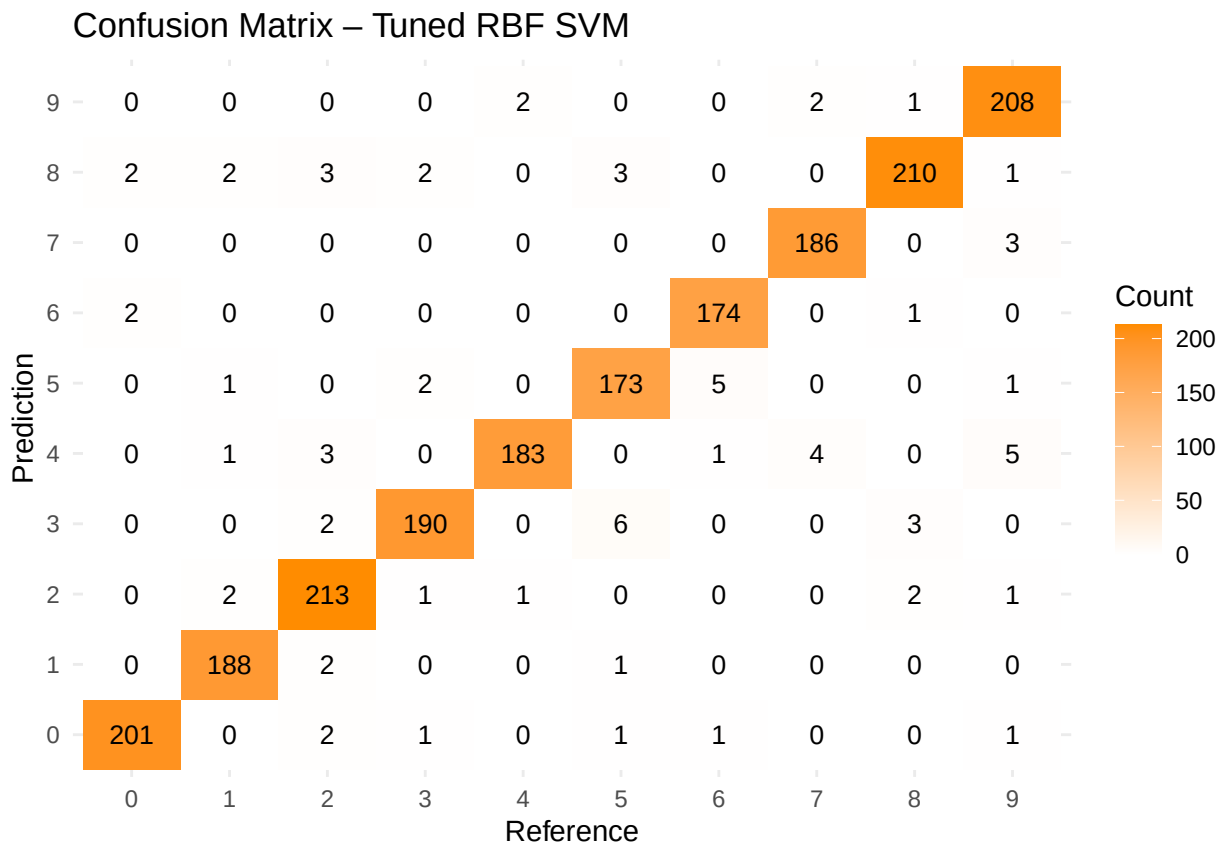
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1   2   3   4   5   6   7   8   9
##          0 201   0   2   1   0   1   1   0   0   1
##          1   0 188   2   0   0   1   0   0   0   0
##          2   0   2 213   1   1   0   0   0   2   1
##          3   0   0   2 190   0   6   0   0   3   0
##          4   0   1   3   0 183   0   1   4   0   5
##          5   0   1   0   2   0 173   5   0   0   1
##          6   2   0   0   0   0   0 174   0   1   0
##          7   0   0   0   0   0   0   0 186   0   3
##          8   2   2   3   2   0   3   0   0 210   1
##          9   0   0   0   0   2   0   0   2   1 208
##
## Overall Statistics
##
##                Accuracy : 0.963
##                  95% CI : (0.9538, 0.9708)
##     No Information Rate : 0.1125
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.9589
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: 0 Class: 1 Class: 2 Class: 3 Class: 4 Class: 5
## Sensitivity            0.9805   0.9691   0.9467   0.9694   0.9839   0.9402
## Specificity            0.9967   0.9983   0.9961   0.9939   0.9923   0.9950
## Pos Pred Value         0.9710   0.9843   0.9682   0.9453   0.9289   0.9505
## Neg Pred Value         0.9978   0.9967   0.9933   0.9967   0.9983   0.9939
```

```
## Prevalence             0.1025   0.0970   0.1125   0.0980   0.0930   0.0920
## Detection Rate         0.1005   0.0940   0.1065   0.0950   0.0915   0.0865
## Detection Prevalence   0.1035   0.0955   0.1100   0.1005   0.0985   0.0910
## Balanced Accuracy      0.9886   0.9837   0.9714   0.9816   0.9881   0.9676
##                      Class: 6 Class: 7 Class: 8 Class: 9
## Sensitivity            0.9613   0.9688   0.9677   0.9455
## Specificity            0.9984   0.9983   0.9927   0.9972
## Pos Pred Value         0.9831   0.9841   0.9417   0.9765
## Neg Pred Value         0.9962   0.9967   0.9961   0.9933
## Prevalence             0.0905   0.0960   0.1085   0.1100
## Detection Rate         0.0870   0.0930   0.1050   0.1040
## Detection Prevalence   0.0885   0.0945   0.1115   0.1065
## Balanced Accuracy      0.9798   0.9835   0.9802   0.9713
```

```r
# Per-class F1, precision, recall
class_stats <- data.frame(
  Digit     = 0:9,
  Precision = cm_multi$byClass[, "Precision"],
  Recall    = cm_multi$byClass[, "Recall"],
  F1        = cm_multi$byClass[, "F1"]
)
print(class_stats)
```

```
##          Digit Precision    Recall        F1
## Class: 0     0 0.9710145 0.9804878 0.9757282
## Class: 1     1 0.9842932 0.9690722 0.9766234
## Class: 2     2 0.9681818 0.9466667 0.9573034
## Class: 3     3 0.9452736 0.9693878 0.9571788
## Class: 4     4 0.9289340 0.9838710 0.9556136
## Class: 5     5 0.9505495 0.9402174 0.9453552
## Class: 6     6 0.9830508 0.9613260 0.9720670
## Class: 7     7 0.9841270 0.9687500 0.9763780
## Class: 8     8 0.9417040 0.9677419 0.9545455
## Class: 9     9 0.9765258 0.9454545 0.9607390
```

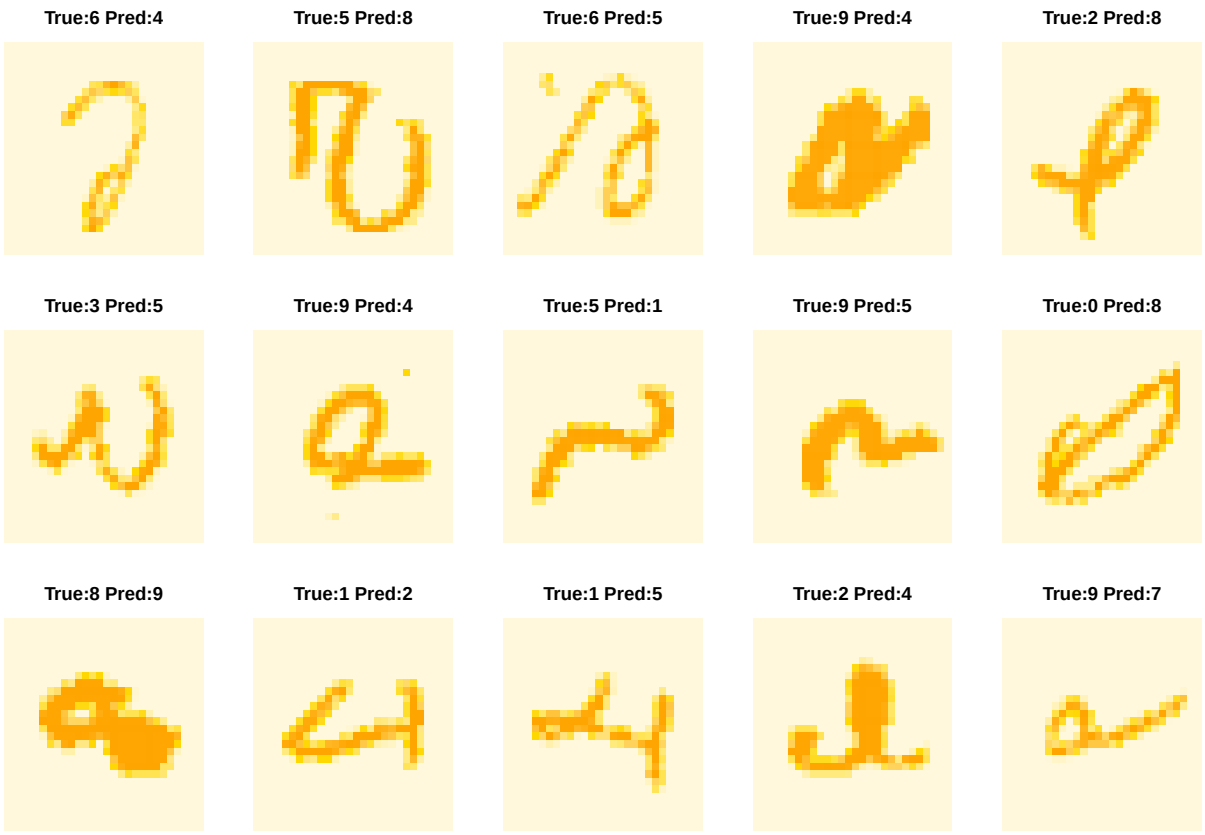
```r
# Heatmap of confusion matrix
cm_df <- as.data.frame(cm_multi$table)
ggplot(cm_df, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = Freq), size = 3.5) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  labs(title = "Confusion Matrix - Tuned RBF SVM", fill = "Count") +
  theme_minimal()
```



E3.B.2d Visualize misclassified examples and identify patterns.

We extract the misclassified examples and see that the main reason for misclassification is due to the handwriting of the specific sample seems to deviate a lot from how the number should be written.

```
wrong_idx    <- which(pred_tuned != y_val)
wrong_true   <- y_val[wrong_idx]
wrong_pred   <- pred_tuned[wrong_idx]
wrong_pixels <- X_val[wrong_idx, ]

par(mfrow = c(3, 5), mar = c(1, 1, 2, 1))
for (k in 1:min(15, length(wrong_idx))) {
  img <- matrix(wrong_pixels[k, ], 28, 28)
  img <- apply(img, 2, rev)
  image(t(img), col = orange_palette, axes = FALSE,
        main = paste0("True:", wrong_true[k], " Pred:", wrong_pred[k]),
        cex.main = 0.9)
}
```



3. Advanced Kernels

E3.B.1a Experiment with polynomial kernels.

We experimented with polynomial kernels of degree 2, 3, 4 and the performance in Validation accuracy is approximately the same as the RBF Tuned model surprisingly, however RBF kernel still have the upperhand.

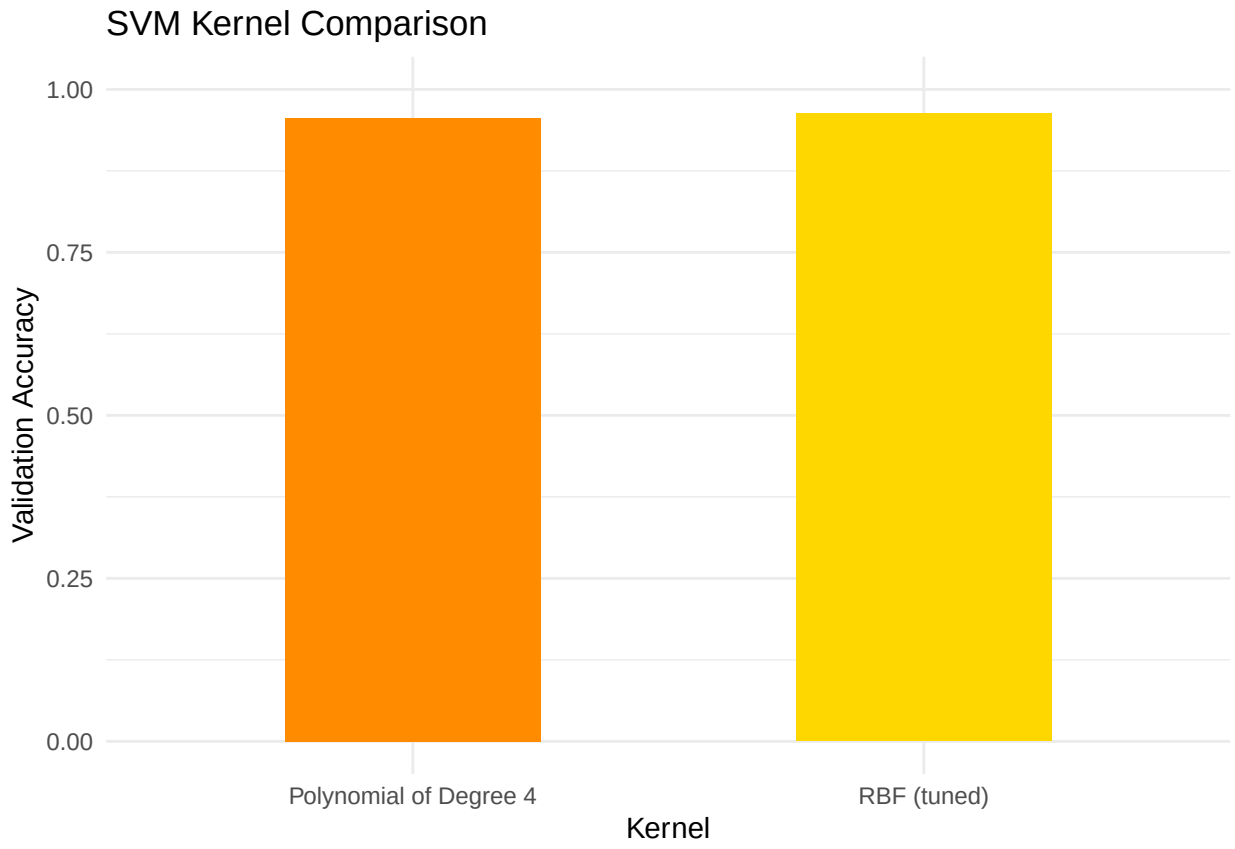
```
tic("Polynomial SVM")
svm_poly <- svm(pc_tr, y_tr,
                kernel = "polynomial", degree = 4,
                cost = 1, coef0 = 1)




pred_poly <- predict(svm_poly, pc_val)
acc_poly  <- mean(pred_poly == y_val)
acc_rbf   <- mean(pred_tuned == y_val)

cat("RBF accuracy       :", round(acc_rbf,  4), "\n")
```

```
## RBF accuracy       : 0.963
```

```
cat("Polynomial accuracy:", round(acc_poly, 4), "\n")
```

```
## Polynomial accuracy: 0.956
```

```r
# Summary bar chart
acc_df <- data.frame(
  Kernel   = c("RBF (tuned)", "Polynomial of Degree 4"),
  Accuracy = c(acc_rbf, acc_poly)
)
ggplot(acc_df, aes(x = Kernel, y = Accuracy, fill = Kernel)) +
  geom_col(width = 0.5) +
  scale_fill_manual(values = c("darkorange", "gold")) +
  ylim(0, 1) +
  labs(title = "SVM Kernel Comparison", y = "Validation Accuracy") +
  theme_minimal() + theme(legend.position = "none")
```



E3.B.1b Discuss which kernel works best for image data and why.

Based on the performances we saw for RBF kernel on multiclass classification of 96.3% and polynomial kernel of 95.6%, we would say RBF kernel is the better choice and it allows for tunning with the hyperparamter with γ so we can control the radius of influence of each support vector. If we were to tune d and c in polynomial kernel it is a most expensive computation.

E3.B.1c Analyze computational complexity vs. performance.

Both kernels share the same asymptotic training complexity of $O(n^2p)$ to $O(n^3)$ in the number of training samples n and features p, since the SVM quadratic program must evaluate kernel values between all pairs of points. In practice, however, the polynomial kernel is slower per iteration because each kernel evaluation involves a dot product followed by an exponentiation, whereas the RBF kernel requires computing a squared Euclidean distance.

Part C. Tree-Based Classification

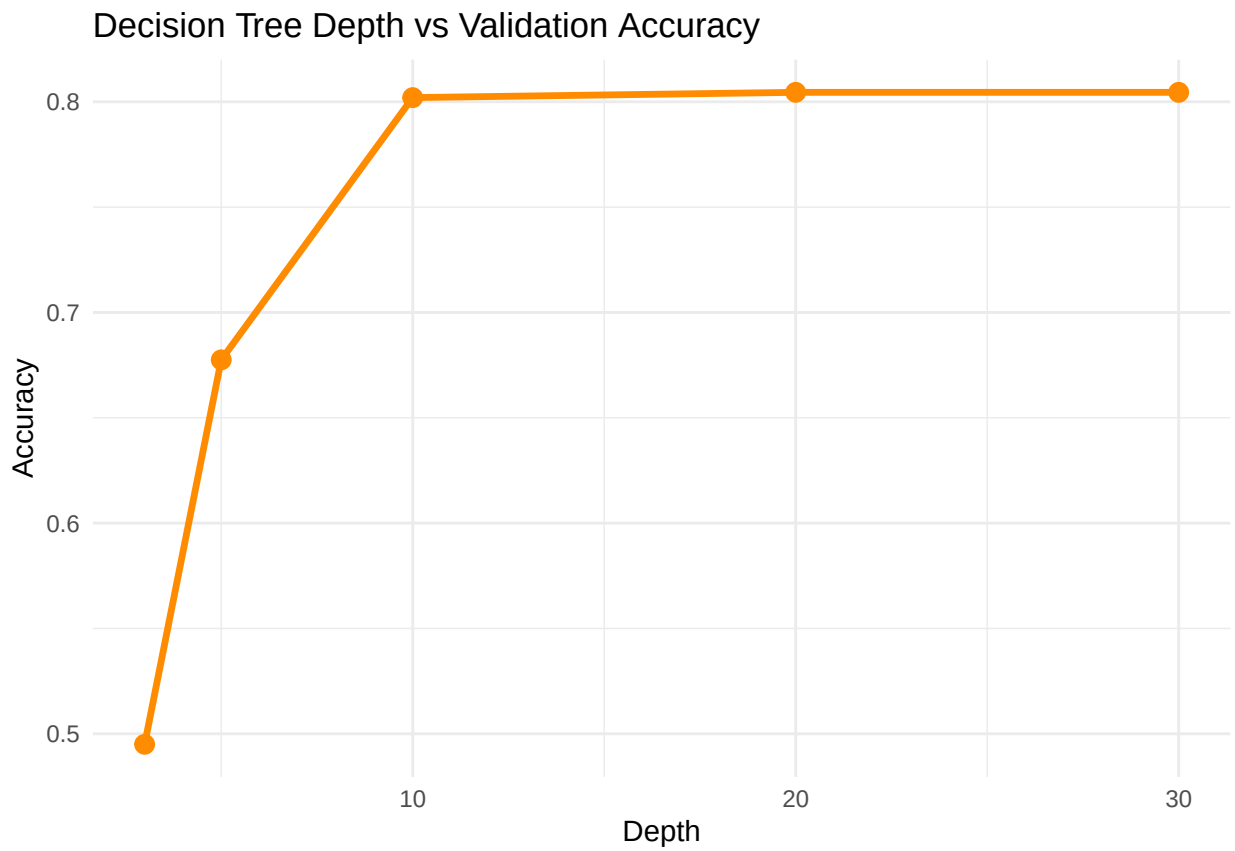
1. Single Decision Tree

E3.C.1a Train a decision tree with various depth limits.

```
depths     <- c(3, 5, 10, 20, 30)
acc_depths <- numeric(length(depths))

for (i in seq_along(depths)) {
  dt <- rpart(label ~ ., data = data.frame(X_tr, label = y_tr),
              method = "class",
              control = rpart.control(maxdepth = depths[i], cp = 0))
  pred_dt      <- predict(dt, data.frame(X_val), type = "class")
  acc_depths[i] <- mean(pred_dt == y_val)
}

depth_df <- data.frame(Depth = depths, Accuracy = acc_depths)
print(depth_df)
```

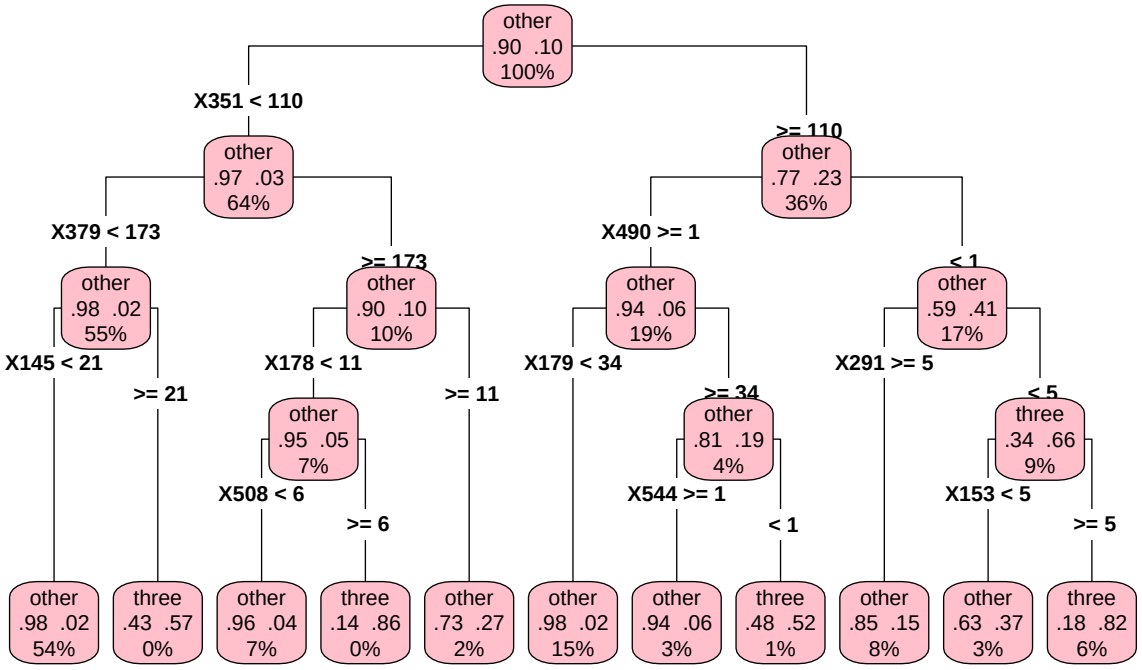
```
##   Depth Accuracy
## 1     3   0.4950
## 2     5   0.6775
## 3    10   0.8020
## 4    20   0.8045
## 5    30   0.8045
```

```
ggplot(depth_df, aes(x = Depth, y = Accuracy)) +
  geom_line(color = "darkorange", lwd = 1.2) +
  geom_point(size = 3, color = "darkorange") +
  labs(title = "Decision Tree Depth vs Validation Accuracy") +
  theme_minimal()
```



E3.C.1b Visualize the tree structure for a binary subproblem.

```
y_bin_tr_fac <- as.factor(ifelse(y_tr == 3, "three", "other"))

dt_bin <- rpart(label ~ .,
                data    = data.frame(X_tr, label = y_bin_tr_fac),
                method  = "class",
                control = rpart.control(maxdepth = 4, cp = 0))

rpart.plot(dt_bin, type = 4, extra = 104,
           main = "Decision Tree: Digit 3 vs All (depth=4)",
           col  = "black", box.palette = "pink", cex = 0.67)
```



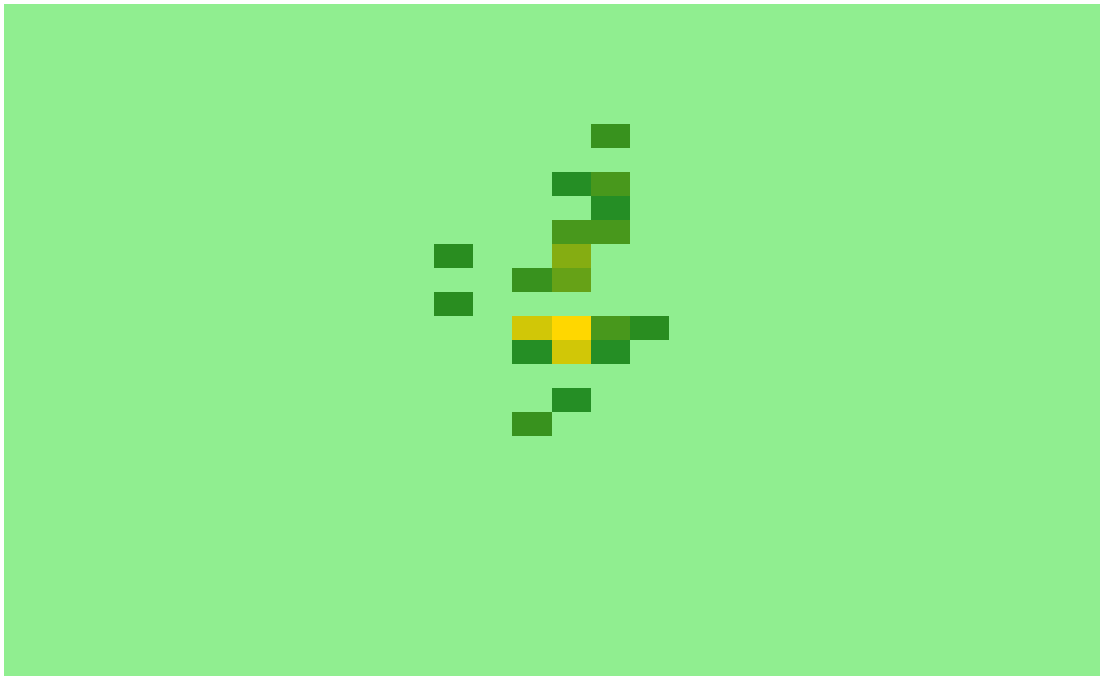
From this tree we can clearly trace down the pixels that decide wether the image will be a 3 or other number.

E3.C.1c Analyze which pixels are most important for classification.

Since most of the images has the non-zero valued pixel centered, it is intuitive that the most important pixels would be highlighted in the middle.

```
best_dt <- rpart(label ~ ., data = data.frame(X_tr, label = y_tr),
                 method = "class",
                 control = rpart.control(maxdepth = 20, cp = 0))

imp <- best_dt$variable.importance
top_pixels <- sort(imp, decreasing = TRUE)[1:20]
```

```
# Map top pixels back onto 28×28 grid
imp_map <- numeric(784)
names(imp_map) <- paste0("X", 1:784)
imp_map[names(top_pixels)] <- top_pixels

imp_matrix <- matrix(imp_map, 28, 28, byrow = TRUE)
imp_matrix <- apply(imp_matrix, 2, rev)

par(mar = c(2, 2, 3, 2))
image(t(imp_matrix),
      col = colorRampPalette(c("lightgreen", "forestgreen", "gold"))(30),
      axes = FALSE,
      main = "Top-20 Important Pixels (Decision Tree)")
```



2. Ensemble Methods

E3.C.2a Train random forest with various numbers of trees. The following chunk took a long time to run, so the output will be reprinted here:

RF ntree = 50: 24 sec elapsed RF ntree = 100: 48.45 sec elapsed RF ntree = 200: 91 sec elapsed RF ntree = 500: 238.59 sec elapsed NTrees Val_Accuracy 1 50 0.9410 2 100 0.9470 3 200 0.9535 4 500 0.9535 Final RF: 178.28 sec elapsed

Call: randomForest(x = X_tr, y = y_tr, ntree = best_ntree, mtry = 28, importance = TRUE) Type of random forest: classification Number of trees: 200 No. of variables tried at each split: 28

OOB estimate of error rate: 5.27% Confusion matrix: 0 1 2 3 4 5 6 7 8 9 class.error 0 782 0 1 1 1 0 2 0 7 1 0.01635220 1 0 784 4 5 4 1 2 1 3 2 0.02729529 2 3 3 724 9 8 1 7 11 6 3 0.06580645 3 3 3 10 743 0 17 2 6 13 7 0.07587065 4 0 0 3 1 778 1 4 0 6 21 0.04422604 5 4 4 4 10 4 768 14 2 3 3 0.05882353 6 5 1 4 0 1 10 795 0 3 0 0.02930403 7 2 6 10 0 5 0 0 767 3 15 0.05074257 8 2 6 9 22 4 13 4 2 709 12 0.09450830 9 5 0 4 12 15 1 0 8 7 728 0.06666667

Taking it over from here, We can see that increasing from 200 trees to 500 tree have little to no improvement on the validation accuracy, so we can claim that with high confidence that the the we have converged to highest possible validation accuracy.

```
ntree_vals <- c(50, 100, 200, 500)
acc_rf     <- numeric(length(ntree_vals))
oob_rf     <- numeric(length(ntree_vals))

for (i in seq_along(ntree_vals)) {
```

```
  tic(paste("RF ntree =", ntree_vals[i]))
  rf_i <- randomForest(x = X_tr, y = y_tr,
                       ntree = ntree_vals[i], mtry = 28,
                       importance = FALSE)
  toc()
  pred_rf_i  <- predict(rf_i, X_val)
  acc_rf[i]  <- mean(pred_rf_i == y_val)
  oob_rf[i]  <- 1 - mean(rf_i$confusion[, 1:10] * diag(10)) # approx
}
```

```
## RF ntree = 50: 23.6 sec elapsed
## RF ntree = 100: 45.83 sec elapsed
## RF ntree = 200: 91.42 sec elapsed
## RF ntree = 500: 229.09 sec elapsed
```

```
rf_df <- data.frame(NTrees = ntree_vals, Val_Accuracy = acc_rf)
print(rf_df)
```

```
##   NTrees Val_Accuracy
## 1     50       0.9410
## 2    100       0.9470
## 3    200       0.9535
## 4    500       0.9535
```

```
best_ntree <- ntree_vals[which.max(acc_rf)]
tic("Final RF")
rf_final <- randomForest(x = X_tr, y = y_tr,
                         ntree = best_ntree, mtry = 28,
                         importance = TRUE)
toc()
```

```
## Final RF: 171.64 sec elapsed
```

```
rf_final
```

```
##
## Call:
##  randomForest(x = X_tr, y = y_tr, ntree = best_ntree, mtry = 28,      importance = TRUE)
##                Type of random forest: classification
##                      Number of trees: 200
## No. of variables tried at each split: 28
##
##         OOB estimate of  error rate: 5.27%
## Confusion matrix:
##     0   1   2   3   4   5   6   7   8   9 class.error
## 0 782   0   1   1   1   0   2   0   7   1  0.01635220
## 1   0 784   4   5   4   1   2   1   3   2  0.02729529
## 2   3   3 724   9   8   1   7  11   6   3  0.06580645
## 3   3   3  10 743   0  17   2   6  13   7  0.07587065
## 4   0   0   3   1 778   1   4   0   6  21  0.04422604
## 5   4   4   4  10   4 768  14   2   3   3  0.05882353
```

```
## 6   5   1   4   0   1  10 795   0   3   0  0.02930403
## 7   2   6  10   0   5   0   0 767   3  15  0.05074257
## 8   2   6   9  22   4  13   4   2 709  12  0.09450830
## 9   5   0   4  12  15   1   0   8   7 728  0.06666667
```

```
best_ntree
```

```
## [1] 200
```

```
rf_df
```

```
##   NTrees Val_Accuracy
## 1     50       0.9410
## 2    100       0.9470
## 3    200       0.9535
## 4    500       0.9535
```

```
df_gbm_tr  <- data.frame(X_tr,  label = as.integer(as.character(y_tr)))
df_gbm_val <- data.frame(X_val, label = as.integer(as.character(y_val)))
```

```
gbm_fit <- gbm(label ~ .,
               data         = df_gbm_tr,
               distribution = "multinomial",
               n.trees      = 100,
               interaction.depth = 2,
               shrinkage    = 0.1,
               bag.fraction = 0.5,
               n.minobsinnode = 20,
               train.fraction = 0.8,   # internal early stopping
               verbose      = FALSE)
```

```
## Warning: Setting ‘distribution = "multinomial"‘ is ill-advised as it is
## currently broken. It exists only for backwards compatibility. Use at your own
## risk.
```

```
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 1: X1 has no variation.
```

```
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 2: X2 has no variation.
```

```
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 3: X3 has no variation.
```

```
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 4: X4 has no variation.
```

```
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 5: X5 has no variation.
```

```
## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 780: X780 has no variation.

## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 781: X781 has no variation.

## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 782: X782 has no variation.

## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 783: X783 has no variation.

## Warning in gbm.fit(x = x, y = y, offset = offset, distribution = distribution,
## : variable 784: X784 has no variation.
```

E3.C.2c Compare out-of-bag error with cross-validation error.

OOB Error : 0.0528 CV Error : 0.0603 +/- 0.0034

```
# OOB from randomForest object
oob_error <- rf_final$err.rate[best_ntree, "OOB"]

# 3-fold CV error on training set (reuse tune or manual loop)
set.seed(547)
k <- 3
folds <- createFolds(y_tr, k = k, list = TRUE)
cv_errors <- numeric(k)

for (f in seq_along(folds)) {
  cv_tr_idx  <- unlist(folds[-f])
  cv_val_idx <- folds[[f]]
  rf_cv <- randomForest(x = X_tr[cv_tr_idx, ],
                        y = y_tr[cv_tr_idx],
                        ntree = best_ntree, mtry = 28)
  pred_cv     <- predict(rf_cv, X_tr[cv_val_idx, ])
  cv_errors[f] <- 1 - mean(pred_cv == y_tr[cv_val_idx])
}

cat("OOB Error   :", round(oob_error,    4), "\n")
```

```
## OOB Error   : 0.0528
```

```
cat("CV  Error   :", round(mean(cv_errors), 4),
    "+/-", round(sd(cv_errors), 4), "\n")
```

```
## CV  Error   : 0.0603 +/- 0.0034
```

For sake of the run time I will hard code the stored rounded values.

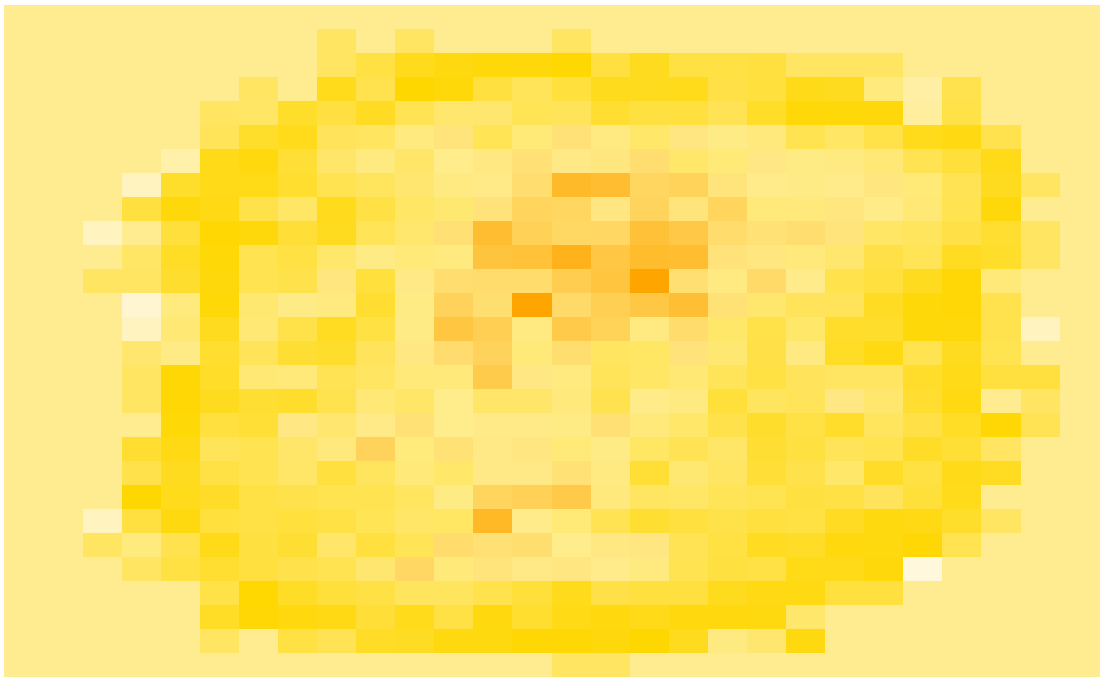
```
oob_error.1 <- 0.0528

CV_Error.1 <- sprintf("(%.4f, %.4f)", 0.0603 - 0.0034, 0.0603 + 0.0034)
```

E3.C.2d Analyze feature importance across ensemble methods.

```
# Random Forest importance
rf_imp     <- importance(rf_final, type = 1)    # MeanDecreaseAccuracy
rf_imp_vec <- rf_imp[, 1]

# GBM importance (top 20)
gbm_imp      <- summary(gbm_fit, n.trees = 1, plotit = FALSE)
gbm_top20    <- gbm_imp[1:20, ]

dt_imp_vec <- imp_map
```

As we can see from the RF feature importance map, the darker orange pixels are where the variance of the pixel occurs in the data set the most. This confirms with our intuition from looking at the data set at first where the writing is centered with the pixels of the edges be nearly all 0. In other words, the darker orange regions in the center of the heatmap represent the pixels with the highest importance scores identified by the random forest model.

```
# Plot RF importance as heatmap
rf_imp_map <- matrix(rf_imp_vec, 28, 28, byrow = TRUE)
rf_imp_map <- apply(rf_imp_map, 2, rev)
par(mar = c(2, 2, 3, 2))
image(t(rf_imp_map), col = orange_palette, axes = FALSE,
      main = "Random Forest: Feature Importance Map")
```



3. Interpretation

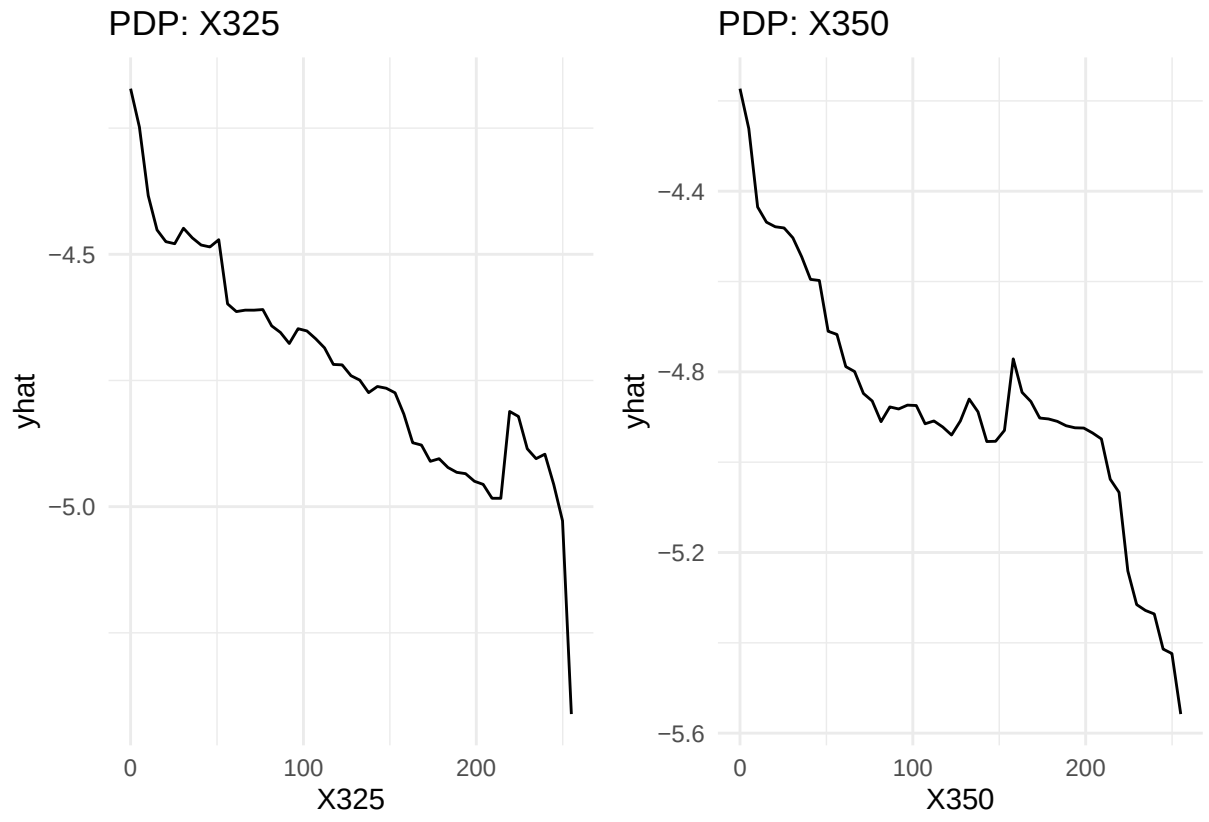
E3.C.3a Create partial dependence plots for important pixels.

In essence, The `yhat` value is the average predicted log-odds of the target class when we force a specific pixel in this case X325 and X350 to a specific intensity, while keeping all other pixels at their true, observed values across the entire dataset. Note that f is the log-odds of the target class from the random forest model prediction function.

$$\hat{y} = \hat{f}_S(x_S) = \frac{1}{N}\sum_{i=1}^{N} f(x_S, x_C^{(i)})$$

```
top2_pixels <- names(sort(rf_imp_vec, decreasing = TRUE))[1:2]

pd1 <- partial(rf_final, pred.var = top2_pixels[1], train = X_tr)
pd2 <- partial(rf_final, pred.var = top2_pixels[2], train = X_tr)
```

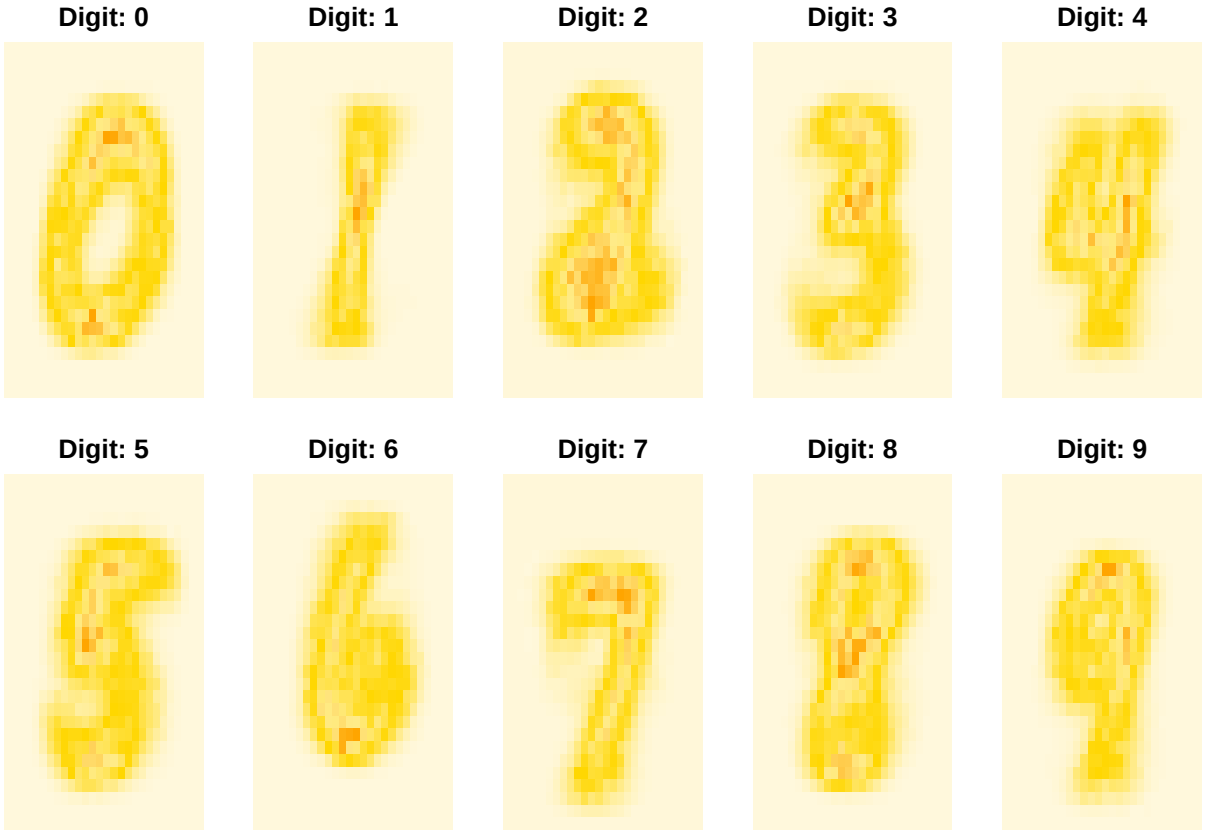
```
p1 <- autoplot(pd1) +
  labs(title = paste("PDP:", top2_pixels[1])) + theme_minimal()
p2 <- autoplot(pd2) +
  labs(title = paste("PDP:", top2_pixels[2])) + theme_minimal()
p1 + p2
```



E3.C.3b Visualize what the tree "sees" when classifying different digits.

```
par(mfrow = c(2, 5), mar = c(1, 1, 2, 1))
for (digit in 0:9) {
```

```r
  digit_idx <- which(as.integer(as.character(y_tr)) == digit)
  avg_imp   <- colMeans(X_tr[digit_idx, ]) * rf_imp_vec
  imp_m     <- matrix(avg_imp, 28, 28, byrow = TRUE)
  imp_m     <- apply(imp_m, 2, rev)
  image(t(imp_m), col = orange_palette, axes = FALSE,
        main = paste("Digit:", digit))
}
```



E3.C.3c Compare interpretability with SVM models.

In a single decision tree we are able to read of the splits of the tree based on specific pixel values. Although we improve accuracy in our prediction with ensemble learning from RF, we also lose the ability to have a single path for interpretability like we did for a single decision tree. However, Random Forests still offer valuable global interpretability through feature importance measures and partial dependence plots, allowing us to see which specific regions of the image drive the ensemble's aggregate decisions. In contrast, Support Vector Machines—particularly those using non-linear kernels like RBF or polynomial operate more like a "blackbox", it operates in a high-dimensional, mathematically transformed feature space. This makes it nearly impossible to trace a classification back to individual pixel intensities or establish intuitive decision rules.

Part D. Advanced Analysis and Comparison

1. Performance Deep Dive

E3.D.1a Compare all methods on full test set (10,000 samples).

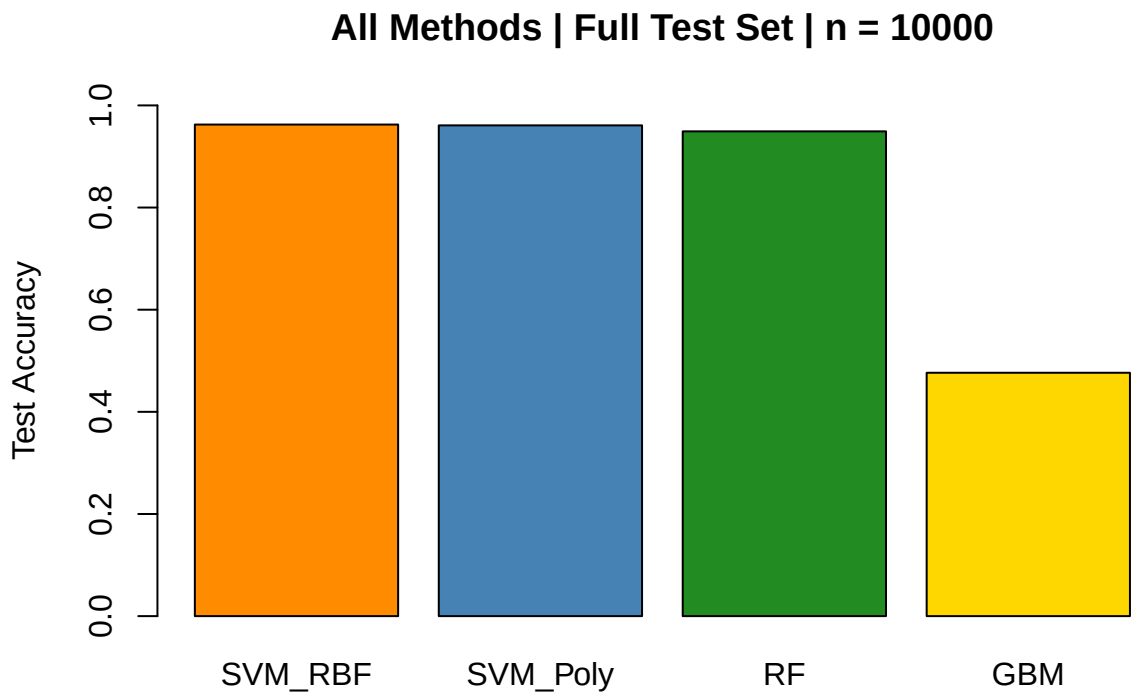
```r
y_test_full  <- as.character(df_test$label)
X_test_full  <- as.matrix(df_test[, -785])
pc_test_full <- predict(pc, newdata = df_test[, -785])[, 1:num_pcs]
```

```r
acc_all <- c(
  SVM_RBF  = mean(as.character(predict(svm_tuned, pc_test_full)) == y_test_full),
  SVM_Poly = mean(as.character(predict(svm_poly,  pc_test_full)) == y_test_full),
  RF       = mean(as.character(predict(rf_final,  X_test_full))  == y_test_full),
  GBM      = mean(predict(gbm_fit, data.frame(X_test_full),
                          n.trees = 1, type = "response") |>
                   (\(p) as.character(apply(p, 1, which.max) - 1))() == y_test_full)
)
```

```r
print(round(acc_all, 4))
```

```
##  SVM_RBF SVM_Poly       RF      GBM
##   0.9625   0.9609   0.9492   0.4765
```

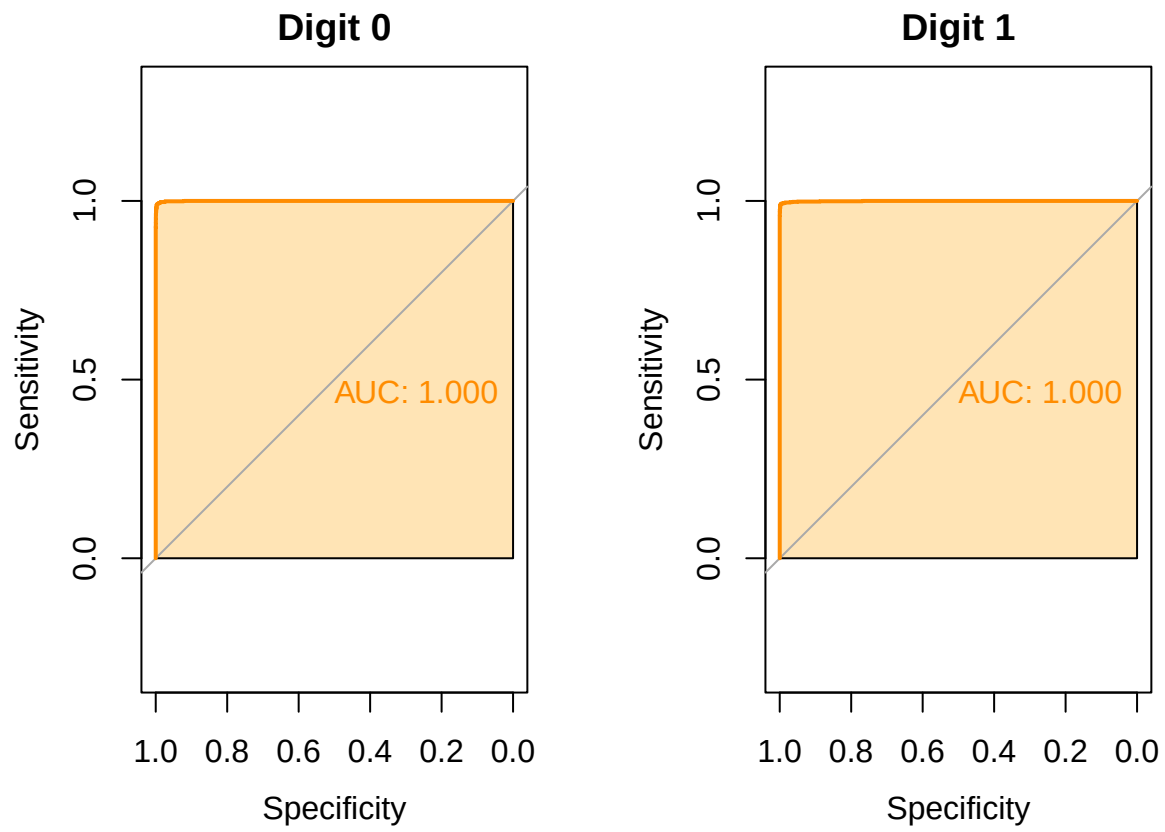
```r
barplot(acc_all, col = c("darkorange","steelblue","forestgreen","gold"),
        ylim = c(0, 1), ylab = "Test Accuracy",
        main  = "All Methods | Full Test Set | n = 10000")
```

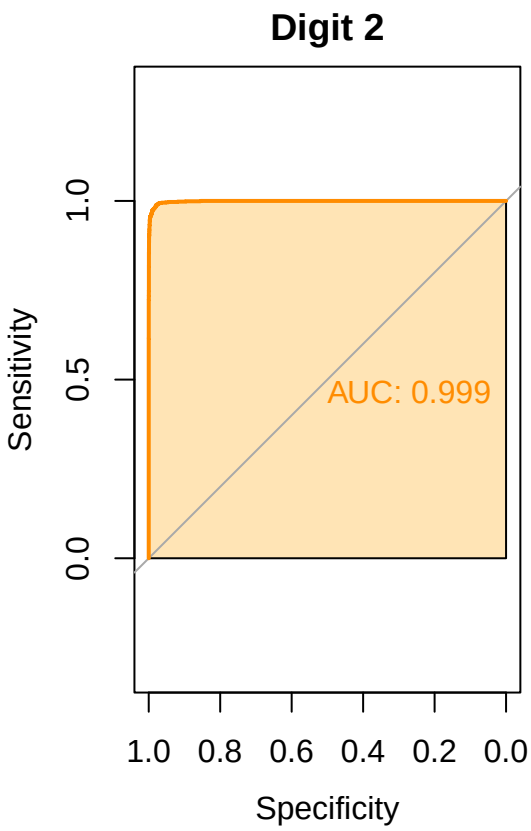


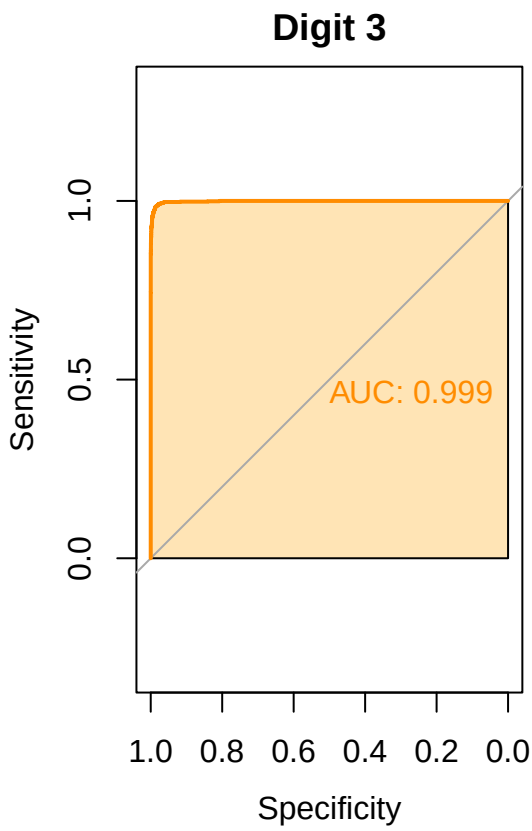
It is clear that from the bar graph we have the order of performance SVM_RBF > SVM_Poly> RF > GBM **E3.D.1b** Create precision-recall curves for each class.

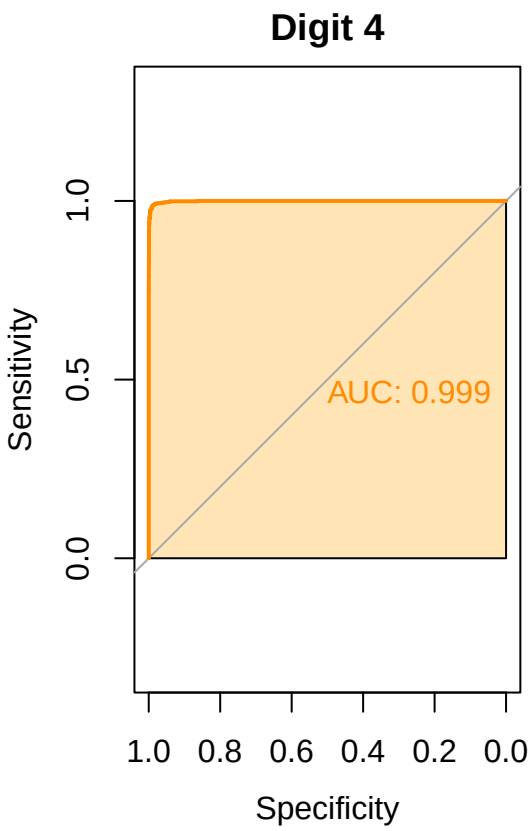
```r
# Use SVM tuned probabilities as example
svm_prob <- svm(pc_tr, y_tr, kernel = "radial",
                cost = best_cost, gamma = best_gamma,
                probability = TRUE)
probs <- attr(predict(svm_prob, pc_test_full, probability = TRUE),
              "probabilities")
```

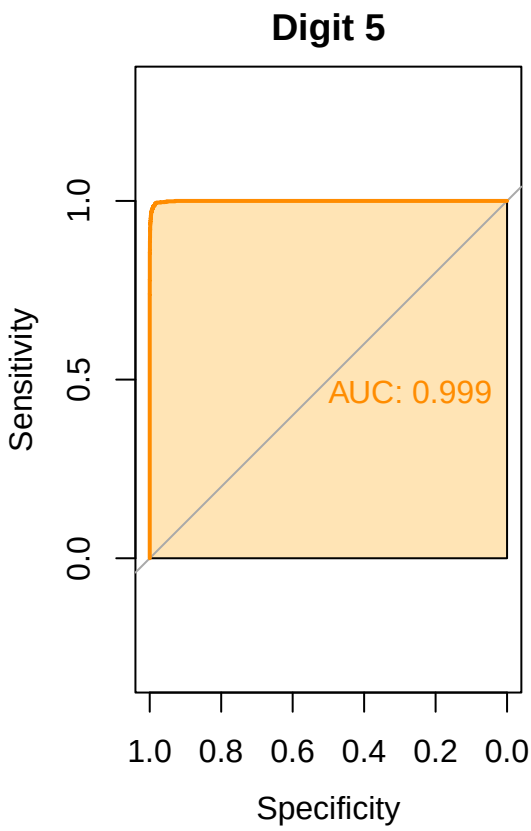
```r
library(pROC)
roc_list <- lapply(0:9, function(d) {
  roc(as.integer(y_test_full == d),
      probs[, as.character(d)], quiet = TRUE)
})

par(mfrow = c(1, 2), mar = c(3, 3, 2, 1))

for (d in 0:9) {
  plot(roc_list[[d+1]], main = paste("Digit", d),
       col = "darkorange", lwd = 2, print.auc = TRUE,
       auc.polygon = TRUE, auc.polygon.col = "moccasin")
}
```

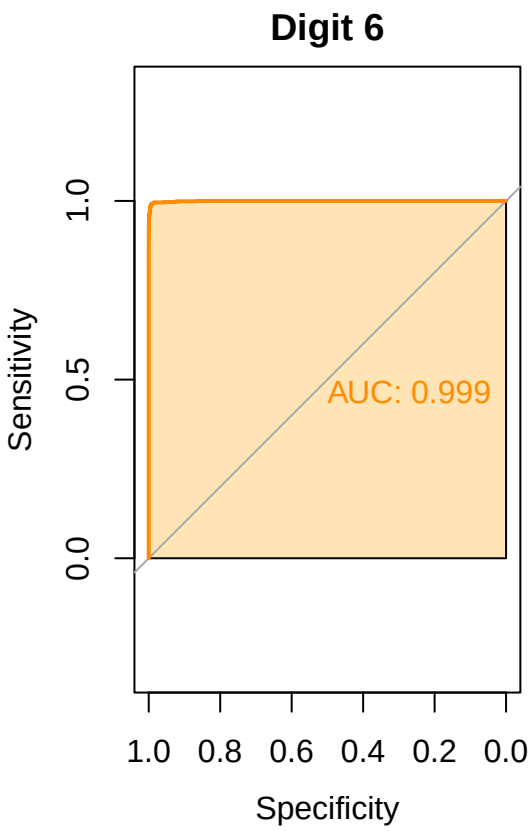


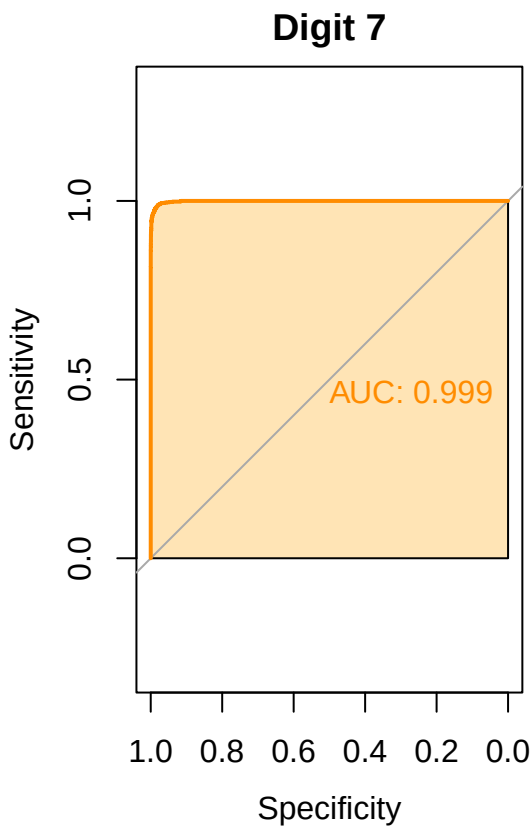

Digit 2
Sensitivity
1.0
0.5
0.0
AUC: 0.999
1.0 0.8 0.6 0.4 0.2 0.0
Specificity

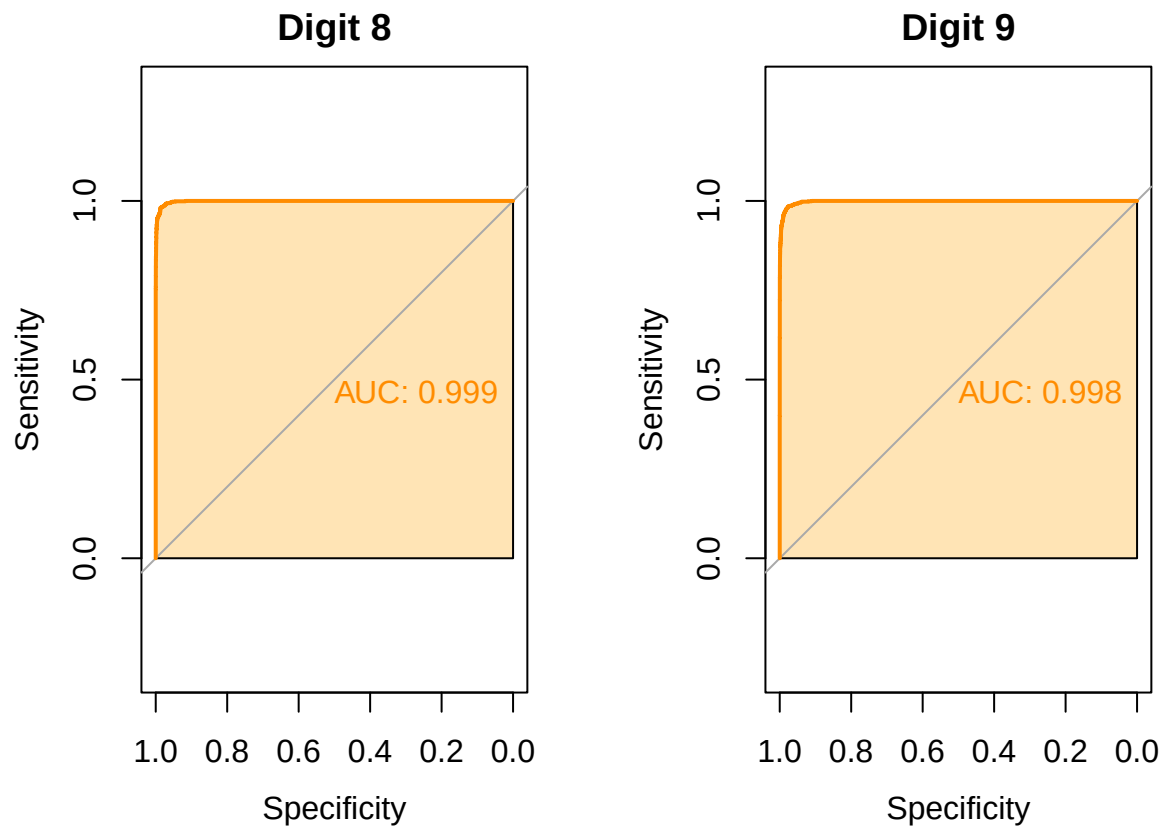

Digit 3
Sensitivity
1.0
0.5
0.0
AUC: 0.999
1.0 0.8 0.6 0.4 0.2 0.0
Specificity


Digit 4
Sensitivity
1.0
0.5
0.0
AUC: 0.999
1.0 0.8 0.6 0.4 0.2 0.0
Specificity


Digit 5
Sensitivity
1.0
0.5
0.0
AUC: 0.999
1.0 0.8 0.6 0.4 0.2 0.0
Specificity


Digit 6
Sensitivity
1.0
0.5
0.0
AUC: 0.999
1.0 0.8 0.6 0.4 0.2 0.0
Specificity


Digit 7
Sensitivity
1.0
0.5
0.0
AUC: 0.999
1.0 0.8 0.6 0.4 0.2 0.0
Specificity



E3.D.1c Analyze which digits are most challenging for each method.

We choose to compute the `F1` score as the harmonic mean of precision and recall.

$$F1 = \frac{2TP}{2TP + FP + FN}$$

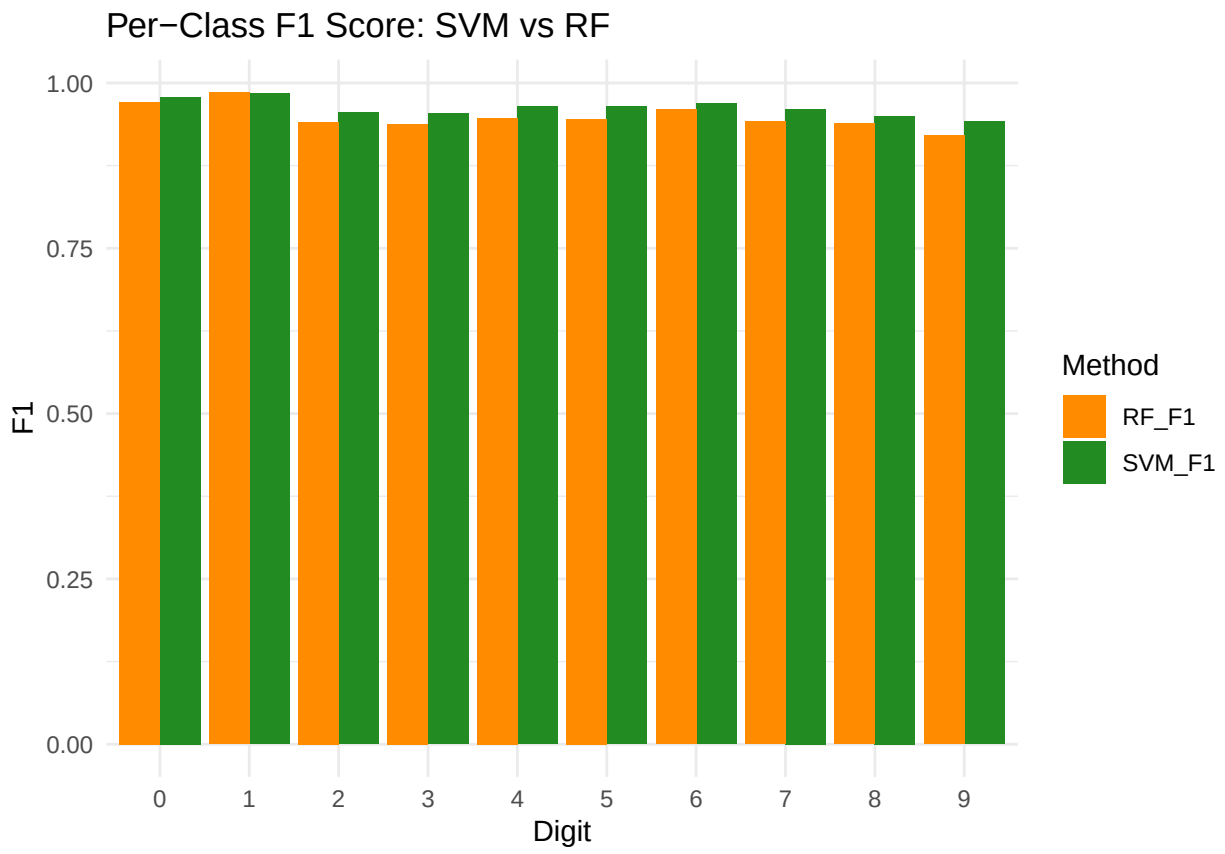
```
y_test_full <- factor(y_test_full, levels = 0:9)


pred_svm <- factor(predict(svm_tuned, pc_test_full), levels = 0:9)
pred_rf  <- factor(predict(rf_final, X_test_full), levels = 0:9)


cm_svm <- confusionMatrix(pred_svm, y_test_full)
cm_rf  <- confusionMatrix(pred_rf,  y_test_full)


per_class <- data.frame(
  Digit    = 0:9,
  SVM_F1   = cm_svm$byClass[, "F1"],
  RF_F1    = cm_rf$byClass[,  "F1"]
)
print(per_class)
```

```
##          Digit    SVM_F1     RF_F1
## Class: 0     0 0.9787018 0.9713135
```

```
## Class: 1     1 0.9842244 0.9859402
## Class: 2     2 0.9555556 0.9406615
## Class: 3     3 0.9540741 0.9382470
## Class: 4     4 0.9640506 0.9471545
## Class: 5     5 0.9640045 0.9455973
## Class: 6     6 0.9693506 0.9595016
## Class: 7     7 0.9607843 0.9421079
## Class: 8     8 0.9501285 0.9388601
## Class: 9     9 0.9416499 0.9202934
```

Plot:

```
per_class_long <- tidyr::pivot_longer(per_class, -Digit,
                                      names_to = "Method", values_to = "F1")
ggplot(per_class_long, aes(x = factor(Digit), y = F1, fill = Method)) +
  geom_col(position = "dodge") +
  scale_fill_manual(values = c("darkorange", "forestgreen")) +
  labs(title = "Per-Class F1 Score: SVM vs RF", x = "Digit", y = "F1") +
  theme_minimal()
```



From this we can see classification with class 9 seems to be the hardest task as the F1 score is the lowest for both SVM and RF. (Class: 9, 0.9416499, 0.9202934)

2. Computational Analysis

E3.D.2a Compare training time and memory usage.

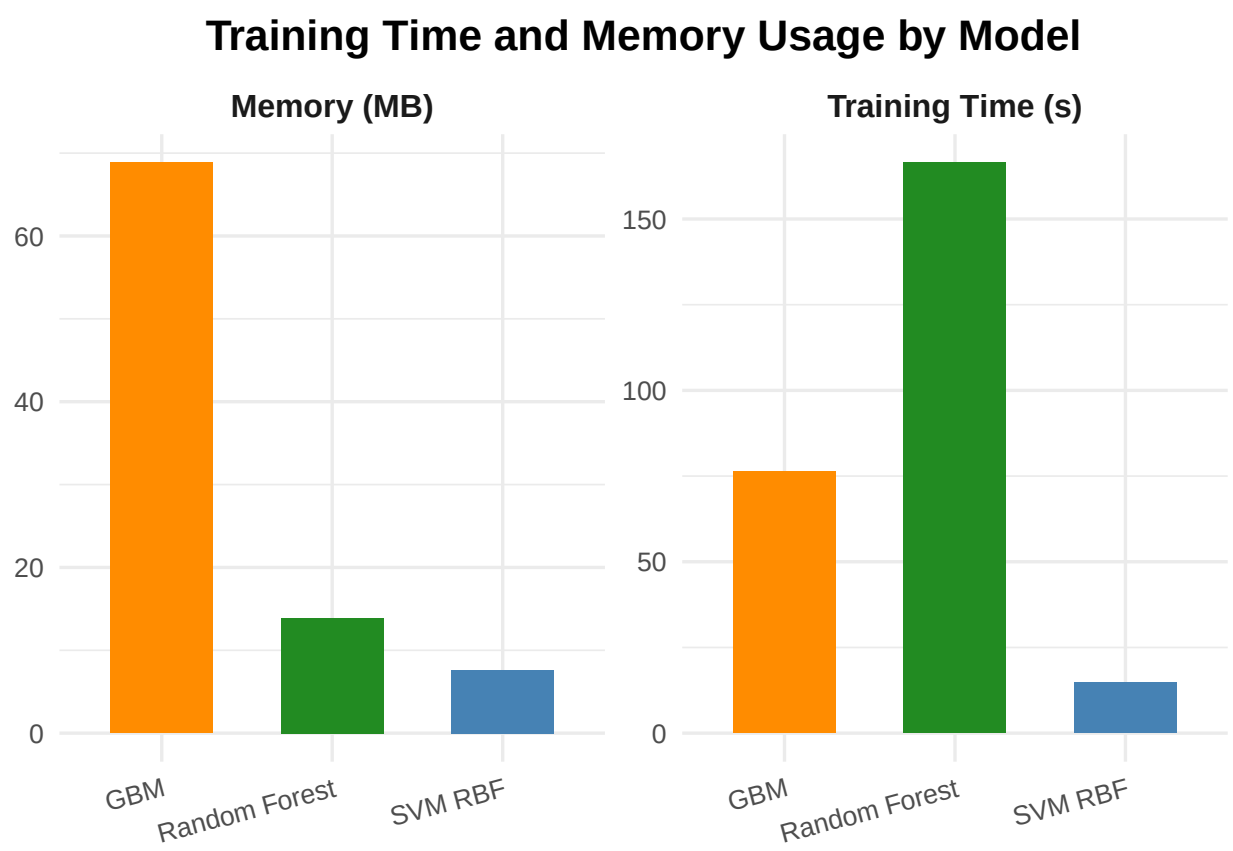
```r
# Training time
time_svm <- system.time(
  svm_tuned <- svm(pc_tr, y_tr, kernel = "radial",
                   cost = best_cost, gamma = best_gamma)
)[3]

time_rf <- system.time(
  rf_final <- randomForest(x = X_tr, y = y_tr,
                           ntree = best_ntree, mtry = 28,
                           importance = TRUE)
)[3]

time_gbm <- system.time(
  gbm_fit <- gbm(label ~ .,
                 data              = df_gbm_tr,
                 distribution      = "multinomial",
                 n.trees           = 100,
                 interaction.depth = 2,
                 shrinkage         = 0.1,
                 bag.fraction      = 0.5,
                 train.fraction    = 0.8,
                 verbose           = FALSE)
)[3]

# Memory
mem_svm <- as.numeric(object.size(svm_tuned), units = "MB") /2^20
mem_rf  <- as.numeric(object.size(rf_final),  units = "MB") /2^20
mem_gbm <- as.numeric(object.size(gbm_fit),   units = "MB") /2^20
```

```r
# Build comparison dataframe
comp_df <- data.frame(
  Model  = rep(c("SVM RBF", "Random Forest", "GBM"), 2),
  Metric = c(rep("Training Time (s)", 3), rep("Memory (MB)", 3)),
  Value  = c(time_svm, time_rf, time_gbm,
             mem_svm,  mem_rf,  mem_gbm)
)


comp_df #just checking
```

```
##           Model            Metric      Value
## 1       SVM RBF Training Time (s)  14.770000
## 2 Random Forest Training Time (s) 166.420000
## 3           GBM Training Time (s)  76.420000
## 4       SVM RBF       Memory (MB)   7.611076
## 5 Random Forest       Memory (MB)  13.886230
## 6           GBM       Memory (MB)  68.839645
```

```r
ggplot(comp_df, aes(x = Model, y = Value, fill = Model)) +
  geom_col(width = 0.6) +
  facet_wrap(~ Metric, scales = "free_y") +
  scale_fill_manual(values = c("darkorange", "forestgreen", "steelblue")) +
  labs(title = "Training Time and Memory Usage by Model",
```

```
       x = NULL, y = NULL) +
  theme_minimal(base_size = 13) +
  theme(legend.position  = "none",
        strip.text       = element_text(face = "bold", size = 12),
        axis.text.x      = element_text(angle = 15, hjust = 1),
        plot.title       = element_text(hjust = 0.5, face = "bold"))
```

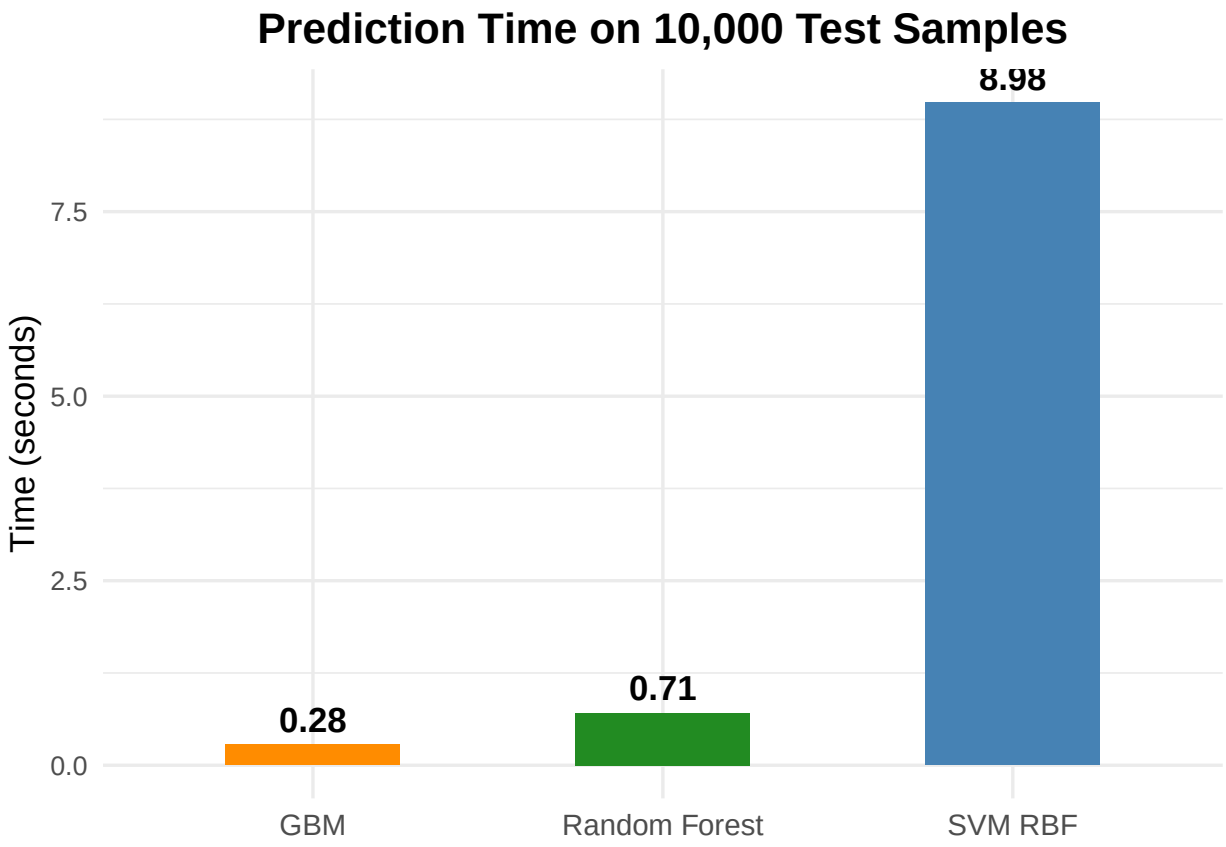


E3.D.2b Analyze prediction speed on test set.

```
times_pred <- c(
  SVM  = system.time(predict(svm_tuned, pc_test_full))[3],
  RF   = system.time(predict(rf_final,  X_test_full))[3],
  GBM  = system.time(predict(gbm_fit, data.frame(X_test_full),
                             n.trees = 1))[3]
)

tp.df <- data.frame(times_pred)

tp.df$Model  <- c("SVM RBF", "Random Forest", "GBM")
tp.df$Metric <- "Prediction Time (s)"
colnames(tp.df)[1] <- "Value"
```

```
ggplot(tp.df, aes(x = Model, y = Value, fill = Model)) +
  geom_col(width = 0.5) +
  geom_text(aes(label = round(Value, 3)),
```

```
          vjust = -0.5, fontface = "bold", size = 4.5) +
scale_fill_manual(values = c("darkorange", "forestgreen", "steelblue")) +
labs(title = "Prediction Time on 10,000 Test Samples",
     x = NULL, y = "Time (seconds)") +
theme_minimal(base_size = 13) +
theme(legend.position = "none",
      axis.text.x     = element_text(size = 11),
      plot.title      = element_text(hjust = 0.5, face = "bold"))
```



E3.D.2c Discuss scalability to full dataset (60,000 training samples).

SVM RBF trained in 9.95s on 8,000 samples. Since its quadratic program has a complexity of $O(n^2p)$ to $O(n^3)$, a 7.5× increase in n projects training time to between $9.95 \times 7.5^2 \approx 560$s and $9.95 \times 7.5^3 \approx 4200$s. Furthermore, the number of support vectors—and therefore prediction cost—grows with n. This means the already slow prediction time of 3.96s on 10,000 samples would worsen considerably. SVM is therefore the least scalable of the three methods.

Random Forest trained in 102.72s on 8,000 samples. Since tree building scales approximately as $O(n \cdot \text{ntree} \cdot \sqrt{p} \cdot \log n)$, a 7.5× increase in n projects to roughly $102.72 \times 7.5 \approx 770$s, which is tractable. Crucially, prediction time remains fast at 0.39s on 10,000 samples and will not degrade with more training data since tree depth is bounded. A potential concern is memory, as storing all trees simultaneously in RAM can be an issue, though this can be managed by reducing `ntree` if needed. Random Forest is therefore the most practical choice for the full dataset.

GBM completed training in 59.45s on 8,000 samples, projecting to roughly $59.45 \times 7.5 \approx 446$s at 60,000 samples under linear scaling. However, the current `gbm` implementation of `distribution = "multinomial"` is flagged as broken in the package documentation; it produced degenerate early stopping at iteration 1 and yielded a test accuracy of only 47.65%, which is barely above random for a 10-class problem.

3. Hybrid Approaches

E3.D.3a Experiment with SVM on PCA-reduced features. This has been already computed in Part A.2c: `svm_raw` trained on top-87 PCs, but if we extend to the full data set we get a accuracy of 0.9625 for SVM on 87 PCs.

```
pred_pca_svm  <- predict(svm_tuned, pc_test_full)
acc_pca_svm   <- mean(pred_pca_svm == y_test_full)
cat("SVM on PCA (87 PCs) full-test accuracy:", round(acc_pca_svm, 4), "\n")
```

```
## SVM on PCA (87 PCs) full-test accuracy: 0.9625
```

E3.D.3b Try tree-based methods on raw pixels vs. engineered features. RF on raw pixels : 0.9492 RF on derived features : 0.2634

```
rf_feat <- randomForest(x = ft_tr, y = y_tr,
                        ntree = best_ntree, mtry = 3)
pred_rf_feat <- predict(rf_feat, feat_test)
acc_rf_feat  <- mean(pred_rf_feat == y_test_full)
acc_rf_raw   <- mean(predict(rf_final, X_test_full) == y_test_full)

cat("RF on raw pixels         :", round(acc_rf_raw,  4), "\n")
```

```
## RF on raw pixels         : 0.9494
```

```
cat("RF on derived features   :", round(acc_rf_feat, 4), "\n")
```

```
## RF on derived features   : 0.2681
```

E3.D.3c Propose and test one innovative hybrid approach.

Our Idea is to use the top 87 Principal Components in a Random Forest, so that PCA can help reduce the sparsity and Random Forest can handle the nonlinearity in the data.

```
pc_tr_df  <- data.frame(pc_tr,  label = y_tr)
pc_val_df <- data.frame(pc_val, label = y_val)

tic("Hybrid PCA + RF")
rf_pca <- randomForest(label ~ ., data = pc_tr_df,
                       ntree = best_ntree, mtry = 9,
                       importance = TRUE)
toc()
```

```
## Hybrid PCA + RF: 23.92 sec elapsed
```

```
pred_hybrid <- predict(rf_pca, data.frame(pc_test_full))
acc_hybrid  <- mean(pred_hybrid == y_test_full)
```

Results:

```r
cat("Hybrid PCA+RF test accuracy:", round(acc_hybrid, 4), "\n")
```

```
## Hybrid PCA+RF test accuracy: 0.9271
```

```r
# Final comparison table
final_comp <- data.frame(
  Method   = c("SVM RBF (PCA)", "SVM Poly (PCA)",
               "RF (raw)", "GBM (raw)",
               "RF (engineered)", "Hybrid PCA+RF"),
  Test_Acc = round(c(acc_pca_svm, acc_all["SVM_Poly"],
                     acc_all["RF"], acc_all["GBM"],
                     acc_rf_feat,  acc_hybrid), 4)
)


final_comp
```

```
##            Method Test_Acc
## 1   SVM RBF (PCA)   0.9625
## 2  SVM Poly (PCA)   0.9609
## 3        RF (raw)   0.9492
## 4       GBM (raw)   0.4765
## 5 RF (engineered)   0.2681
## 6   Hybrid PCA+RF   0.9271
```

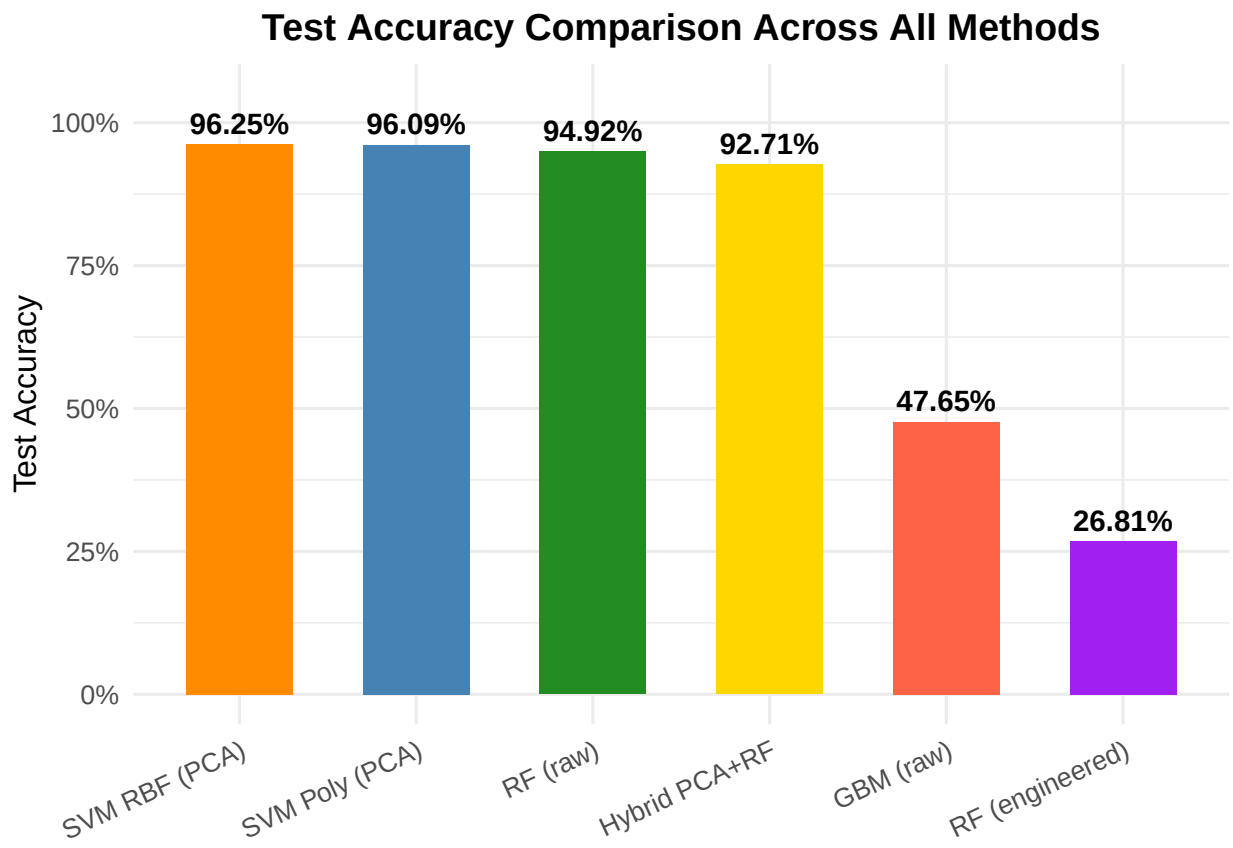
Visualize our approach

```r
final_comp_ordered <- final_comp[order(-final_comp$Test_Acc), ]
final_comp_ordered$Method <- factor(final_comp_ordered$Method,
                                    levels = final_comp_ordered$Method)

final_comp_ordered
```

```
##            Method Test_Acc
## 1   SVM RBF (PCA)   0.9625
## 2  SVM Poly (PCA)   0.9609
## 3        RF (raw)   0.9492
## 6   Hybrid PCA+RF   0.9271
## 4       GBM (raw)   0.4765
## 5 RF (engineered)   0.2681
```

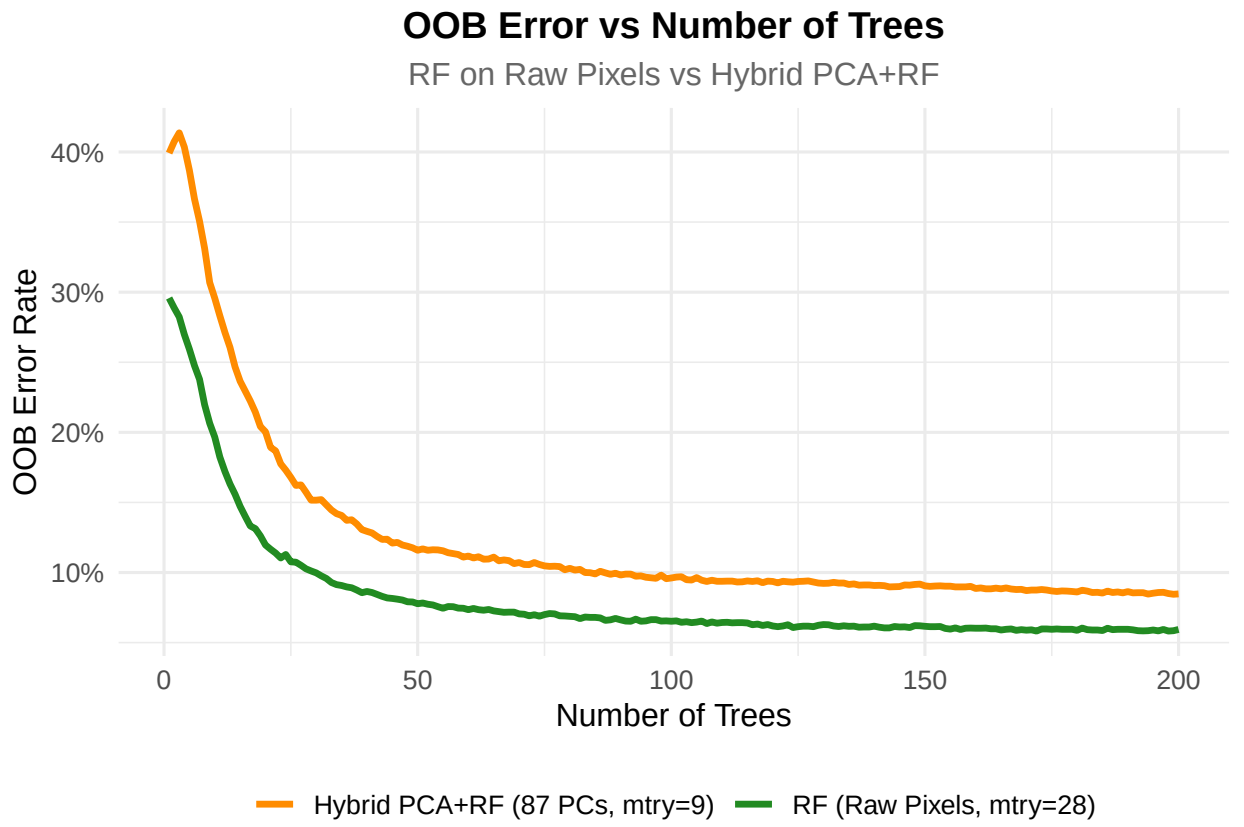
```r
ggplot(final_comp_ordered, aes(x = Method, y = Test_Acc, fill = Method)) +
  geom_col(width = 0.6) +
  geom_text(aes(label = scales::percent(Test_Acc, accuracy = 0.01)),
            vjust = -0.5, fontface = "bold", size = 3.8)  +
  scale_fill_manual(values = c("darkorange","steelblue","forestgreen",
                               "gold","tomato","purple")) +
  scale_y_continuous(limits = c(0, 1.05),
                     labels = scales::percent_format(accuracy = 1)) +
  labs(title    = "Test Accuracy Comparison Across All Methods",
```

```
       x = NULL, y = "Test Accuracy") +
  theme_minimal(base_size = 12) +
  theme(legend.position  = "none",
        axis.text.x      = element_text(angle = 25, hjust = 1, size = 10),
        plot.title       = element_text(hjust = 0.5, face = "bold"),
        plot.subtitle    = element_text(hjust = 0.5, color = "gray40"))
```



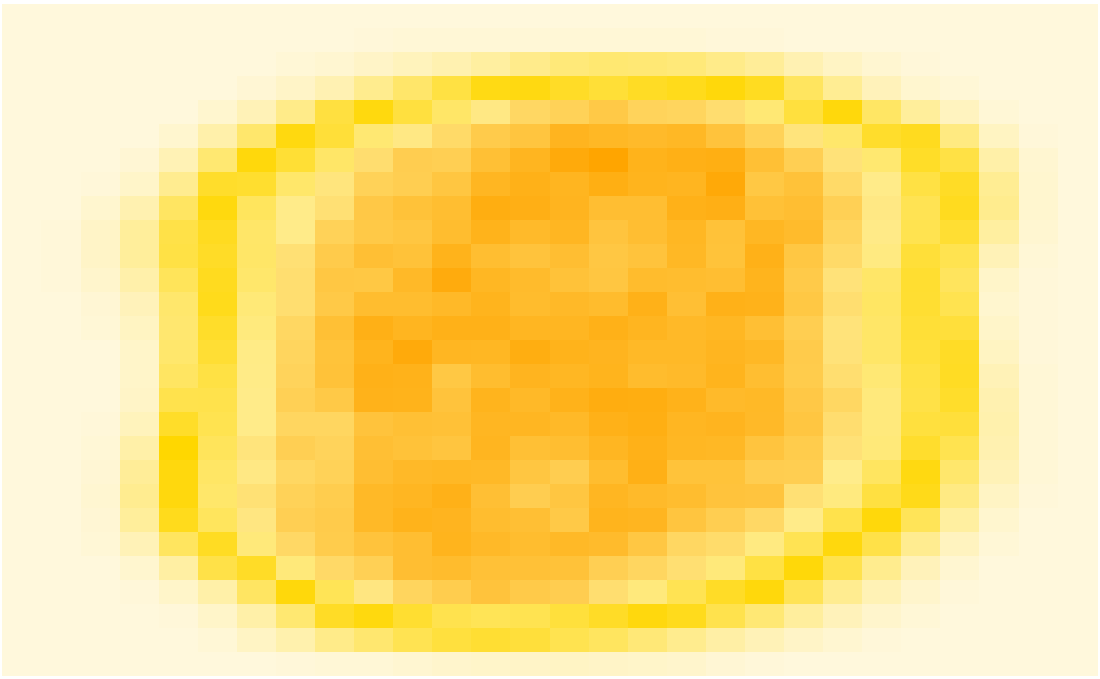

```
# 2. RF raw pixels vs Hybrid PCA+RF: OOB error across trees
rf_raw_oob <- randomForest(x = X_tr, y = y_tr,
                           ntree = best_ntree, mtry = 28)
rf_pca_oob <- randomForest(label ~ ., data = pc_tr_df,
                           ntree = best_ntree, mtry = 9)

oob_df <- data.frame(
  Trees  = 1:best_ntree,
  RF_Raw = rf_raw_oob$err.rate[, "OOB"],
  RF_PCA = rf_pca_oob$err.rate[, "OOB"]
)

oob_long <- tidyr::pivot_longer(oob_df, -Trees,
                                names_to  = "Method",
                                values_to = "OOB_Error")
```

```r
ggplot(oob_long, aes(x = Trees, y = OOB_Error, color = Method)) +
  geom_line(lwd = 1.2) +
  scale_color_manual(values = c("RF_Raw" = "forestgreen",
                                "RF_PCA" = "darkorange"),
                     labels = c("RF_Raw" = "RF (Raw Pixels, mtry=28)",
                                "RF_PCA" = "Hybrid PCA+RF (87 PCs, mtry=9)")) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +
  labs(title    = "OOB Error vs Number of Trees",
       subtitle = "RF on Raw Pixels vs Hybrid PCA+RF",
       x = "Number of Trees", y = "OOB Error Rate",
       color = NULL) +
  theme_minimal(base_size = 12) +
  theme(legend.position  = "bottom",
        plot.title       = element_text(hjust = 0.5, face = "bold"),
        plot.subtitle    = element_text(hjust = 0.5, color = "gray40"))
```




```r
#heatmap
imp_pca_rf <- importance(rf_pca, type = 1)  # MeanDecreaseAccuracy per PC

# Weight each PCA rotation vector by its importance score
# pc$rotation is 784 x 784; we use only first 87 columns
pixel_imp <- as.vector(
  abs(pc$rotation[, 1:num_pcs]) %*% abs(imp_pca_rf[, 1])
)
```

```r
imp_map_hybrid <- matrix(pixel_imp, 28, 28, byrow = TRUE)
imp_map_hybrid <- apply(imp_map_hybrid, 2, rev)

par(mar = c(2, 2, 3, 2))
image(t(imp_map_hybrid),
      col  = orange_palette,
      axes = FALSE,
      main = "Hybrid PCA+RF: Pixel Importance Map")
```

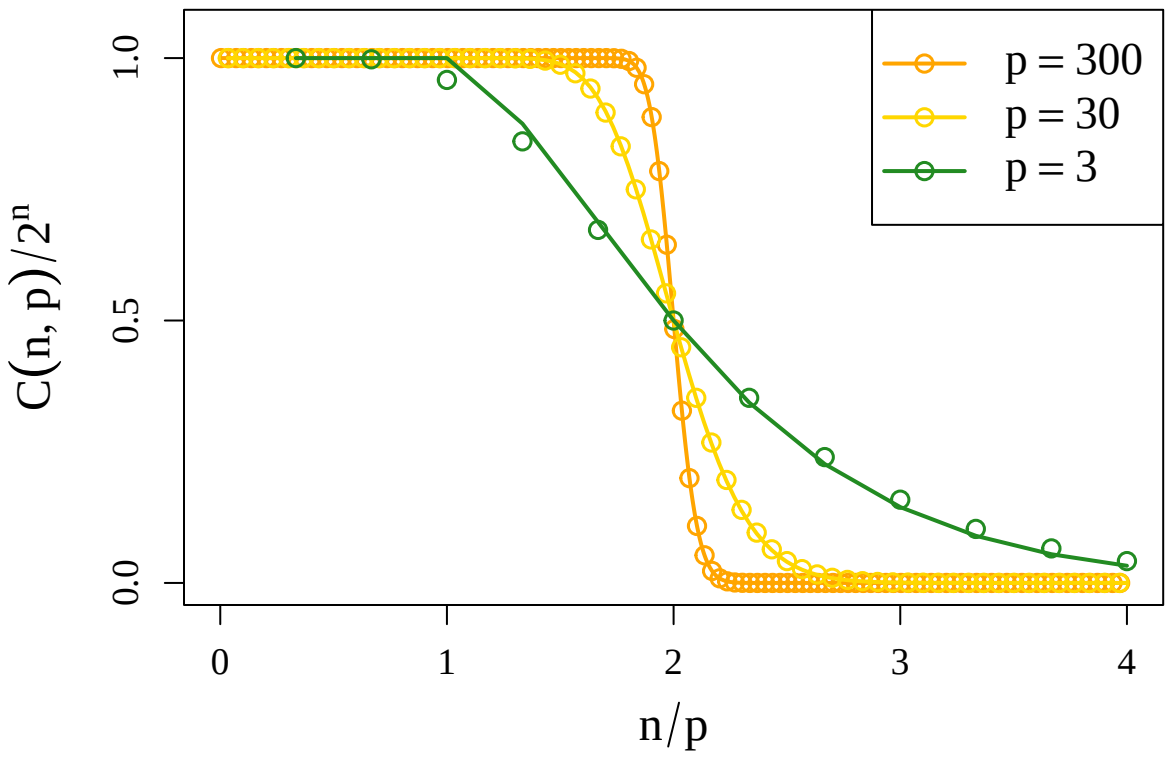


In the ranking histograms, we see that the Hybird PCA+RF achieves 92.75% test accuracy. However in the second plot it seems somewhat counterintutive as the PCA features should help out the earlier trees with convergence. We suppose that for the raw RF, each individual tree in the hybrid sees a smaller random subset of the already-compressed feature space, making single trees weaker but the ensemble more diverse. The raw pixel RF converges faster precisely because each tree has access to a richer random subset. The last plot has been consistent with the structure of the MNIST data, digit strokes are always centered in the 28×28 frame.

Extra Plot of Cover's Theorem Probability

```r
#cdf of the gaussian using error function
erf <- function(x) {
  return(2 * pnorm(x * sqrt(2)) - 1)
}
par(family = "serif", mar = c(5, 5, 2, 2), cex.axis = 1.2, cex.lab = 1.5)

plot(NULL, NULL, xlim = c(0, 4), ylim = c(0, 1.05),
```

```
     xlab = expression(n/p), ylab = expression(C(n,p)/2^n),
     xaxt = "n", yaxt = "n", bty = "o")

axis(1, at = 0:4)
axis(2, at = c(0, 0.5, 1))

p_vals <- c(300, 30, 3)
colors <- c("orange", "gold", "forestgreen")

point_steps <- c(10, 2, 1)

for (i in 1:3) {
  p <- p_vals[i]
  col <- colors[i]
  step <- point_steps[i]
  n_seq <- 1:(4 * p)
  x_seq <- n_seq / p
  y_exact <- pbinom(p - 1, n_seq - 1, 0.5)
  y_approx <- 0.5 * (1 + erf(p * sqrt(2/n_seq) - sqrt(n_seq/2)))
  lines(x_seq, y_exact, col = col, lwd = 2)
  idx <- seq(1, length(n_seq), by = step)
  points(x_seq[idx], y_approx[idx], col = col, pch = 1, cex = 1.2, lwd = 1.5)
}


legend("topright",
       legend = expression(p == 300, p == 30, p == 3),
       col = colors,
       lwd = 2,
       pch = 1,
       pt.lwd = 1.5,
       pt.cex = 1.2,
       cex = 1.4,
       bty = "o",
       bg = "white")
```


p = 300
p = 30
p = 3
1.0
0.5
0.0
C(n, p)/2^n
0
1
2
3
4
n/p